C# и .NET — методичка и практикум

От первой программы к разработке и сопровождению приложений

C# 14 · .NET 10 LTS
Редакция 2026-10-22

Последовательное изучение языка C# и платформы .NET. Теория, самостоятельные задания и сквозной проект. Пятый выпуск: главы 01–35, лабораторные L01–L04 и программа из 60 глав. Главы 36–60 пока представлены планом. Все написанные учебные страницы включены в эту книгу.

Оглавление

Как проходить практикум C# и .NET 4

01. C# и .NET, рабочая среда 6

02. Синтаксис и диагностика 9

03. Типы, числа и операции 12

04. Ветвления и циклы 15

05. Методы и декомпозиция 18

06. Строки, null и культура 21

07. Массивы и базовые коллекции 24

08. Классы и инварианты 26

09. Ошибки, ресурсы и первые файлы 29

10. Практикум ExpenseLedger CLI 32

11. Наследование и интерфейсы 34

12. Структуры, записи и равенство 37

13. Обобщения и контракты типов 40

14. Делегаты, лямбды и события 43

15. Перечисление и итераторы 46

16. LINQ в памяти и в запросах 49

17. Шаблоны и расширение синтаксиса 52

18. Практикум библиотеки предметной области 55

19. Сборки и модель исполнения 58

20. Память и время жизни 61

21. Атрибуты, reflection и dynamic 64

22. Структура исходников и инструменты компилятора 67

23. Ссылки, span и unsafe 70

24. Версии языка и чтение старого кода 73

25. Коллекции и алгоритмы BCL 76

26. Текст, время и регулярные выражения 79

27. Файлы, потоки и сериализация 82

28. HTTP и сетевые клиенты 85

29. Host, DI, настройки и журналы 88

30. Практикум утилиты импорта 91

31. Task и async await 94

32. Потоки и синхронизация 98

33. Параллельные конвейеры 101

34. Измерение и оптимизация 104

35. Практикум фонового обработчика 107

L01. Консольный журнал расходов 110

L02. Библиотека правил и отчётов 113

L03. Воспроизводимая утилита импорта 116

L04. Ограниченный фоновый конвейер 119

Подсказки и разбор контрольных вопросов 122

Карта покрытия языка C# 130

Источники и версии 132

Программа практикума C# и .NET 135

Как проходить практикум C# и .NET

Этот курс учит писать, проверять и сопровождать программы на C#. Начинать можно без другого языка программирования; достаточно уметь работать с файлами и терминалом. Первая цель — самостоятельно сделать консольный отчёт по расходам. Затем программа развивается в библиотеку, импортёр, сервис с базой данных и клиент выбранной специализации.

Что уже можно изучать

Подготовлены главы 01–35, 105 заданий, подсказки, запускаемые примеры и лабораторные L01–L04. Продолжение подробно расписано в программе; будущие главы отмечены как план. Это позволяет проходить материал последовательно и видеть, какие знания ещё предстоит получить.

Один учебный цикл

1. Прочитайте цель главы и попробуйте ответить на контрольные вопросы.
2. Перепечатайте небольшой пример и до запуска предскажите результат.
3. Измените одно условие: пустой ввод, границу диапазона, тип или порядок действий.
4. Решите A и B самостоятельно. В C переносите приём на изменившееся требование.
5. Запишите вход, ожидаемый и фактический результат. Объясните расхождение, если оно есть.
6. Сверьтесь с подсказками и повторите задачу без готового решения.

Удобный ритм занятия — 15–25 минут чтения, 40–60 минут кода, 10 минут разбора. Это ориентир для одного подхода, а не обещание освоить главу за час. Если не удаётся объяснить строку программы, уменьшите пример до нескольких инструкций.

Уровни практики

Уровень	Что требуется	Когда задача решена
A	Применить один приём по образцу	Результат совпал, каждая строка объяснима
B	Самому выбрать и связать несколько приёмов	Пройдены обычные и граничные случаи
C	Изменить требования или диагностировать дефект	Старое поведение сохранено, новое проверено
L	Собрать законченный этап проекта	Выполнен весь контракт, другой человек может запустить программу

Для B и C проверяйте хотя бы обычный случай, нижнюю и верхнюю границы, ошибочный ввод. Для коллекций добавляйте пустой набор и дубликат; для файлов — отсутствие и повреждение; для async в главах 31–35 — отмену и частичный сбой.

Рабочие файлы

Главы находятся в `docs`, законченные примеры — в `examples`, условия лабораторных — в `labs`. Свои решения создавайте в `work/имя-задачи`; эта папка не включается в Git. Исходный `ConsoleApp1` — отдельный учебный проект пользователя. Примеры курса собираются отдельными solution: языка и интеграций.

Команды в тексте выполняются из корня репозитория, если явно не сказано иначе. В Windows подходят PowerShell и терминал IDE. Пути в командах используют `/`, который понимает .NET. Перед изменением примера полезно сделать копию в `work`.

Быстрый маршрут при опыте другого языка

Прочитайте 01, выполните B и C из 03, 05–09 и сдайте L01. Если все критерии выполнены, можно переходить к системе типов. Особое внимание уделите `decimal`, nullable-аннотациям, копированию ссылок, `IDisposable` и культурно зависимому разбору строк: привычки другого языка здесь часто приводят к ошибкам.

Как пользоваться книгами

Книги из `Books` — дополнительное углубление. Для первых глав они не обязательны. Книга Рихтера полезна при изучении CLR, типов и ресурсов. Книги Freeman и Yan нужны для веб-раздела. Версии ASP.NET Core 6/8 в этих изданиях отличаются от платформы курса, поэтому команды и API сверяются с актуальной документацией. Названия файлов и назначение приведены в списке источников.

Проверка первого результата

После главы 10 вы должны без образца разобрать строку расхода, выбрать тип суммы, объяснить поведение общей ссылки на список, закрыть файл при ошибке и вернуть понятный код завершения. Если какой-то пункт вызывает затруднение, вернитесь к соответствующей главе. Готовность определяется этими действиями, а не количеством прочитанных страниц.

Начать главу 01

01. C# и .NET, рабочая среда

Результат: вы создаёте проект, собираете и запускаете его из терминала и понимаете, какой инструмент участвует на каждом шаге. Предыдущие знания: маршрут.

Язык, платформа и библиотека

C# задаёт правила исходного кода: какие значения существуют, как вызвать метод и как описать тип. Компилятор Roslyn проверяет код и обычно создаёт сборку с промежуточным кодом IL и метаданными. Среда CLR загружает сборку, управляет выполнением, исключениями и памятью. JIT обычно переводит IL нужных методов в машинный код во время работы. Позже мы разберём предварительную компиляцию, в том числе Native AOT: схема исполнения не всегда ограничивается JIT.

.NET включает среду выполнения и библиотеки. Например, `if` — конструкция языка, а `Console.WriteLine` — метод библиотеки. ASP.NET Core, EF Core и WPF решают прикладные задачи; они не являются частью синтаксиса C#.

```
Program.cs + .csproj
        ↓ dotnet build / Roslyn
сборка .dll с IL и метаданными
        ↓ dotnet run / CLR и JIT
выполняющаяся программа
```

SDK содержит средства разработки и сборки. Runtime нужен для выполнения уже собранного framework-dependent приложения. Наличие runtime не гарантирует, что можно собрать проект. Современный .NET работает на нескольких ОС; .NET Framework — отдельная Windows-платформа, важная при сопровождении старых систем.

Среда курса

Используем .NET 10 LTS и C# 14. Соответствие подтверждено в описании C# 14; срок поддержки .NET 10 до 14 ноября 2028 указан в политике .NET. Дата выбора среды: 22 октября 2026. Preview-функции для упражнений не нужны.

Проверьте установку:

```
dotnet --info
dotnet --list-sdks
```

Нужен SDK `10.0.x`. Если его нет, установите .NET 10 SDK. Редактор выбирайте по удобству: Visual Studio с поддержкой .NET 10, Rider или VS Code с C#-инструментами. Обязательный минимум курса воспроизводится CLI и не зависит от конкретной IDE.

Первая программа

Создайте новый проект в папке собственных решений:

```
dotnet new console -n HelloCourse -o work/HelloCourse --framework net10.0
dotnet build work/HelloCourse
dotnet run --project work/HelloCourse
```

В `Program.cs` замените содержимое:

```
Console.WriteLine("Начинаю изучать C#");
Console.WriteLine(7 + 5);
```

Получатся две строки: сообщение и 12. Одна инструкция заканчивается ;. Вызов метода указывает действие, аргумент в скобках — данные для него. Кавычки обозначают строковый литерал; без кавычек 7 + 5 вычисляется как сумма.

Проект содержит настройки, например:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
</Project>
```

TargetFramework (TFM) выбирает платформу и доступные API. Nullable включает анализ возможного null. ImplicitUsings подключает распространённые пространства имён: поэтому в первой строке не пришлось писать System.Console. LangVersion отдельно выбирает версию синтаксиса. Менять её на preview или latest для выполнения базовых заданий не требуется.

Что делают команды

Команда	Назначение
dotnet restore	Разрешить зависимости проекта и подготовить assets
dotnet build	Проверить и скомпилировать
dotnet run	Собрать при необходимости и запустить
dotnet test	Выполнить тестовые проекты через настроенный test runner
dotnet publish	Подготовить файлы для поставки приложения

Обычно build и run вызывают restore автоматически. Пакет — внешняя зависимость, namespace — группа имён, а assembly — единица сборки и загрузки. Это разные понятия. О поведении создания проектов см. dotnet new.

Solution объединяет несколько проектов для работы в IDE и CLI. Для курса используется examples/CSharpCourse.slnx. В bin находятся результаты сборки, в obj — промежуточные данные. Эти каталоги не редактируют как исходники.

Запуск готового примера

```
dotnet run --project examples/Foundations -- 01
```

-- отделяет параметры приложения от параметров dotnet run. Проверьте, что получены Hello, C#! и 12. Затем запустите пример из IDE с breakpoint на выводе суммы и выполните одну инструкцию за раз.

Практика

01-A. Создайте HelloCourse, выведите своё имя и результат 17 * 3. Критерий: две отдельные строки, вторая — 51; сборка без ошибок.

01-B. Создайте вторую программу Distance: скорость 72 км/ч, время 3 часа. Критерий: строка Distance: 216 km. Отдельно запустите build, затем run с --no-build. Объясните, что произойдёт, если изменить исходник и сразу повторить --no-build.

01-C. Подготовьте публикацию `Distance` командой `dotnet publish work/Distance -c Release -o work/publish-distance`. Запустите `dotnet work/publish-distance/Distance.dll`. Критерий: тот же результат, вы можете указать папку проекта и папку поставки. Это framework-dependent публикация: на другой машине потребуется подходящий runtime.

Проверка понимания

Почему наличие C#-файла недостаточно для запуска? Чем SDK отличается от runtime? Где определяется TFM? Почему установка пакета не равна `using`? Вернитесь к схеме и объясните её без чтения текста.

Подсказки · Далее 02

02. Синтаксис и диагностика

Результат: вы читаете короткую программу, находите область видимости переменной и различаете ошибку компиляции, исключение и неверный результат. Нужна глава 01.

Из чего состоит код

Идентификатор — имя: `total`, `Calculate`, `Expense`. Регистр существенен: `total` и `Total` различаются. Ключевые слова, например `int` и `return`, имеют заданный смысл. Встречающееся в чужом коде `@class` — идентификатор с экранированным ключевым словом. Для собственных имён выбирайте понятные слова: `expenseCount` точнее, чем `x1`.

Литерал задаёт значение прямо в коде: `42`, `"food"`, `true`. Выражение вычисляет значение: `price * count`. Инструкция выполняет действие: `Console.WriteLine(total);`. Блок в фигурных скобках объединяет инструкции.

```
int price = 25;
int count = 4;
int total = price * count;
Console.WriteLine(total); // 100
```

= присваивает значение, а == сравнивает. Комментарии `//` идут до конца строки; `/* ... */` могут занимать несколько строк. Поясняйте в комментариях причину решения: пересказ `count++` словами «увеличить count» редко помогает.

Переменные и область видимости

Объявление связывает имя с типом. До чтения локальной переменной компилятор должен доказать, что ей присвоено значение. Поля и элементы массивов имеют значения по умолчанию, но это не разрешает читать неинициализированную локальную переменную.

```
int total = 10;
{
    int bonus = 2;
    total += bonus;
}
Console.WriteLine(total); // 12
// Console.WriteLine(bonus); // ошибка: bonus вне области видимости
```

Область видимости определяет, где доступно имя; время жизни — как долго существует значение или объект. Эти понятия связаны, но не совпадают. В дальнейшем замыкание может удерживать объект после выхода из блока.

Точка входа

В новых консольных шаблонах используется top-level код. Компилятор создаёт точку входа. В проекте может быть только один файл с такими инструкциями; они идут перед объявлениями типов в этом файле. Вторую программу нельзя просто дописать в другой файл того же проекта.

Традиционная форма того же начала программы:

```
namespace Course;

internal class Program
{
    private static void Main(string[] args)
    {
        Console.WriteLine("Hello");
    }
}
```

Это альтернативный полный `Program.cs`, а не дополнение к top-level инструкциям. `namespace` группирует имена, `class` объявляет тип, `Main` — метод точки входа. `args` содержит аргументы командной строки. Значение `static` разберём в главах 05 и 08. `using System;` позволяет писать `Console` вместо `System.Console`.

Три вида дефектов

Вид	Пример	Как искать
Ошибка компиляции	Неизвестное имя, отсутствует `;`	Сначала первая содержательная диагностика build
Ошибка выполнения	Обращение за границы массива	Тип исключения, сообщение и стек вызовов
Логическая ошибка	Вместо скидки добавили наценку	Проверка на заранее рассчитанных данных

Например, `int total = "42";` не компилируется: строку нельзя неявно присвоить целому. Вычисление `10 / divisor` при `int divisor = 0` выбросит исключение во время выполнения. А `double average = 5 / 2;` выполнится и даст 2: деление сначала было целочисленным. Исправление — `5.0 / 2`, а не смена текста вывода.

Работа отладчиком

Поставьте breakpoint перед вычислением `total`. Запустите отладку. В Locals или Watch посмотрите `price` и `count`, затем выполните Step Over. Проверьте новое значение `total`. Step Into входит в вызываемый метод, Step Out завершает текущий. Call Stack показывает путь вызовов.

Запись ошибки должна содержать: входные данные, ожидаемый результат, фактический результат, место расхождения и причину. «Не работает» не даёт другому человеку воспроизводимого сценария. Если несколько ошибок возникают после одной пропущенной скобки, сначала исправьте её и пересоберите.

Практика

02-A. Запустите `dotnet run --project examples/Foundations -- 02`. До запуска вычислите значения из исходника. Критерий: результат `total=100`.

02-B. В своей копии по очереди внесите три дефекта: переименуйте `price` только в объявлении; поделите на переменную со значением ноль; замените `*` на `+`. Критерий: для каждого дефекта записаны категория, средство обнаружения и исправление. Не оставляйте намеренно сломанный код в проекте с другими упражнениями.

02-C. Поставьте условный breakpoint перед выводом: `total > 90`. Для `price=25, count=4` он должен сработать, для `price=10, count=4` — нет. Критерий: объясните состояние переменных в каждой точке, не добавляя диагностический вывод.

Проверка понимания

Можно ли прочитать локальную переменную без присваивания? Почему переменная внутри блока недоступна снаружи? Почему программа с логической ошибкой может успешно собраться? Чем top-level код отличается от `Main` для читателя?

Официальное описание: структура программы.

Подсказки · Далее 03

03. Типы, числа и операции

Результат: вы выбираете тип данных, предсказываете арифметику и проверяете переполнение. Нужна глава 02.

Тип задаёт допустимые действия

`int` хранит целое, `bool` — `true` или `false`, `string` — текст. Тип определяет диапазон, операции и правила преобразования. `int` — псевдоним `System.Int32`, а не другой тип. `var count = 3;` тоже объявляет `int`: компилятор вывел тип из выражения. Позже присвоить этой переменной строку нельзя. `var` не равен `dynamic`.

Тип	Свойства	Типичное применение
`sbyte/byte`	8 бит, со знаком/без знака	Компактные значения, байты
`short/ushort`	16 бит	Форматы и совместимость
`int/uint`	32 бита	Счётчики, целые значения
`long/ulong`	64 бита	Большие счётчики
`nint/nuint`	Размер платформенного указателя	Interop
`float/double`	Двоичная плавающая точка, около 6–9/15–17 значащих цифр	Измерения и расчёты
`decimal`	Десятичное представление, 28–29 значащих цифр	Суммы с десятичными долями
`char`	16-битная кодовая единица UTF-16	Работа с текстовым представлением
`bool`	Два логических значения	Условия

Для `int` диапазон от −2 147 483 648 до 2 147 483 647. Значения `MinValue` и `MaxValue` лучше использовать в проверках, чем перепечатывать границы. Выбор беззнакового типа сам по себе не валидирует бизнес-правило «количество положительное»: ноль по-прежнему допустим.

Литералы, константы и преобразования

```
int count = 1_000;
long population = 8_000_000_000L;
double ratio = 0.5;
float scale = 0.5f;
decimal price = 19.90m;
const int MaxItems = 100;
int empty = default; // 0
Console.WriteLine(price * 3); // числовое значение 59.70
```

Суффикс `m` создаёт `decimal`. Без него `19.90` — `double`. `const` требует вычислимое при компиляции значение. Понятие `readonly` относится к полям и будет рассмотрено отдельно.

Неявное преобразование `int → long` сохраняет значение. Обратное обычно требует явного cast: `int small = checked((int)large);`. Явность не гарантирует отсутствие потери данных. Даже некоторые неявные преобразования к floating-point теряют точность. Подробности: числовые преобразования.

`(int)3.9` даёт `3`, а `(int)(-3.9)` — `-3`: дробь отбрасывается к нулю. Это не правило округления денежных значений. Для округления используйте `Math.Round` и явно выберите правило, например `MidpointRounding.ToEven`.

Деление, округление и переполнение

```
Console.WriteLine(5 / 2);        // 2
Console.WriteLine(5 / 2.0);      // 2.5
Console.WriteLine(5 % 2);        // 1
Console.WriteLine(0.1 + 0.2 == 0.3);    // False
Console.WriteLine(0.1m + 0.2m == 0.3m); // True

int value = int.MaxValue;
int wrapped = unchecked(value + 1); // int.MinValue
// int failed = checked(value + 1); // OverflowException
```

Сначала выполняется операция с типами её операндов, затем присваивание результату. Поэтому `long total = count * price;` не спасает от переполнения умножения двух `int`. Исправление — `checked((long)count * price)`.

`decimal` удобен для десятичных величин, но тоже имеет конечную точность: `1m / 3m` не представляется точно. Валюта, число десятичных знаков и момент округления остаются правилами приложения. Операции `decimal` при выходе за диапазон выбрасывают исключение и внутри `unchecked`.

`checked` действует на находящиеся в нём операции, а не автоматически на тело вызванного метода. Переполнение целочисленного выражения, вычислимого при компиляции, обычно выявляется уже компилятором.

Логика, приоритет и биты

`&&` и `||` применяют короткое замыкание: правый операнд вычисляется только при необходимости. Выражение `denominator != 0 && numerator / denominator > 2` не делит на ноль. `&` и `|` над `bool` вычисляют обе стороны; над целыми они выполняют побитовые операции. `^` — XOR, а не степень; для степени служит `Math.Pow`.

`1 << 3` даёт 8. В `flags & 4` проверяется конкретный бит; это не логическое «и» двух условий. У `>>` и `>>>` различается заполнение старших битов для отрицательных чисел. Подробные приёмы понадобятся в главе о enum и битовых флагах.

В `2 + 3 * 4` умножение выполняется раньше сложения. Скобки `(2 + 3) * 4` меняют результат. Не смешивайте изменение переменной через `++` и её несколько чтений в одном сложном выражении: разнесённые инструкции проще проверить.

Разбор расчёта

Стоимость трёх товаров по `19.90m` равна `59.70m`. При скидке 10%:

```
decimal subtotal = 19.90m * 3;
decimal total = Math.Round(subtotal * 0.90m, 2, MidpointRounding.ToEven);
Console.WriteLine(total); // числовое значение 53.73
```

Текстовое представление точки или запятой зависит от культуры; глава 06 закрепит формат вывода. Запуск примера: `dotnet run --project examples/Foundations -- 03`.

Практика

03-A. До запуска предскажите `7 / 3`, `7 / 3m`, `-7 % 3`, `(int)2.9`. Критерий: объяснены тип и значение каждого результата, а не только скопирован вывод.

03-B. Рассчитайте заказ: цена `125.50m`, количество 4, скидка `15m` процентов. Округление итоговой суммы — два знака, ToEven. Критерий: `426.70`. Проверьте скидки 0 и 100, количество 1. Проверку неверных входов добавите после главы 04.

03-C. Для `int count = 50_000; int price = 50_000;` получите точное значение `2_500_000_000` типа `long`. Покажите вариант, где `checked` обнаруживает ошибку, и вариант, где преобразование до умножения её предотвращает.

Проверка понимания

Почему `var` сохраняет статическую типизацию? Почему `double` не подходит для точного равенства десятичных сумм? Чем округление отличается от cast? Почему изменение типа результата после операции может быть слишком поздним?

Подсказки · Далее 04

04. Ветвления и циклы

Результат: программа принимает решения, повторяет действия и заканчивает обработку при конце ввода. Нужна глава 03.

Условие и выбор

Условие `if` имеет тип `bool`. Число `1` не преобразуется в истину автоматически.

```
int age = 20;
if (age < 0)
{
    Console.WriteLine("Некорректный возраст");
}
else if (age < 18)
{
    Console.WriteLine("Несовершеннолетний");
}
else
{
    Console.WriteLine("Совершеннолетний");
}
```

Порядок веток существенен: выполняется первая подходящая. Для интервала от 1 до 100 включительно нужно `value >= 1 && value <= 100`; запись `1 <= value <= 100` неверна.

Оператор `switch` удобен для конечного набора команд. Выражение `switch` возвращает значение:

```
string command = "sum";
string description = command switch
{
    "sum" => "Сумма",
    "count" => "Количество",
    _ => "Неизвестная команда"
};
Console.WriteLine(description);
```

В `switch statement` ветка с инструкциями обычно заканчивается `break`, `return` или `throw`. Неявного проваливания в следующую непустую ветку нет. `_ =>` в switch expression — запасной случай. Развитые шаблоны будут в главе 17.

Цикл и его границы

`for` подходит, когда известен счётчик:

```
int total = 0;
for (int i = 1; i <= 5; i++)
{
    total += i;
}
Console.WriteLine(total); // 15
```

Перед каждой итерацией проверяется условие. После тела выполняется `i++`. Инвариант этого цикла: перед итерацией с номером `i` переменная total хранит сумму от 1 до `i - 1`. Это утверждение позволяет объяснить результат и найти ошибку границы.

`while` проверяет условие перед телом, `do ... while` — после, поэтому do выполняется хотя бы раз. `foreach` последовательно получает элементы коллекции: например, `foreach (char letter in "cat") Console.WriteLine(letter);`.

`break` завершает ближайший цикл, `continue` переходит к следующей итерации, `return` выходит из метода целиком. В цикле while перед continue нужно обеспечить продвижение: иначе значение, вызвавшее continue, может обрабатываться бесконечно. В чужом коде встречается `goto`; он переносит управление к разрешённой метке. Для обычного ввода понятнее цикл и return; специальные ограничения goto разбираются по справочнику.

Конец потока и пользовательская команда

`Console.ReadLine()` возвращает строку либо `null`, если поток закончился. Это не то же самое, что пустая строка после Enter. Обозначение `string?` допускает отсутствие строки; подробно оно рассматривается в главе 06.

Полный пример для собственного `Program.cs`:

```
long total = 0;
while (true)
{
    string? line = Console.ReadLine();
    if (line is null || line == "stop")
    {
        break;
    }

    if (!int.TryParse(line, out int value))
    {
        Console.WriteLine("error");
        continue;
    }

    total = checked(total + value);
}
Console.WriteLine(total);
```

`TryParse` возвращает успех и через `out` записывает число. Не пытайтесь использовать полученное число при `false`. Здесь текст `abc` — ожидаемый неверный ввод, поэтому он обрабатывается обычной веткой. Если гигантский поток переполнит `long`, checked выявит ошибку; стратегию обработки исключений изучим в главе 09.

Для входа `10`, `abc`, `-3`, `stop` вывод — `error`, затем `7`. Вводить строки нужно по отдельности. EOF можно проверить передачей файла стандартному вводу; при интерактивном вводе Windows обычно используют Ctrl+Z и Enter.

Проверка границ

Для задачи «сумма от 1 до n» проверьте `n=0`, `1`, `5`, `-1`. Заранее определите контракт: здесь ноль даёт ноль, отрицательное n — ошибку. Тест `n=5` один не выявит все ошибки. Условия `i < n` и `i <= n` отличаются ровно одним элементом, который может потеряться незаметно.

Практика

04-A. Напишите сумму от 1 до n через for. Вход задайте переменной. Критерии: 0 → 0, 1 → 1, 5 → 15, отрицательное число → понятное сообщение. Для результата используйте `long`.

04-B. Напишите приведённый сумматор по описанию, затем добавьте число успешно прочитанных значений. Критерии: вход `10/abc/-3/stop` даёт сумму 7, количество 2 и одно сообщение об ошибке; немедленный EOF — сумму 0 и количество 0. Знак `/` здесь разделяет строки, а не является вводом.

04-C. Ограничьте допустимое число диапазоном −1000…1000 и завершайте программу после трёх ошибочных строк подряд. Успешная строка сбрасывает счётчик ошибок. Критерии: `x/y/5/z/q/stop` не

останавливается раньше stop; `x/y/z/5` завершается до обработки 5.

Проверка понимания

Чем отличаются break и return? Когда тело while не выполнится ни разу? Почему пустая строка и EOF требуют разных решений? Как инвариант помогает проверить цикл?

Официальное описание: инструкции выбора и повторения.

Подсказки · Далее 05

05. Методы и декомпозиция

Результат: вы выделяете проверяемое вычисление из ввода-вывода и объясняете передачу аргументов. Нужна глава 04.

Контракт метода

Метод получает параметры, выполняет работу и возвращает результат. Контракт определяет допустимые входы и обещанный результат. Например, расчёт скидки принимает неотрицательную сумму и процент от 0 до 100. Пока вызывающая сторона проверяет эти условия через if.

Полный вариант в top-level `Program.cs`:

```
decimal result = CalculateTotal(125.50m, 4, 15m);
Console.WriteLine(result); // числовое значение 426.70

static decimal CalculateTotal(decimal price, int count, decimal discountPercent)
{
    decimal subtotal = price * count;
    decimal discounted = subtotal * (1 - discountPercent / 100m);
    return Math.Round(discounted, 2, MidpointRounding.ToEven);
}
```

В этом контексте объявлена локальная static-функция в сгенерированной точке входа. В классе похожее объявление является static-методом. `static` означает, что для вызова не нужен экземпляр класса; static локальная функция также не захватывает окружающие переменные.

Метод с типом результата `void` не возвращает значение. Оператор `return;` может досрочно завершить его. В методе с результатом все нормально завершающиеся пути должны вернуть значение.

Выделенное вычисление не читает Console и не пишет файл. Оно легче проверяется: одинаковые аргументы дают один результат. Вывод остаётся ответственностью вызывающего кода.

Параметр и аргумент

Параметр — имя в объявлении, аргумент — значение при вызове. Именованные аргументы делают понятнее несколько одинаково типизированных значений:

```
decimal total = CalculateTotal(price: 100m, count: 2, discountPercent: 10m);
```

Необязательный параметр задаёт значение по умолчанию в объявлении. Это значение подставляется при компиляции вызывающего кода: изменение default в библиотеке не обновляет уже скомпилированного клиента автоматически.

Перегрузка — несколько методов с одним именем и разными наборами параметров. Только возвращаемого типа для различения недостаточно. Для начальных упражнений предпочитайте разные понятные имена спорным перегрузкам.

`params` принимает переменное число аргументов. Классический пример:

```
static int Sum(params int[] values)
{
    int total = 0;
    foreach (int value in values)
    {
        total = checked(total + value);
    }
    return total;
}
```

`Sum(1, 2, 3)` вернёт 6, `Sum()` — 0. Массивы подробно разбираются в главе 07. Новые варианты params для коллекций изучаются после обобщений.

Передача по значению и по ссылке

По умолчанию копируется значение аргумента. Для int это число. Для ссылки на объект копируется ссылка; два имени могут указывать на один объект. Фраза «все объекты передаются по ссылке» скрывает важное различие между копированием ссылки и ref-параметром.

```
int number = 10;
Change(number);
Console.WriteLine(number); // 10
Increase(ref number);
Console.WriteLine(number); // 11

static void Change(int value) => value = 99;
static void Increase(ref int value) => value++;
```

`ref` даёт методу доступ к переменной вызывающей стороны. Она должна быть заранее инициализирована. `out` позволяет методу присвоить результат; он обязан присвоить его на всех нормально завершающихся путях. `in` предоставляет чтение аргумента по ссылке и не гарантирует отсутствие всех копий или ускорение. Начинайте с обычных параметров; модификаторы вводите ради понятного контракта.

Try-паттерн

Ожидаемо неверный текст удобно выражать результатом bool и out-значением:

```
static bool TryReadCount(string text, out int count)
{
    count = 0;
    if (!int.TryParse(text, out int parsed) || parsed < 1 || parsed > 100)
    {
        return false;
    }

    count = parsed;
    return true;
}
```

Вызывающий код сначала проверяет bool. Для `"12"` получит true и 12; для `"0"`, `"abc"`, `"101"` — false и 0. Парсинг ещё не означает валидацию: `int.TryParse("0", ...)` успешен, но правило количества запрещает ноль.

Рекурсия и размер метода

Рекурсивный метод вызывает себя и должен приближаться к базовому случаю. Для факториала `0! = 1`, а для n > 0 результат равен n × (n−1)!. Каждый незавершённый вызов требует места в стеке; очень глубокая рекурсия опасна. Для длинных линейных последовательностей цикл обычно проще. C# не обещает автоматическое устранение хвостовой рекурсии.

Хорошая граница метода — одна ответственность, которую можно назвать и проверить. Метод, который спрашивает путь, читает файл, считает скидки и печатает отчёт, связывает несколько независимых причин изменения.

Практика

05-A. Выделите `CalculateTotal` и вызовите его для трёх случаев: 100×2, скидка 10 → 180; 5×1, скидка 100 → 0; 0×3, скидка 0 → 0. Критерий: внутри метода нет Console.

05-B. Реализуйте `TryReadCount` без копирования. Критерии: `"1"` и `"100"` успешны, `"0"`, `"101"`, `"-2"`, `"x"` отклонены. При отказе out-параметр равен 0.

05-C. Создайте рекурсивный и циклический расчёт факториала с контрактом `0 <= n <= 20` и результатом long. Критерии: 0 → 1, 5 → 120, 20 → 2432902008176640000. Вход 21 отклоняется до вычисления. Объясните, почему checked не заменяет проверку контракта.

Проверка понимания

Почему изменение int-параметра не изменяет исходную переменную? Как отличается out от ref? Где проверять аргументы и где обрабатывать пользовательский текст? Почему метод без ввода-вывода проще тестировать?

Официальное описание параметров

Подсказки · Далее 06

06. Строки, null и культура

Результат: вы задаёте однозначный формат данных и обрабатываете отсутствие значения. Нужна глава 05.

Строка и символ

`string` — неизменяемая последовательность кодовых единиц UTF-16. Методы `Trim`, `Replace` и `ToUpperInvariant` возвращают строку; исходный объект не меняется.

```
string original = " food ";
string cleaned = original.Trim();
Console.WriteLine(original.Length); // 6
Console.WriteLine(cleaned.Length);  // 4
Console.WriteLine("\U0001F600".Length); // 2 кодовые единицы UTF-16, emoji
```

Один `char` не всегда соответствует одному видимому символу. `Rune` представляет Unicode scalar value, а составной видимый символ может включать несколько таких значений. Обрезание пользовательского текста по индексам char способно разрезать суррогатную пару. Для текстовых элементов существует `StringInfo`; практическая работа с Unicode будет в главе 26.

Запись и построение строк

```
string escaped = "first\nsecond";       // перевод строки
string path = @"C:\data\expenses.txt"; // обратные слэши буквально
string category = "food";
decimal amount = 12.50m;
string message = $"{category}: {amount:F2}";
string json = """{"category":"food","amount":12.50}""";
Console.WriteLine(message);
```

В строке с `$` выражения внутри фигурных скобок вычисляются. `:F2` задаёт два знака после десятичного разделителя, но не закрепляет культуру. Raw literal использует тройные кавычки; для многострочных raw strings важен отступ перед закрывающими кавычками.

Для нескольких фрагментов достаточно интерполяции. При накоплении большого текста в цикле `StringBuilder` позволяет уменьшить создание промежуточных строк. Не заменяйте все строки на StringBuilder без измерения и понятной причины.

Правила сравнения

Для команды, ключа протокола или категории из формата данных удобно явно выбрать:

```
bool same = string.Equals("FOOD", "food", StringComparison.OrdinalIgnoreCase);
Console.WriteLine(same); // True
```

`Ordinal` сравнивает кодовые единицы, `OrdinalIgnoreCase` дополнительно игнорирует регистр по соответствующим правилам .NET. Пользовательская алфавитная сортировка требует культурных правил. Сравнение строк нельзя универсально заменить вызовами `ToLower()`: это создаёт строки и скрывает выбор культуры. Подробнее: сравнение строк.

Число и его текстовое представление

У числа нет «запятой внутри». Разделитель относится к разбору и форматированию текста. Для машинного формата курса используем точку, запрет разделителей тысяч и явный InvariantCulture.

```
using System.Globalization;

bool ok = decimal.TryParse(
    "12.50",
    NumberStyles.AllowDecimalPoint,
    CultureInfo.InvariantCulture,
    out decimal amount);

Console.WriteLine(ok); // True
Console.WriteLine(amount.ToString("F2", CultureInfo.InvariantCulture)); // 12.50
```

Выбранные стили не разрешают знак, пробелы, экспоненту или разделители тысяч. Разрешённые формы определяются контрактом: `TryParse` не гарантирует ровно два знака после точки. Например, для строгой суммы в лабораторной мы отдельно проверим цифры и дробную часть.

В интерфейсе для человека можно выбрать его культуру и объяснить допустимый ввод. Нельзя незаметно использовать системную культуру для постоянного файла: перенос программы на другую машину тогда меняет смысл данных.

Отсутствие значения

`int?` — `Nullable<int>`: число либо отсутствие числа. `string?` — nullable-аннотация ссылочного типа для анализа компилятором. Она не создаёт новый CLR-тип строки и не вставляет автоматическую проверку во время выполнения. Подробнее: nullable reference types.

```
string? input = Console.ReadLine();
if (input is null)
{
    Console.WriteLine("Конец ввода");
    return;
}

string name = input.Trim();
Console.WriteLine(name.Length); // здесь input уже проверен
```

`null`, `""` и `"  "` — разные состояния. `string.IsNullOrWhiteSpace` объединяет их для задач, где ни одно не считается заполненным именем.

```
string? comment = null;
int length = comment?.Length ?? 0;
comment ??= "Без комментария";
Console.WriteLine(length);  // 0
Console.WriteLine(comment); // Без комментария
```

`?.` прекращает доступ, если слева null. `??` выбирает запасное значение при null, а `??=` присваивает его только при null. Оператор `!` после выражения лишь подавляет предупреждение компилятора. `comment!.Length` всё равно упадёт, если comment действительно null. Не устраняйте предупреждения массовым добавлением `!`.

Практика

06-A. Для `null`, пустой строки, трёх пробелов и `"  C#  "` определите, допустимо ли имя. Критерий: первые три отклонены, четвёртое преобразовано в `"C#"`. Различите завершение ввода и введённое пустое имя.

06-B. Напишите `TryReadAmount`: допустимы неотрицательные десятичные числа с точкой, без знака, пробелов и разделителя тысяч. Используйте явный NumberStyles. Критерии: `12.50` → 12.5m, `0` → 0m; `12,50`, `1,000`, `-1`, x — отказ. Вывод успешных сумм — F2 и InvariantCulture.

06-C. Уточните контракт B: от 1 до 999999.99 включительно, целая часть обязательна, после точки ровно одна или две цифры; форма без точки разрешена. Критерии: `1`, `1.5`, `1.50` успешны; `.5`, `1.`, `1.234`, `1000000` отклонены. Повторите проверки при CurrentCulture `ru-RU` и `en-US`: результат должен совпасть.

Проверка понимания

Почему Length не всегда считает видимые символы? Что именно гарантирует `string?`? Почему `!` не лечит NullReferenceException? Чем InvariantCulture отличается от Ordinal по назначению? Почему «парсится как decimal» слабее контракта собственного формата?

Подсказки · Далее 07

07. Массивы и базовые коллекции

Результат: вы храните набор значений, выбираете структуру под операцию и учитываете общие ссылки. Нужна глава 06.

Массив и индекс

Массив хранит фиксированное число элементов одного типа. Индексы начинаются с нуля; последний — `Length - 1`.

```
int[] values = [10, 20, 30]; // коллекционное выражение, C# 12+
Console.WriteLine(values[0]);  // 10
Console.WriteLine(values[^1]); // 30, первый с конца
int[] firstTwo = values[..2]; // новая копия [10, 20]
```

`new int[3]` создаёт три нуля. `new string[3]` создаёт три null, даже если тип элементов написан как string и nullable-анализ включён. Заполненность массива — отдельная ответственность. Индекс за границами приводит к исключению. Для пустого массива обращение к `[^1]` тоже ошибочно.

Прямоугольный массив объявляют `int[,] matrix = new int[2, 3];`. Ступенчатый `int[][] rows` — массив ссылок на массивы, строки могут иметь разную длину. Это разные модели: вложенный foreach и размеры нужно выбирать соответственно.

Копия ссылки и копия данных

```
int[] original = [1, 2];
int[] alias = original;
alias[0] = 9;
Console.WriteLine(original[0]); // 9

int[] copy = original[..];
copy[0] = 7;
Console.WriteLine(original[0]); // 9
```

Присваивание копирует ссылку на тот же массив. Срез массива создаёт новый массив. Если элементы сами ссылки на изменяемые объекты, копия массива всё ещё содержит ссылки на те же элементы: это поверхностное копирование. Для span семантика среза будет другой — представление той же памяти, глава 23.

Выбор коллекции

Структура	Для чего	Важные свойства
`List<T>`	Упорядоченный изменяемый список	Индекс O(1), поиск O(n), добавление в конец обычно O(1) амортизированно
`Dictionary<TKey,TValue>`	Поиск значения по ключу	В среднем O(1), уникальные ключи
`HashSet<T>`	Уникальные элементы	В среднем O(1) для поиска/добавления
`Queue<T>`	Первый пришёл — первый вышел	Enqueue/Dequeue
`Stack<T>`	Последний пришёл — первый вышел	Push/Pop

T — параметр типа. `List<decimal>` хранит decimal; детальное устройство обобщений будет позже. Оценка O(n) означает рост работы пропорционально размеру набора, а не конкретное число миллисекунд. Хеш-коллекции не гарантируют постоянное время для любого входа; качество хешей и сравнение важны.

```
List<decimal> amounts = [12.50m, 7.50m];
amounts.Add(3m);
decimal total = 0m;
foreach (decimal amount in amounts)
{
    total += amount;
}
Console.WriteLine(total); // числовое значение 23.00
```

`Count` — текущее число элементов списка, `Capacity` — ёмкость внутреннего хранения. Изменять состав списка при foreach обычно нельзя: перечислитель обнаруживает изменение. Для удаления используйте отдельный список результатов, RemoveAll или подходящий индексный цикл.

Группировка без LINQ

```
Dictionary<string, decimal> totals = new(StringComparer.OrdinalIgnoreCase);
string[] categories = ["food", "travel", "FOOD"];
decimal[] amounts = [12.50m, 30m, 7.50m];

for (int i = 0; i < categories.Length; i++)
{
    string category = categories[i];
    totals.TryGetValue(category, out decimal previous);
    totals[category] = previous + amounts[i];
}
Console.WriteLine(totals["food"]);   // числовое значение 20.00
Console.WriteLine(totals["travel"]); // числовое значение 30.00
```

При отсутствующем ключе TryGetValue возвращает false и default для значения. Для decimal это 0, что здесь удобно. Доступ `totals["missing"]` для чтения выбросит исключение; для записи индексатор добавляет или обновляет элемент. Add с уже существующим ключом — ошибка.

Не используйте порядок Dictionary как договор отчёта. Если результат должен воспроизводиться, извлеките ключи в List и вызовите `Sort(StringComparer.Ordinal)`, затем выводите по этому порядку. Параллельные массивы в примере требуют одинаковых длин; в следующей главе объединим связанные данные в объект.

Практика

07-A. Для массива `[3, 1, 4]` найдите сумму, минимум и максимум без LINQ. Критерии: 8, 1, 4. Для пустого массива сумма 0, а минимум/максимум явно отсутствуют; не выдавайте ноль за найденный минимум.

07-B. Реализуйте приведённую группировку самостоятельно. Критерии: food=20.00, travel=30.00; `FOOD` и `food` объединяются; вывод отсортирован. Добавьте четвёртую запись transport=0.01 и проверьте общую сумму 50.01.

07-C. Сделайте очередь из команд `add`, `report`, `quit` и обработайте её до quit. Сравните результат с теми же командами в Stack. Затем создайте HashSet идентификаторов `a, b, a`: критерий — два уникальных элемента. Объясните выбор структур словами.

Проверка понимания

Чем различаются Length и Count? Когда присваивание массива копирует элементы? Почему readonly-ссылка на List не обещает неизменность его содержимого? Почему нужен comparer и почему нельзя считать словарь автоматически отсортированным?

Официальный обзор коллекций

Подсказки · Далее 08

08. Классы и инварианты

Результат: вы объединяете связанные данные и поддерживаете допустимое состояние объекта. Нужна глава 07.

Тип и экземпляр

Класс описывает данные и действия. Экземпляр — отдельный объект этого класса. Если категория и сумма лежат в разных массивах, их длины и индексы легко перепутать. Объект Expense хранит их вместе.

Ниже полный тип для отдельного файла `Expense.cs`:

```
namespace Course;

public sealed class Expense
{
    public string Category { get; }
    public decimal Amount { get; }

    public Expense(string category, decimal amount)
    {
        if (string.IsNullOrWhiteSpace(category))
        {
            throw new ArgumentException("Категория обязательна.", nameof(category));
        }
        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(nameof(amount), "Сумма должна быть положительной.");
        }

        Category = category.Trim();
        Amount = amount;
    }
}
```

`throw` завершает нормальный ход создания объекта исключением; обработка — в главе 09. `nameof(amount)` получает имя параметра как строку и сохраняет связь при переименовании. После успешного конструктора всегда есть непустая категория и положительная сумма: это инвариант объекта. `sealed` запрещает наследование от класса; наследование рассмотрим позже.

В `Program.cs` того же проекта:

```
using Course;

Expense expense = new("food", 12.50m);
Console.WriteLine(expense.Category);
// expense.Amount = -1m; // не компилируется: доступного setter нет
```

Здесь два файла одного проекта, а не две самостоятельные программы. Запускаемый пример главы находится в проекте Foundations.

Поле, свойство и доступность

Поле хранит значение непосредственно. Свойство задаёт доступ через get/set. Авто-свойство позволяет компилятору создать backing field. Публичный set разрешает изменение; private set оставляет его внутри класса; get без set в этом примере задаётся при создании.

Модификатор	Доступ
`public`	Доступен потребителям типа

Модификатор	Доступ
`private`	Только внутри содержащего типа
`internal`	Внутри сборки
`protected`	В типе и производных типах
`protected internal`	В сборке или в производном типе
`private protected`	В производном типе в той же сборке

Тип верхнего уровня без модификатора обычно internal, члены класса — private. Вложенные типы имеют свои правила области видимости. Папка проекта не задаёт namespace автоматически.

`this` обозначает текущий экземпляр. `static`-поле принадлежит типу, поэтому все экземпляры наблюдают одно состояние. Такая общая изменяемая переменная усложняет тестирование и конкуренцию; не превращайте её в универсальное хранилище.

Неизменяемость и инициализация

`readonly`-поле можно присвоить при объявлении или в соответствующем конструкторе. Если оно содержит List, запрещено присвоить другой список; добавлять элементы в существующий список всё ещё можно. Неизменяемость ссылки не равна неизменяемости объекта.

`init` позволяет присваивание в фазе инициализации, а `required` требует указания члена при создании по правилам компилятора. Например, object initializer выглядит как `new Options { Title = "Report" }`. Ни init, ни required сами по себе не проверяют, что строка непустая, а число положительное.

Конструктор в нашем примере связывает проверку и создание. Внешнему коду нельзя предложить сначала `new Expense()`, затем отдельно «не забудьте вызвать Validate». Для типов с изменяемым состоянием методы должны сохранять инвариант после каждого действия.

Общие ссылки

```
List<Expense> first = [new("food", 12.50m)];
List<Expense> second = first;
second.Add(new("travel", 30m));
Console.WriteLine(first.Count); // 2
```

Списки first и second — два имени для одного объекта. Создание нового `List<Expense>(first)` копирует набор ссылок, но не создаёт новых Expense. В данном примере элементы предоставляют только чтение, поэтому совместное использование упрощается. Для изменяемого элемента оно требует осторожности.

Композиция означает, что один объект содержит другой и делегирует ему работу. Для журнала расходов естественнее хранить список Expense внутри Ledger, чем наследовать Ledger от List. Так можно разрешить только предметные операции и сохранять правила.

Практика

08-A. Создайте Expense в отдельном файле и два разных экземпляра. Критерий: изменение списка с одним объектом не создаёт автоматически второй объект; покажите совпадение ссылок через `ReferenceEquals` при присваивании.

08-B. Создайте `Product` с непустым Name и Price > 0, только get-свойства. Критерии: `("Book", 15m)` создаётся; пустое имя, пробелы, 0 и −1 отклоняются конструктором. Попытка присвоить Price снаружи не компилируется. Проверяйте такой контрпример в отдельной копии.

08-C. Создайте `Basket`, который содержит `private List<Product>`, метод Add и метод Total. Не выдавайте наружу изменяемый список. Критерий: два товара 15m и 7.5m дают 22.5m, пустая корзина — 0m. Объясните, какое правило проще поддерживать благодаря закрытому списку.

Проверка понимания

Почему public set может разрушить инвариант? Что гарантирует required и чего не гарантирует? Почему readonly List не становится immutable? Чем экземпляр отличается от типа и от ссылки?

Официальное описание классов

Подсказки · Далее 09

09. Ошибки, ресурсы и первые файлы

Результат: вы разделяете неверные данные и сбой операции, закрываете ресурсы и читаете текстовый файл. Нужна глава 08.

Когда возвращать результат, когда бросать исключение

Неправильная строка из пользовательского файла ожидаема: Try-метод сообщает об отказе и причине. Вызов конструктора с нарушенным контрактом — ошибка использования API: уместно исключение. Отказ файловой системы нельзя предсказать полной предварительной проверкой: его нужно обработать.

Исключение прерывает обычное выполнение и ищет подходящий catch по стеку вызовов. Если подходящего обработчика нет, процесс обычно завершится с ошибкой. Не используйте пустой catch: он превращает сбой в незаметно неверный результат.

```
try
{
    string text = File.ReadAllText("expenses.txt");
    Console.WriteLine(text);
}
catch (FileNotFoundException)
{
    Console.Error.WriteLine("Файл расходов не найден.");
}
catch (UnauthorizedAccessException)
{
    Console.Error.WriteLine("Недостаточно прав для чтения файла.");
}
catch (IOException ex)
{
    Console.Error.WriteLine($"Ошибка чтения: {ex.Message}");
}
```

Более конкретный catch пишут перед общим. Сообщение полезно человеку, но программа не должна разбирать локализованный текст Exception.Message для принятия решения. Данные для диагностики — тип, stack trace и контекст операции.

Повторное выбрасывание и finally

В catch запись `throw;` передаёт то же исключение выше, сохраняя исходный стек. `throw ex;` меняет информацию о месте повторного выбрасывания и затрудняет диагностику. Новое исключение с исходным в InnerException уместно, когда нужно выразить другой уровень абстракции.

`finally` выполняет завершающие действия при нормальном выходе и при обычном раскручивании стека. Это не гарантия при аварийном завершении процесса или отключении питания. Фильтр `catch (IOException ex) when (...)` принимает исключение только при истинном условии; не включайте в фильтр побочные действия, на которые рассчитывает программа.

Ресурс и сборщик мусора

Файл, сокет и подключение могут владеть ограниченным ресурсом ОС. GC управляет памятью; он не обещает немедленного закрытия файлов при выходе переменной из области видимости. `IDisposable.Dispose` выражает освобождение, а using обеспечивает вызов Dispose на выходе из блока.

```
using System.Text;

using (StreamReader reader = new(
    "expenses.txt",
    new UTF8Encoding(false, true),
    detectEncodingFromByteOrderMarks: false))
{
    string? line;
    while ((line = reader.ReadLine()) is not null)
    {
        Console.WriteLine(line);
    }
}
```

Здесь UTF-8 декодер отклоняет некорректные байты. Режим с BOM требует отдельного решения; контракт лабораторной допускает UTF-8 BOM и удаляет его явно. Форма `using var reader = ...;` освобождает ресурс в конце текущей области видимости. Она удобна, но большой метод может удерживать файл дольше, чем требуется.

Рабочая директория и путь

Относительный путь считается от рабочей директории процесса, которая не обязана совпадать с папкой исходника или исполняемого файла. Для диагностики полезен `Environment.CurrentDirectory`. Для составления пути используйте `Path.Combine(folder, "expenses.txt")`, для явного абсолютного представления — `Path.GetFullPath(path)`.

`File.Exists` не делает последующее чтение безопасным: между проверкой и открытием файл может исчезнуть, права могут измениться. Обрабатывайте ошибку самой операции. Не вставляйте произвольное пользовательское имя в доверенный путь без проверки границ: в веб-приложениях это особенно важно.

`File.ReadAllLines` загружает все строки в память. `File.ReadLines` перечисляет лениво: ошибка может возникнуть во время foreach, поэтому try должен охватывать перечисление, а не только создание последовательности.

Коды завершения

Обычный результат пишут в stdout, диагностику — в stderr. В top-level программе `return 0;` сообщает успех, `return 1;` — ошибку по договору приложения. Разным категориям ошибок можно назначить разные коды. Это позволяет автоматизации понять результат, не разбирая текст сообщения.

Не печатайте итоговый успешный отчёт после частичной ошибки, если контракт требует обработать файл целиком. Сначала прочитайте и проверьте данные, затем формируйте результат.

Практика

09-A. Создайте UTF-8 файл из трёх строк, прочитайте его using+StreamReader. Критерий: ровно три строки в исходном порядке; после выхода из using файл можно снова открыть.

09-B. Напишите утилиту подсчёта непустых строк. Контракт: путь — один аргумент, успех → число в stdout и код 0; ошибка аргументов → stderr и код 2; ошибка чтения → stderr и код 3. Критерии: пустой файл → 0, файл `a/пустая строка/b` → 2, отсутствующий файл → код 3 без результата в stdout.

09-C. Добавьте strict UTF-8 и приём файла с неверным байтом. Он должен завершаться диагностикой декодирования, не подменяя байт символом замены. Продемонстрируйте освобождение ресурса при исключении внутри блока чтения. Критерий: повторное открытие файла после catch возможно.

Проверка понимания

Что освобождает GC и что освобождает Dispose? Чем throw отличается от throw ex? Почему File.Exists не исключает гонку? Почему try вокруг создания File.ReadLines недостаточен? Что увидит вызывающий скрипт при записи ошибки в stdout вместо stderr?

Обработка исключений · using

Подсказки · Далее 10

10. Практикум ExpenseLedger CLI

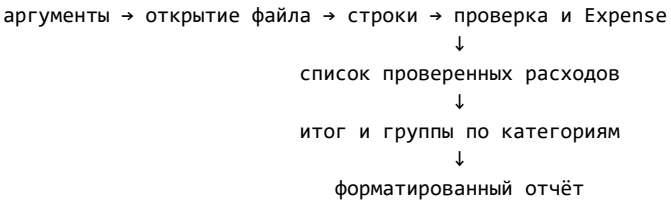
Результат: готовая консольная программа читает расходы, проверяет формат и выводит отчёт. Нужны главы 01–09. Подробный договор — в лабораторной L01.

Постановка задачи

Пользователь хранит небольшую выгрузку расходов в UTF-8 файле. Ему нужны число записей, сумма и суммы по категориям. Формат не должен зависеть от языка Windows. В ошибочной выгрузке нужно указать номер строки, чтобы её можно было исправить. При ошибке данных успешный отчёт не выдаётся.

Это первая объединённая работа: она требует почти всех уже изученных приёмов. Сначала реализуйте её в `work/ExpenseLedger`. Полный пример в `examples/ExpenseLedger` используйте для сравнения после собственной попытки.

Разделение ответственности



У программы четыре границы: аргументы процесса, файловая система, текстовый формат и модель данных. Каждая требует своего решения. Неверный decimal нельзя превращать в 0: это создаст фиктивный расход. Нельзя также складывать данные до проверки всех строк и печатать частичный результат как полный.

Разумный первый набор методов: `TryParseLine`, `TryParseAmount`, `ReadAll`, `WriteReport`. Два первых отвечают за данные, следующие — за I/O. Класс Expense сохраняет инварианты. Сложные generic-абстракции на этом этапе не нужны.

Шаги реализации

1. Создайте консольный проект net10.0, включите nullable и убедитесь, что он запускается.
2. Обработайте отсутствие пути и лишние аргументы. Проверьте код завершения.
3. Напишите разбор суммы на отдельных строках. Проверьте границы до чтения файла.
4. Разберите строку на три поля и проверьте дату, категорию и сумму.
5. Создайте тип Expense и запретите недопустимое состояние.
6. Прочитайте файл, сохранив физические номера строк, включая пустые.
7. Посчитайте итог и группы через Dictionary. Отсортируйте ключи явно.
8. Добавьте обработку ошибок чтения, строгую кодировку и раздельные stdout/stderr.
9. Пройдите все сценарии приёмки и напишите короткую инструкцию запуска.

Даты используют новый для вас тип `DateOnly` из BCL: он хранит календарную дату без времени и зоны. `TryParseExact` с шаблоном `yyyy-MM-dd` проверяет реальную дату, например отклоняет 2026-02-30. Не пытайтесь проверить календарь только количеством символов и тире.

Разобранный алгоритм отчёта

Для каждой записи увеличьте общее число и общую сумму. Получите предыдущую сумму категории через TryGetValue и запишите новую. Категории по договору нечувствительны к регистру; нормализуйте их к нижнему ASCII-регистру и используйте Ordinal для ключей.

После окончания расчёта создайте список ключей словаря, отсортируйте его и выведите значения в F2 и InvariantCulture. Сумма для входа из примера — 50.00 независимо от CurrentCulture. Приведение регистра допустимо здесь потому, что формат категорий ограничен ASCII и результат должен иметь каноническое написание. Для произвольных пользовательских строк правила шире.

Проверка решения

Проверьте success, пустой файл, повтор категории с разным регистром, несуществующий файл, невозможную дату, отрицательную сумму, лишнее поле и неправильную кодировку. Для каждого случая проверяются три наблюдаемых результата: stdout, stderr и exit code.

Готовый пример запускается так:

```
dotnet run --project examples/ExpenseLedger -- labs/01-expense-ledger/data/valid.txt
python tools/verify.py
```

Проверка собственной реализации с тем же контрактом:

```
dotnet build work/ExpenseLedger -c Release
python tools/verify.py --ledger-dll work/ExpenseLedger/bin/Release/net10.0/ExpenseLedger.dll
```

Скрипт использует временные тестовые файлы и сверяет поведение. Автоматические проверки помогают найти нарушение контракта, но не доказывают качество всех решений. Перед сдачей самостоятельно объясните выбор типов, условия ветвлений и область жизни reader.

Практика

10-A. Реализуйте основной сценарий L01. Критерий: точное совпадение отчёта из лабораторной.

10-B. Реализуйте весь контракт и пройдите сценарии ошибок. Критерии: при любом ошибочном файле stdout пуст, причина в stderr, код соответствует категории сбоя. После ошибки в строке 2 программа не должна печатать итог строки 1.

10-C. В собственной ветке добавьте необязательный фильтр месяца `yyyy-MM`. Сначала отдельно запишите новый контракт аргументов. Полный файл всё равно валидируется, затем фильтруются записи. Критерии: месяц без записей → пустой отчёт; ошибочная строка вне выбранного месяца всё равно отклоняется. Базовые проверки B должны проходить при запуске без фильтра. Готовый пример реализует только A/B.

Допуск к следующему блоку

Объясните без запуска, что произойдёт с `1,50`, `2026-02-30`, `FOOD`, UTF-16 файлом и EOF в пустом файле. Покажите в коде единственное место определения допустимого формата суммы. Попросите другого человека запустить программу по вашей инструкции в новой папке.

После сдачи переходите к главе 11: интерфейсы, равенство, обобщения, делегаты, LINQ и библиотека предметной области изучаются в следующем готовом блоке 11–18. Состояние наполнения — в PROGRESS.

Подсказки · Лабораторная

11. Наследование и интерфейсы

Результат: вы вызываете разные реализации через общий контракт и отличаете переопределение от скрытия. Нужны главы 08–10. Запускаемый пример: `dotnet run --project examples/TypesAndQueries -- 11`.

Подстановка объекта

Переменная имеет статический тип, известный компилятору. Объект имеет фактический тип во время выполнения. Переменная базового типа может ссылаться на объект производного типа. Доступные члены выбираются по статическому типу; реализация virtual-метода — по фактическому.

```
Report report = new DetailedReport();
Console.WriteLine(report.Title); // Detailed

abstract class Report
{
    public abstract string Title { get; }
    public virtual string Format() => Title;
}

sealed class DetailedReport : Report
{
    public override string Title => "Detailed";
}
```

Это самостоятельный Program.cs: инструкции идут перед типами. Abstract-класс нельзя создать через new. Abstract-член требует реализации в конкретном потомке. Virtual имеет реализацию по умолчанию; override заменяет её в цепочке виртуальных вызовов. Sealed запрещает дальнейшее наследование; sealed override запрещает следующее переопределение.

У класса один непосредственный базовый класс, но интерфейсов может быть несколько. Конструктор потомка вызывает конструктор базы, при необходимости через `base(...)`. Не вызывайте переопределяемый метод из конструктора базы: он может обратиться к ещё не инициализированному состоянию потомка.

Скрытие не равно переопределению

```
BaseLabel first = new NewLabel();
NewLabel second = new();
Console.WriteLine(first.Name());  // base
Console.WriteLine(second.Name()); // new

class BaseLabel
{
    public string Name() => "base";
}
class NewLabel : BaseLabel
{
    public new string Name() => "new";
}
```

New явно сообщает о скрытии члена. Здесь выбор зависит от статического типа переменной. Если нужен единый полиморфный вызов, объявите базовый метод virtual и используйте override. Отсутствие предупреждения после добавления new ещё не доказывает правильность проекта.

Override некоторых методов и get-only свойств может возвращать более конкретный ссылочный тип, чем база: это ковариантный результат. Например, базовый Clone возвращает Report, а производный —

DetailedReport. Параметры нельзя произвольно сузить: контракт базы должен оставаться применимым.

Интерфейс как граница

Вместо «сервис знает List» сформулируйте нужную ему операцию хранения:

```
public interface IExpenseStore
{
    bool TryAdd(Expense expense);
    IReadOnlyList<Expense> Snapshot();
}
```

Это фрагмент контракта L02; тип Expense появится в её библиотеке. TryAdd возвращает false при повторном идентификаторе. Snapshot обещает отдельный снимок данных, который не изменится после следующего добавления. Сигнатура не описывает эти обещания полностью: их нужно записать в документации и проверить на каждой реализации.

Реализация `InMemoryExpenseStore : IExpenseStore` хранит Dictionary внутри. Сервис принимает IExpenseStore в конструкторе. При замене хранилища правила расчёта не меняются. Это внедрение зависимости вручную; DI-контейнер для такого проектирования не требуется.

Явная реализация и default members

Если два интерфейса имеют конфликтующие члены, можно реализовать их явно:

```
interface IText { string Render(); }
interface IJson { string Render(); }

sealed class View : IText, IJson
{
    string IText.Render() => "text";
    string IJson.Render() => "{}";
}
```

`((IJson)new View()).Render()` вызывает JSON-вариант. Вызова `new View().Render()` здесь нет. Явная реализация доступна через соответствующий интерфейс; модификатор public перед ней не нужен.

Интерфейс может содержать реализацию метода по умолчанию. Она вызывается через интерфейс и не добавляет обычный унаследованный член к самому классу. Такой механизм полезен для развития API, но требует продуманного разрешения конфликтов. Интерфейс не хранит обычные поля экземпляра.

Static abstract и static virtual члены описывают возможности самого типа, например оператор или Parse. Доступ к ним обычно выполняется через параметр обобщённого типа с нужным ограничением. Это не виртуальный вызов через экземпляр. Применение — generic math в главе 13.

Когда выбирать композицию

Наследование утверждает, что потомок можно использовать вместо базы с сохранением обещаний. Если в базовом хранилище Add разрешён, потомок ReadOnlyStore, который всегда бросает NotSupportedException на Add, нарушает ожидания такого контракта.

Композиция означает «содержит и использует». Ledger содержит хранилище, не является хранилищем. Разделите интерфейсы чтения и записи, если потребителям нужны разные возможности. Не создавайте интерфейс только ради наличия буквы I: он полезен на настоящей границе поведения.

Практика

11-A. Повторите пример со скрытием, затем замените new на virtual/override в соответствующих местах. Критерий: сначала base/new, после изменения оба вызова дают new. Объясните выбор реализации.

11-B. Опишите `ITextWriter.Write(string)` и две реализации: накопление строк в памяти и вывод в Console. Метод формирования отчёта принимает интерфейс. Критерий: обе реализации получают одинаковые три строки, а тест памяти не перехватывает Console.

11-C. Разделите `IStore` с Read/Add на `IReader` и `IWriter`. Критерий: компонент отчёта компилируется только с IReader; подстановка реализации чтения не требует метода Add, бросающего исключение. Приведите пример нарушенного обещания старого интерфейса.

Проверка понимания

Чем статический тип отличается от фактического? Почему new не заменяет override? Что именно нужно тестировать помимо сигнатур интерфейса? Когда явная реализация полезна? Почему default interface method не всегда доступен через переменную конкретного класса?

Интерфейсы · Полиморфизм

Подсказки · Далее 12

12. Структуры, записи и равенство

Результат: вы выбираете семантику значения или идентичности и создаёте устойчивый ключ коллекции. Нужна глава 11. Запуск: `dotnet run --project examples/TypesAndQueries -- 12`.

Значение и место хранения

При присваивании struct копируются его поля. При присваивании ссылки на class копируется ссылка. Это описание семантики, а не правило «struct всегда в стеке»: структура может быть полем объекта, элементом массива или оказаться упакованной в managed heap.

```
Point first = new(1, 2);
Point second = first;
Console.WriteLine(first == second); // True

public readonly record struct Point(int X, int Y);
```

Readonly struct запрещает обычное изменение полей экземпляра после создания. Если поле содержит ссылку на List, сам List по-прежнему может меняться. Большие структуры дороги при многократном копировании; размер и сценарий важнее желания избежать new.

Default структуры существует независимо от проверок её конструктора. `default(Money)` и элементы `new Money[10]` могут содержать нулевые поля без вызова пользовательского конструктора. В L02 нулевая сумма — допустимый Money, но расход с нулём запрещён. Так контракт типа совместим с default, а более строгое правило находится в Expense.

Records и обычные классы

Обычный class по умолчанию имеет ссылочное равенство; record создаёт реализацию равенства по данным. `record` без struct означает record class. Record не гарантирует глубокую неизменяемость: изменяемые поля и свойства допустимы, а вложенные объекты сохраняют собственную семантику равенства.

```
Person first = new("Ann");
Person second = new("Ann");
Console.WriteLine(first == second);                // True
Console.WriteLine(ReferenceEquals(first, second)); // False
Person renamed = first with { Name = "Bob" };
Console.WriteLine(first.Name);                     // Ann

public sealed record Person(string Name);
```

With создаёт поверхностную копию. У записи с List в свойстве исходник и копия могут ссылаться на один список. Два отдельных списка с одинаковыми элементами сами по себе не становятся равными из-за включения в record. Если нужен глубокий набор значений, спроектируйте его явно.

Позиционные параметры record создают публичные свойства и Deconstruct. У обычного `class Worker(string name)` primary constructor делает параметр доступным в теле типа, но автоматически не объявляет public Name. Это разные соглашения синтаксиса. Позиционный record struct по умолчанию допускает изменение свойств; readonly record struct — нет.

Договор Equals и GetHashCode

Если `a.Equals(b)` истинно, хеши должны совпадать. Обратное неверно: коллизии допустимы. Равенство должно быть рефлексивным, симметричным и транзитивным для обычных значений типа. Для самостоятельно реализованного value object согласуйте `Equals(object?)`, `IEquatable<T>.Equals`, GetHashCode и операторы ==/!=.

Не изменяйте поля, участвующие в хеше, после добавления объекта в Dictionary или HashSet: объект может остаться в контейнере, но перестать находиться ожидаемым способом. Не сохраняйте GetHashCode в файл как постоянный идентификатор — он не обещает стабильность между процессами.

Идентичность сущности и равенство снимков данных — разные вопросы. В L02 записи сравниваются по всем данным, а уникальность в хранилище определяется только Id. Одинаковый Id с другой суммой всё равно является конфликтом добавления.

Boxing и unboxing

```
int original = 7;
object boxed = original; // boxing: отдельное упакованное значение
original = 9;
Console.WriteLine((int)boxed); // 7
long wide = (int)boxed;        // сначала unboxing int, затем int -> long
```

`(long)boxed` выбросит InvalidCastException: внутри упакован именно int. Передача структуры через object или интерфейс может вызвать boxing. Generic-коллекция `List<int>` позволяет хранить числа без упаковки каждого элемента как object. Оценивать влияние упаковки на скорость будем измерениями в главе 34.

Enum и кортежи

```
[Flags]
enum Permission { None = 0, Read = 1, Write = 2, Export = 4 }
```

Отдельные флаги выбирают степенями двойки. `Read | Write` объединяет, проверка через & выделяет бит. Атрибут Flags не запрещает неизвестные биты. Приведение `(Permission)128` возможно; входные значения проверяются отдельно. Enum.IsDefined подходит для отдельных именованных значений, но не для всех комбинаций Flags.

Кортеж `(int Count, decimal Total)` удобен для локального результата из двух величин. Он основан на ValueTuple; имена элементов не задают новую несовместимую разновидность типа. Деконструкция `var (count, total) = summary;` присваивает компоненты локальным переменным. Для публичной предметной модели отдельный именованный тип обычно лучше выражает правила.

Практика

12-A. Создайте readonly record struct Point и HashSet из (1,2), (1,2), (2,1). Критерий: Count=2. У двух отдельных record class Person("Ann") проверьте одновременно равенство данных и несовпадение ссылок.

12-B. Реализуйте Money для одной условной валюты: decimal, неотрицательное значение, не более двух значащих знаков после точки; ноль допустим. Добавьте сложение. Критерии: default равен Money(0), 1.20+2.30=3.50; −1 и 1.001 отклоняются. Уточнение: 1.2300m допустимо, поскольку значение точно выражается двумя знаками.

12-C. Создайте record с `List<string> Tags` и выполните with. Измените Tags копии. Критерий: воспроизведено общее изменение; затем устраните его отдельной копией списка. Объясните, почему для элементов-классов даже этого может быть недостаточно.

Проверка понимания

Может ли структура находиться внутри объекта в heap? Почему конструктор не запрещает default? Когда record сравнивает вложенные коллекции по ссылкам? Почему нельзя менять ключ после вставки? Чем идентификатор сущности отличается от GetHashCode?

Records · Равенство

Подсказки · Далее 13

13. Обобщения и контракты типов

Результат: вы пишете повторно используемый алгоритм с проверкой типов при компиляции. Нужны главы 11–12. Запуск: `dotnet run --project examples/TypesAndQueries -- 13`.

Зачем параметр типа

Вместо контейнера object, который допускает смешение данных и требует cast, объявите тип элемента:

```
static T FirstOr<T>(IReadOnlyList<T> items, T fallback)
{
    return items.Count == 0 ? fallback : items[0];
}
```

Это функция для размещения в Program.cs. `FirstOr(new[] { 3, 5 }, 0)` возвращает int 3. Компилятор выводит T из аргументов; иногда тип указывают явно, например `FirstOr<string>(...)`. Вывод не строится только из ожидаемого возвращаемого типа.

Generics не равны текстовой подстановке макроса. Параметры типов присутствуют в metadata, а CLR и компилятор знают закрытые типы вроде `List<int>`. Reflection позволяет различать открытый `typeof(List<>)` и закрытый `typeof(List<int>)`; создание типов по метаданным будет в главе 21.

Ограничения

Без ограничения T не имеет произвольного свойства Amount или метода CompareTo. Ограничение делает нужную возможность частью контракта:

```
static T Min<T>(T left, T right) where T : IComparable<T>
{
    return left.CompareTo(right) <= 0 ? left : right;
}
```

Ограничение	Что оно сообщает
`class` / `class?`	Ссылочный тип с соответствующим nullable-намерением
`struct`	Non-nullable value type; `Nullable<int>` не подходит
`notnull`	Non-nullable reference или value type
`unmanaged`	Value type без управляемых ссылок в составе
Базовый класс / интерфейс	Доступны обещанные им члены
`new()`	Можно вызвать доступный конструктор без параметров

Notnull в nullable-анализе может приводить к предупреждению, а не безусловной ошибке языка; в проектах курса предупреждения считаются ошибками. Он не валидирует данные во время выполнения.

Основное ограничение идёт первым, затем интерфейсы, new() — после обычных ограничений. Struct уже подразумевает возможность создания значения без параметров, поэтому с new() его не объединяют. Unmanaged несовместим с одновременным struct/new(). Полный набор допустимых сочетаний следует проверять по справочнику ограничений.

У специального `where T : default` другая задача: он уточняет отсутствие class/struct-ограничения при override или явной реализации, где важно толкование T?. Это не требование «T имеет default». Антиограничение `allows ref struct` разрешает T быть ref struct, идёт после остальных ограничений и требует соблюдения правил безопасности ссылок; с class/class? его объединить нельзя. Практические ref struct будут в главе 23.

Отсутствие значения и Result

Для произвольного T запись T? нельзя всегда понимать как `Nullable<T>`. При T=string это nullable-аннотация; при T=int без struct-ограничения это не универсальная команда «завернуть int в `Nullable<int>`». Контракт должен явно определять, как представляется отсутствие.

В L02 используем `Result<T> where T : notnull`: либо успех со значением T, либо отказ с кодом ошибки. Фабрики Success и Failure скрывают конструктор. Value доступен только при успехе; при ошибке чтение Value бросает InvalidOperationException. Так результат не допускает комбинацию «успех без значения» или «ошибка без причины».

```
// Фрагмент после объявления Result<T> из L02.
Result<int> result = Result<int>.Success(42);
if (result.IsSuccess)
{
    Console.WriteLine(result.Value);
}
```

Не кодируйте отсутствие результата только как default(T): 0, false и пустая структура могут быть нормальными значениями. Ошибка duplicate_id — ожидаемое бизнес-событие, а не повод аварийно завершать процесс.

Вариантность

Ковариантный параметр `out T` используется как выход. Например, `IEnumerable<string>` можно присвоить `IEnumerable<object>`: чтение string как object безопасно. Контравариантный `in T` описывает вход: обработчик object может обработать и string.

Такие преобразования вариантности действуют для ссылочных типов. `IEnumerable<int>` не превращается в `IEnumerable<object>` автоматически. `List<string>` не является `List<object>`: иначе через вторую ссылку можно было бы записать число. Объявлять in/out разрешено на параметрах интерфейсов и делегатов, а не обычных классов. Окончательные правила позиций учитывают вложенные делегаты, поэтому проверяйте сложные случаи компилятором.

Статические данные и generic math

У `Cache<int>` и `Cache<string>` собственные статические поля. Не ожидайте одного общего счётчика на все закрытые варианты типа.

Статический интерфейсный контракт позволяет выражать операции над T без dynamic:

```
using System.Numerics;

// Метод внутри класса или локальная функция Program.cs.
static T Add<T>(T left, T right) where T : INumber<T> => left + right;
```

INumber обещает числовые операции, включая нужный оператор. Вызов `Add(2, 3)` даёт int 5, а `Add(1.5m, 2m)` — decimal 3.5. Доменный Money не обязан реализовать весь числовой интерфейс: не все действия обычного числа имеют смысл для денег. Берите минимально достаточный контракт.

Практика

13-A. Реализуйте Min с IComparable. Критерии: Min(3,5)=3, Min(5,3)=3, равные числа сохраняют значение. Для строк явно обсудите, какое правило сравнения вам нужно и почему CompareTo зависит от культуры.

13-B. Реализуйте `Result<T>` с закрытым конструктором и фабриками. Критерии: Success(0) является успехом; Failure("duplicate_id") не выдаёт Value; пустой код ошибки и null в `Result<string>.Success` отклоняются при выполнении.

13-C. Объявите `IProducer<out T>` с Get и `IConsumer<in T>` с Accept. Критерий: объяснены безопасные направления преобразования для string/object. Отдельно покажите некомпилируемый контрпример для `List<string>`/`List<object>`.

Проверка понимания

Почему object менее выразителен, чем T? Почему ограничения позволяют вызывать методы T? Почему default(T) не подходит как единственный признак ошибки? Что ограничивает вариантность? Чем статический интерфейсный оператор отличается от вызова virtual-метода объекта?

Подсказки · Далее 14

14. Делегаты, лямбды и события

Результат: вы передаёте поведение как аргумент, понимаете захват переменных и управляете подписками. Нужны главы 11–13. Запуск: `dotnet run --project examples/TypesAndQueries -- 14`.

Функция как значение

Делегат хранит вызываемый метод и при необходимости его целевой объект. Тип делегата определяет параметры и результат.

```
Func<decimal, bool> isLarge = amount => amount >= 1000m;
Console.WriteLine(isLarge(1200m)); // True

static bool IsPositive(decimal amount) => amount > 0m;
Func<decimal, bool> isPositive = IsPositive; // method group
```

Func имеет результат последним параметром типа; Action возвращает void; `Predicate<T>` принимает T и возвращает bool. Можно объявить собственный тип, например `delegate decimal Discount(decimal price);`, если предметное имя помогает понять API.

Лямбда `amount => amount >= 1000m` — короткое описание поведения. Блочное тело `amount => { return amount >= 1000m; }` позволяет несколько инструкций. Анонимный метод `delegate(decimal amount) { return amount > 0m; }` — более старый синтаксис. Если преобразование лямбды задаёт expression tree, результатом будет описание выражения, а не обычный готовый делегат; это понадобится для IQueryable.

Захват переменной

```
decimal limit = 100m;
Func<decimal, bool> exceeds = value => value > limit;
limit = 200m;
Console.WriteLine(exceeds(150m)); // False
```

Замыкание захватывает переменную, а не фиксирует её значение на момент объявления. Если нужен снимок, создайте отдельную переменную и больше её не изменяйте. Захваченные объекты могут жить дольше метода, который создал лямбду.

Распространённая ловушка — переменная for:

```
List<Action> actions = [];
for (int i = 0; i < 3; i++)
{
    actions.Add(() => Console.WriteLine(i));
}
foreach (Action action in actions)
{
    action(); // 3, 3, 3
}
```

Для независимых значений создайте `int captured = i;` внутри тела и захватывайте captured. В современном C# переменная foreach имеет отдельный экземпляр для каждой итерации; не переносите без проверки старые объяснения этого поведения.

Static-лямбда, например `static amount => amount > 0m`, не захватывает окружение. Это выражает намерение и предотвращает случайное удержание объекта, но само по себе не обещает нулевых выделений памяти во всех ситуациях.

Список вызовов

Делегаты поддерживают += и -=, образуя список обработчиков. Они вызываются последовательно, в порядке добавления. Исключение одного обработчика по обычным правилам прекращает вызов следующих. Для делегата с результатом обычный вызов возвращает результат последнего обработчика, а не список всех результатов.

Поэтому цепочку обработчиков не следует использовать как неявный механизм транзакций или накопления результатов. Если нужно собрать ответы или независимо обработать ошибки, опишите алгоритм вызова явно.

Event ограничивает доступ

```
public sealed class Counter
{
    public event EventHandler? Changed;
    public int Value { get; private set; }

    public void Increment()
    {
        Value++;
        Changed?.Invoke(this, EventArgs.Empty);
    }
}
```

Снаружи можно подписаться и отписаться, но нельзя присвоить событию произвольный список или поднять его обычным вызовом. Внутри владельца field-like event ведёт себя как делегат. `EventHandler<TEventArgs>` передаёт данные уведомления; в L02 это созданный Expense.

```
// Фрагмент после создания Counter.
Counter counter = new();
int notifications = 0;
EventHandler handler = (_, _) => notifications++;
counter.Changed += handler;
counter.Increment();
counter.Changed -= handler;
counter.Increment();
Console.WriteLine(notifications); // 1
```

Для отписки от лямбды сохраните делегат в переменной. Новый синтаксически похожий обработчик не обязан совпадать с тем, который добавили раньше. Именованный метод облегчает симметричную отписку.

Время жизни и ошибки наблюдателя

Пока издатель доступен, его событие может удерживать делегат, а делегат — объект подписчика. Если долгоживущий сервис хранит обработчик закрытого окна, окно остаётся достижимым для GC. Владелец подписки должен отписываться в подходящий момент, например Dispose. Граф ссылок объясняет утечку надёжнее ожидания, что принудительный GC обязательно её продемонстрирует.

В Counter состояние изменяется до уведомления. Если обработчик бросит исключение, Value уже увеличен. В L02 Add тоже сначала сохраняет расход, затем поднимает Added. Ошибка наблюдателя не откатывает запись. Такое поведение нужно документировать; надёжная доставка событий требует других механизмов в будущих главах.

Async-обработчик события часто имеет форму async void. Его нельзя ожидать как Task обычным вызовом события; исключения после асинхронного продолжения имеют другие пути обработки. До главы 31 используйте здесь синхронные короткие обработчики и не считайте event гарантией завершения фоновой работы.

Практика

14-A. Напишите метод отбора положительных decimal, принимающий `Func<decimal,bool>`. Критерий: для −1, 0, 2, 5 предикат >0 даёт 2,5; предикат >=5 даёт только 5. Алгоритм перебора не меняется при замене условия.

14-B. Воспроизведите 3,3,3 через for и исправьте на 0,1,2 локальной копией. Критерий: объясните, сколько переменных захвачено в обоих вариантах. Сделайте отдельный вариант со static-лямбдой и явным параметром вместо захвата.

14-C. Подпишитесь на Counter, увеличьте его, отпишитесь и увеличьте ещё раз. Критерий: уведомление одно, значение счётчика два. Добавьте бросающий обработчик: проверьте, что состояние уже изменилось, а следующий обработчик не был вызван. Укажите, какие ссылки нужно разорвать при закрытии подписчика.

Проверка понимания

Что хранит делегат экземплярного метода? Захватывает ли лямбда значение или переменную? Почему event предпочтительнее публичного поля Action? Что произойдёт при исключении подписчика? Почему удаление локальной переменной подписчика не гарантирует его освобождение?

Подписка и отписка

Подсказки · Далее 15

15. Перечисление и итераторы

Результат: вы отличаете рецепт перечисления от готовой коллекции и освобождаете ресурсы при досрочном прекращении чтения. Нужна глава 14. Запуск: `dotnet run --project examples/TypesAndQueries -- 15`.

Что происходит в foreach

`IEnumerable<T>` предоставляет GetEnumerator. Перечислитель `IEnumerator<T>` хранит позицию: MoveNext перемещается к следующему элементу, Current даёт текущий. Не читайте Current до первого успешного MoveNext или после конца последовательности.

Упрощённая модель foreach:

```
using IEnumerator<int> iterator = values.GetEnumerator();
while (iterator.MoveNext())
{
    int value = iterator.Current;
    Console.WriteLine(value);
}
```

Это фрагмент для переменной `IEnumerable<int> values`. Настоящий foreach также поддерживает подходящий pattern GetEnumerator/MoveNext/Current без обязательного объявления интерфейса и имеет специальные варианты для некоторых типов. Основная мысль: он последовательно запрашивает элементы и освобождает перечислитель по правилам конструкции.

Негeneric IEnumerator.Current возвращает object, что может вызвать boxing. Generic-вариант сохраняет статический тип элемента. Перечислитель не обещает потокобезопасность и обычно не допускает произвольного изменения исходной коллекции во время обхода.

Yield создаёт машину состояний

```
static IEnumerable<int> Countdown(int start)
{
    for (int value = start; value > 0; value--)
    {
        yield return value;
    }
}
```

Вызов Countdown(3) ещё не выполняет тело цикла до конца. При MoveNext выполнение продолжается до ближайшего yield return, который выдаёт элемент и приостанавливает метод. Локальные переменные и место продолжения сохраняются сгенерированным объектом. Yield break заканчивает перечисление; достижение конца тела делает то же самое.

`var numbers = Countdown(3);` хранит последовательность. Два foreach обычно запускают два обхода: 3,2,1 и снова 3,2,1. Сам интерфейс IEnumerable не гарантирует повторяемость: пользовательский источник может читать меняющиеся данные или разрешать только один проход.

Когда выполняется проверка

```
static IEnumerable<int> PositiveRange(int count)
{
    if (count < 0)
    {
        throw new ArgumentOutOfRangeException(nameof(count));
    }
    return Iterate(count);

    static IEnumerable<int> Iterate(int count)
    {
        for (int i = 0; i < count; i++)
        {
            yield return i + 1;
        }
    }
}
```

Внешний метод не является итератором: у него нет yield. Поэтому он проверяет count сразу. Если поместить проверку непосредственно в метод с yield, она выполнится только при начале обхода. Разница важна для понятного контракта и выбора границ try/catch.

Методы-итераторы не могут иметь ref/out/in параметры. Yield нельзя поместить в catch или finally, а yield return — в try с catch. Try с finally допустим. Новые версии языка расширяли правила для ref/unsafe рядом с итераторами; их ограничения и сохранение значений через yield изучаются в главе 23.

Ресурс принадлежит перечислению

```
static IEnumerable<string> ReadLines(string path)
{
    using StreamReader reader = new(path);
    string? line;
    while ((line = reader.ReadLine()) is not null)
    {
        yield return line;
    }
}
```

Это обычное текстовое чтение; строгий UTF-8 из L01 при необходимости настраивается дополнительно. Файл открывается при начале обхода. Using остаётся активным между выдачами строк. Когда foreach завершится нормально или через break, Dispose перечислителя выполняет нужное finally и закрывает reader. Если вручную получить перечислитель и забыть Dispose, ресурс может удерживаться.

Опасный вариант — открыть reader в обычном внешнем методе внутри using, вернуть ленивую последовательность на его основе и сразу выйти. К началу обхода reader уже закрыт. Ресурс должен жить столько же, сколько выполняется работа с ним.

Lazy, eager и бесконечные источники

Ленивый источник производит элементы по запросу. Материализация, например `new List<int>(numbers)`, выполняет весь обход и сохраняет результат. Она расходует память, зато даёт собственный набор для повторной работы.

Бесконечный источник можно обработать только с условием остановки. Не пытайтесь материализовать его целиком. Даже ленивый алгоритм может зависнуть, если запрос требует конца источника: например, подсчёт всех элементов.

`IAsyncEnumerable<T>` и await foreach добавляют асинхронное ожидание очередного элемента. Они не означают автоматическую параллельную обработку. Отмена и освобождение через await using рассматриваются после Task в главе 31.

Практика

15-A. Напишите Countdown и выведите первые два элемента через foreach с break. Критерии: для 3 выводятся 3 и 2; для 0 последовательность пуста. Добавьте finally с диагностикой и убедитесь, что оно выполняется при break.

15-B. Сделайте PositiveRange с немедленной проверкой count. Критерий: PositiveRange(-1) бросает исключение до GetEnumerator. Для 3 получаются 1,2,3, для 0 — пустая последовательность.

15-C. Создайте источник со счётчиком выданных элементов. Обойдите его дважды, затем один раз материализуйте и дважды обойдите список. Критерий: в первом случае генерация повторяется, во втором повторно обходятся сохранённые значения. Покажите, где освобождается файл при break в файловом варианте.

Проверка понимания

Чем IEnumerable отличается от IEnumerator? Когда выполняется тело метода с yield? Когда возникнет ошибка чтения файла? Что делает break с using внутри итератора? Гарантирует ли интерфейс, что повторный обход даст те же данные?

Yield

Подсказки · Далее 16

16. LINQ в памяти и в запросах

Результат: вы формулируете преобразование данных, предсказываете момент его выполнения и различаете IEnumerable и IQueryable. Нужны главы 13–15. Запуск: `dotnet run --project examples/TypesAndQueries -- 16`.

Конвейер данных

LINQ связывает операции над последовательностями. Он не заменяет понимание количества проходов, выделения памяти или правил сравнения.

```
int[] values = [5, 1, 4, 2];
IEnumerable<int> query = values
    .Where(static n => n % 2 == 0)
    .Select(static n => n * 10)
    .OrderBy(static n => n);

Console.WriteLine(string.Join(",", query)); // 20,40
```

Where отбирает, Select преобразует, OrderBy упорядочивает. Это extension methods: namespace System.Linq должен быть подключён, в проектах курса он входит в implicit usings. Query syntax выражает часть тех же операций:

```
var query =
    from n in values
    where n % 2 == 0
    orderby n
    select n * 10;
```

Это альтернативное объявление query, а не ещё одна переменная с тем же именем в той же области. Компилятор переводит форму запроса в вызовы методов. Выберите форму, в которой намерение яснее; короткая цепочка не обязана становиться большим query expression.

Когда запрос выполняется

Операция в LINQ to Objects	Выполнение	Что учитывать
Where, Select, Take	Отложенное, обычно по одному элементу	Источник читается при обходе
OrderBy, GroupBy	Отложенное, с накоплением данных	Для полного упорядочения/группировки нужен весь вход
ToList, ToArray, ToDictionary	Немедленное	Создаётся материализованный результат
Sum, Count, Any, First	Терминальное	Возвращает значение; некоторые умеют заканчивать раньше

```
List<int> values = [1, 2];
var query = values.Where(static x => x > 1);
values.Add(3);
Console.WriteLine(string.Join(",", query)); // 2,3
int[] snapshot = query.ToArray();
values.Add(4);
Console.WriteLine(string.Join(",", snapshot)); // 2,3
```

Это отдельный полный фрагмент. Query описывает вычисление, snapshot хранит его результат. Повторное перечисление может повторить чтение файла или работу API. Не помещайте обязательные побочные

действия в Select: оптимизации некоторых терминальных операций способны получить результат без исполнения всех селекторов. Действие выполняйте явным циклом.

Выбор результата

Метод	Пустая последовательность	Несколько совпадений
First	Исключение	Первое
FirstOrDefault	Default или явно заданный fallback	Первое
Single	Исключение	Исключение
SingleOrDefault	Default	Исключение
Any	false	true после обнаружения элемента
All(predicate)	true	Проверка всех до первого false

Single выражает проверяемое обещание уникальности. First не доказывает уникальность. Default может быть нормальным значением: FirstOrDefault для int возвращает 0 и при отсутствии, и когда первым элементом действительно был 0. Уточняйте контракт или выбирайте nullable/result.

Sum для пустого набора decimal даёт 0; Average для пустого набора non-nullable decimal бросает исключение. Aggregate с начальным значением работает и на пустом входе; без него требует хотя бы один элемент. Проверяйте крайние случаи для конкретного overload, особенно при nullable-значениях.

Группировка и сортировка отчёта

```
// entries — IEnumerable<Expense> из L02.
var totals = entries
    .GroupBy(static e => e.Category, StringComparer.Ordinal)
    .Select(static group => new
    {
        Category = group.Key,
        Total = group.Sum(static e => e.Amount.Amount)
    })
    .OrderBy(static row => row.Category, StringComparer.Ordinal)
    .ToArray();
```

Category уже нормализована в модели. Anonymous type здесь удобен для локальной проекции; для публичного результата библиотека использует именованный CategoryTotal. В `e.Amount.Amount` первое Amount — Money расхода, второе — его числовое значение.

ThenBy добавляет вторичный критерий. Новый OrderBy на уже упорядоченном запросе начинает новое основное упорядочение; это частая причина потери первого критерия. Для воспроизводимости задавайте все необходимые критерии, включая порядок при равенстве.

Соединение и плоская последовательность

SelectMany разворачивает последовательность вложенных последовательностей. Например, дни с массивами расходов можно преобразовать в один поток расходов. Join сопоставляет элементы по ключу и возвращает все совпадающие пары; дубликаты ключей могут умножить число строк. GroupJoin возвращает левый элемент вместе с группой совпадений.

Для внешнего соединения нужны также строки без совпадения: классическая форма использует GroupJoin, DefaultIfEmpty и SelectMany. Начиная с .NET 10 существуют также LeftJoin и RightJoin; в старой целевой платформе этих API может не быть. Семантику SQL и null разберём в блоке БД.

Distinct и DistinctBy устраняют повторы по равенству или выбранному ключу; они не доказывают корректность отброшенных повторов. Chunk группирует вход в массивы указанного размера, последний может быть короче. ToDictionary отклоняет повторные ключи, поэтому это не операция группировки.

IEnumerable и IQueryable

Enumerable принимает обычные делегаты и выполняет код .NET над элементами. Queryable часто принимает `Expression<Func<T, bool>>`: дерево описания выражения. Провайдер, например EF Core, переводит поддерживаемые части в запрос к БД.

Не любой C#-метод можно перевести в SQL. AsEnumerable переключает последующие операции на обычное перечисление и не материализует всё немедленно, но может перенести фильтрацию на клиент и увеличить объём получаемых данных. ToList немедленно выполняет запрос. Порядок, сравнение строк и null зависят от конкретного провайдера; опыт LINQ в памяти не заменяет проверку SQL, который реально отправлен серверу.

Практика

16-A. Для 5,1,4,2 получите 20,40; для пустого входа — пустой результат. Сделайте методическую и query-форму. Критерий: одинаковый порядок и значения.

16-B. Для food=12.50, travel=30, food=7.50 получите food=20.00 и travel=30.00. Критерии: порядок категорий явно определён, пустой набор даёт пустой отчёт, повтор food входит в сумму и не теряется через DistinctBy.

16-C. Воспроизведите изменение отложенного результата после Add. Затем поставьте ToArray на нужной границе. Критерий: сохранённый отчёт не меняется. Дополнительно объясните, почему Any()+foreach может дважды обратиться к источнику.

Проверка понимания

Почему наличие var query не означает готовых данных? Чем Single отличается от First? Когда OrderBy начинает накапливать элементы? Зачем указывать comparer? Почему переход на IQueryable меняет ограничения допустимого кода?

Отложенное выполнение · Операции LINQ

Подсказки · Далее 17

17. Шаблоны и расширение синтаксиса

Результат: вы выражаете проверки формы данных через patterns и добавляете удобные операции, сохраняя понятные правила типов. Нужны главы 12–16. Запуск: `dotnet run --project examples/TypesAndQueries -- 17`.

Проверка типа и формы

`value is string text` проверяет тип и при успехе создаёт локальную переменную text. Такой pattern не совпадает с null. `value as string` возвращает строку или null и применим к ссылочным и nullable-типам; `as int` недопустим. Проверка типа не является произвольным числовым преобразованием: упакованный int не совпадает с pattern long.

```
static string Describe(object? value) => value switch
{
    null => "missing",
    int n when n < 0 => "negative integer",
    int => "integer",
    string { Length: 0 } => "empty text",
    string text => $"text length={text.Length}",
    _ => "other"
};
```

Ветки проверяются по порядку. `when` — дополнительное условие. Property pattern сначала требует non-null значение, затем проверяет члены. `is null` проверяет отсутствие ссылки без вызова перегруженного оператора ==.

Relational, logical и вложенные patterns

```
static string Classify(decimal amount) => amount switch
{
    < 0m => "invalid",
    0m => "zero",
    > 0m and < 1000m => "small",
    _ => "large"
};
```

Relational pattern сравнивает с константой. Для изменяемого limit используйте when: `decimal value when value >= limit => ...`. Ключевые слова not, and, or объединяют patterns; они не являются произвольной заменой логических операторов && и || внутри любого выражения. Скобки делают приоритет очевидным.

Для Money из L02 `money is { Amount: >= 1000m }` проверяет значение свойства. У Expense можно написать `{ Amount: { Amount: >= 1000m } }` или extended property pattern `{ Amount.Amount: >= 1000m }`. Не помещайте побочные действия в getter, от вызова которого зависит pattern: порядок вычисления независимых подшаблонов не стоит превращать в бизнес-логику.

Компилятор сообщает о недостижимых ветках и некоторых неохваченных значениях. Резервная ветка `_ => ...` нужна, когда вход шире явно перечисленных случаев. Даже enum может содержать неименованное числовое значение.

Позиционные и списковые patterns

```
// Point — record struct Point(int X, int Y) из главы 12.
bool onVerticalAxis = point is (0, _);
bool startsWithReport = arguments is ["report", var month];
```

Позиционный pattern использует Deconstruct. Для обычного класса можно объявить `void Deconstruct(out int x, out int y)` самостоятельно. Имена локальных переменных при деконструкции не меняют тип

исходного объекта.

List pattern сопоставляет длину и элементы индексируемого значения. В `[1, .., 9]` slice допускает середину произвольной длины; `[.. var rest]` захватывает срез. Это не универсальный способ обработать любой IEnumerable: нужны доступные Count/Length и индексатор либо поддерживаемая compiler pattern форма. Захват среза массива может создавать новую копию.

Extension methods и C# 14

Классическая запись расширения:

```
public static class MoneyPresentation
{
    public static string Display(this Money money) =>
        money.Amount.ToString("F2", System.Globalization.CultureInfo.InvariantCulture);
}
```

Money — тип из L02. Вызов `money.Display()` превращается в статический вызов метода. Расширение не получает доступ к private-состоянию типа и не меняет его виртуальные методы. Если подходящий обычный член экземпляра существует, он имеет приоритет. Выбор extension method зависит от статических типов и доступных namespace.

В C# 14 тот же static-класс может содержать блок:

```
extension(Money money)
{
    public bool IsZero => money.Amount == 0m;
}
```

Этот фрагмент помещается внутрь MoneyPresentation. Теперь доступно `Money.Zero.IsZero`. Extension blocks допускают и статические расширения, а также некоторые операторы. Старый синтаксис с this остаётся полезным для чтения существующих библиотек. Специфичные ограничения и новые варианты языка собраны в главе 24.

Операторы, преобразования и индексаторы

Владелец Money может объявить `public static Money operator +(Money a, Money b)`. Сложение должно сохранять привычный смысл и инварианты. Для точного денежного типа нельзя молча округлять недопустимую дробь только потому, что используется оператор.

Пользовательское преобразование задаётся через `implicit operator` или `explicit operator`. Неявное преобразование уместно, когда потеря информации и неожиданная ошибка не являются обычным результатом. Для потенциально ошибочного ввода понятнее явная фабрика или Try-метод. Перегрузка == требует согласования с Equals/GetHashCode; часть операторов объявляется парами.

Индексатор выражает доступ к элементу:

```
public sealed class Names
{
    private readonly string[] _items = ["Ann", "Bob"];
    public string this[int index] => _items[index];
}
```

У вызова `names[1]` результат Bob. Индексатор может иметь несколько параметров и get/set, но не должен скрывать неожиданно дорогую сетевую операцию за видом простого чтения массива.

Практика

17-A. Реализуйте Describe. Критерии: null → missing, −1 → negative integer, 0 → integer, пустая строка → empty text, "ab" → text length=2. Отдельно проверьте long 1: он должен попасть в other.

17-B. Разберите массив аргументов с patterns: ["report", month] и ["help"]. Критерии: лишний аргумент отклоняется; полученный month затем проходит настоящую проверку формата, поскольку совпадение формы массива не валидирует календарное значение.

17-C. Добавьте Display как extension method и IsZero как extension property C# 14. Критерии: Money(1.5m).Display() → 1.50; default(Money).IsZero → true. Объясните, почему эти расширения не превращают Money в новый подкласс.

Проверка понимания

Когда property pattern исключает null? Чем when отличается от relational pattern? Почему List pattern не заменяет foreach для любого источника? Почему extension method не выполняет виртуальную подстановку? Когда оператор ухудшает понятность API?

Patterns · Extension members

Подсказки · Далее 18

18. Практикум библиотеки предметной области

Результат: правила расходов, контракт хранения и месячный отчёт доступны независимо от Console и файлов. Нужны главы 11–17. Полный договор — в лабораторной L02.

Что меняется после L01

В L01 текстовая строка проходила разбор и участвовала в отчёте. Теперь правила должны использоваться повторно: из консоли, будущего импортёра и веб-сервиса. Текст `12.50` относится к формату ввода, положительность расхода — к предметной модели. Если смешать их в одном методе, каждый новый интерфейс будет копировать проверки.

Добавляется самостоятельная библиотека ExpenseLedger.Core. В ней нет Main: другие проекты вызывают её публичные типы. Классическая структура csproj:

```
<Project Sdk="Microsoft.NET.Sdk" />
```

В примерах курса net10.0 и остальные настройки наследуются от Directory.Build.props. При создании отдельного проекта вне examples используйте `dotnet new classlib --framework net10.0` или явно задайте TargetFramework в собственном csproj.

Зависимости проектов

```
TypesAndQueries (демонстрационный клиент) → ExpenseLedger.Memory → ExpenseLedger.Core
                └──────────────────────────────────────────────────────────┘
CoreChecks (проверки) ──────────────────→ обе библиотеки
```

Стрелка означает ProjectReference. Core объявляет IExpenseStore, потому что описывает операции, нужные сервису. Memory реализует этот контракт через Dictionary. Core не ссылается на конкретное хранилище, Console, чтение файлов или другой клиент.

Такое направление не требует фреймворка. Клиент явно создаёт хранилище и передаёт его в конструктор сервиса. Подмена тестовой реализацией не меняет расчёт.

Типы и их обещания

Тип	Обещание
Money	Неотрицательная величина одной условной денежной единицы, точность до сотых, default=0
Expense	Непустой Id, нормализованная категория, положительная сумма не выше 999999.99
`Result<T>`	Успех со значением либо отказ со стабильным кодом
IExpenseStore	Добавление по уникальному Id и отдельный read-only снимок
LedgerService	Добавление с уведомлением и детерминированный месячный отчёт
CategoryTotal	Неизменяемые данные строки сформированного отчёта

L02 не смешивает разные валюты. Если нужны валюты, контракт Money должен включить валюту, запрет смешения и правила конвертации. Не угадывайте валюту по культуре ОС.

Expense — record class с get-only свойствами. With не должен позволять присвоить недопустимую сумму. Внешний код не получает List, принадлежащий хранилищу. IReadOnlyList сам по себе не гарантирует

неизменность: реализация возвращает обёртку над собственной копией элементов. Поскольку Expense неизменяем, такой снимок достаточен для текущего контракта.

Ожидаемый отказ и исключение

Повторный Id — ожидаемый результат Add: Failure("duplicate_id"). Нарушение контракта конструктора, например отрицательный Money, приводит к исключению. Result не должен заставлять вызывающего молча игнорировать ошибку: сначала проверьте IsSuccess, затем обращайтесь к Value.

Событие Added поднимается после сохранения. На дубле уведомления нет. Если обработчик бросит исключение, добавление уже произошло и отката нет. Не используйте такой event как надёжную очередь бизнес-событий. Проект однопоточный; проверка уникальности нескольких одновременно работающих клиентов — задача более позднего блока.

Построение отчёта

1. Проверьте year/month даже при пустом хранилище.
2. Получите один снимок расходов.
3. Выберите записи нужного месяца и года.
4. Сгруппируйте нормализованные категории, посчитайте число и сумму.
5. Отсортируйте Ordinal и материализуйте готовые строки.
6. Верните отдельное read-only представление результата.

Сумма строится через Aggregate(Money.Zero, ...), поэтому не нужен неявный переход Money в decimal. Группы LINQ в памяти можно перебрать несколько раз: они уже накоплены GroupBy. Повторное перечисление исходного внешнего источника — другое действие с иной стоимостью.

Демонстрация и проверки

```
dotnet run --project examples/TypesAndQueries -- 18
dotnet run --project examples/CoreChecks
python tools/verify.py
```

Ожидаемая демонстрация:

```
duplicate_id
food=20.00 (2)
travel=30.00 (1)
notifications=4
```

Три расхода относятся к январю, четвёртый — к февралю. В отчёте за январь только три. Повторное добавление первого расхода не меняет данные и не увеличивает notifications.

Тесты проверяют не внутренний Dictionary, а обещания интерфейса. Один набор контрактных проверок применяется к двум реализациям: Dictionary-хранилищу и независимому тестовому варианту на List. Это выявляет случайную зависимость сервиса от деталей хранения.

Практика

18-A. Создайте собственные Core, Memory и консольный клиент. Критерий: зависимости направлены к Core, а расчёт не обращается к Console или File.

18-B. Выполните L02 целиком. Критерии: дубликат не перезаписывает данные, снимок и готовый отчёт не меняются после следующего Add, неверный месяц отклоняется даже на пустом входе.

18-C. Добавьте минимальный фильтр по категории в новый метод отчёта. Зафиксируйте, нормализуется ли вход фильтра. Критерии: food/FOOD дают один результат, некорректный фильтр отклоняется, исходный метод сохраняет поведение. Готовый образец реализует основной договор A/B; расширение выполните самостоятельно.

Допуск к следующему блоку

Покажите, где задаётся денежная точность, где определяется уникальность и кто владеет подпиской. Объясните, почему record-равенство не используется вместо ключа Id. Подмените хранилище без редактирования сервиса и сохраните результат всех проверок.

Далее: 19. Сборки и модель исполнения. Продолжение включает CLR, память, reflection, инструменты компилятора и работу со ссылками.

Подсказки · Лабораторная L02

19. Сборки и модель исполнения

Результат: вы объясняете путь исходника до выполнения и находите сборку, тип и IL метода. Нужны главы 11–18. Полный пример — RuntimeLab.

Что получается после build

Компилятор разбирает C#, связывает имена с типами и проверяет правила языка. Обычная сборка .NET содержит IL — инструкции для CLR — и метаданные: имена типов, сигнатуры, ссылки на другие сборки, атрибуты. Файл использует формат PE, в том числе на Linux. Расширение DLL само по себе не говорит, является ли библиотека управляемой.

Assembly — единица поставки и идентичности; namespace — способ группировать имена в исходниках. Два проекта могут объявлять типы в одном namespace. Полное имя типа ещё не задаёт его идентичность: важны сборка и контекст загрузки. Module — часть структуры сборки; обычный SDK-проект создаёт один модуль. Исторические multi-module сборки .NET Framework не нужны для разделения нашего решения: для этого уже есть проекты Core и Memory. Устройство сборок.

В каталоге Release найдите следующие файлы:

Файл	Для чего нужен
RuntimeLab.dll	Управляемая сборка с точкой входа и типами
RuntimeLab.exe на Windows	Apphost, запускающий приложение
RuntimeLab.runtimeconfig.json	Требования и настройки runtime
RuntimeLab.deps.json	Описание зависимостей
RuntimeLab.pdb	Символы для связи выполнения с исходниками

Не переименовывайте DLL зависимостей как способ исправить загрузку: ссылка содержит идентичность, а приложение имеет собственные правила разрешения зависимостей.

Общая система типов

CTS задаёт общие правила типов CLR, поэтому библиотеку C# можно вызывать из F#. CLS — более узкий набор правил для публичного API, удобного разным .NET-языкам. Например, публичный контракт на unsigned-типах не всегда CLS-совместим. В библиотеке можно включить проверку атрибутом сборки; это не означает, что все детали реализации обязаны укладываться в CLS.

```
// Фрагмент отдельной библиотеки, до объявлений namespace/type.
[assembly: CLSCompliant(true)]
```

`internal` ограничивает обычный доступ текущей сборкой. Другой проект не становится «своим» от совпадения namespace. Для тестов иногда объявляют friend assembly:

```
[assembly: System.Runtime.CompilerServices.InternalsVisibleTo("MyLibrary.Tests")]
```

Имя относится к сборке, а не каталогу. Для strong-named сборок нужны дополнительные правила ключей. Friend-доступ открывает все internal-члены указанной сборке; это не механизм изоляции недоверенного кода. Сначала предпочитайте проверки публичного контракта, как в L02.

От IL к машинному коду

```
.cs → компилятор C# → IL + metadata → загрузка CLR → JIT → код процессора
```

JIT компилирует нужные методы во время выполнения. Tiered compilation может сначала создать быстрый в компиляции вариант, затем заменить его оптимизированным. Поэтому «первый вызов» и steady state имеют разную стоимость. Оптимизатор вправе встроить метод и убрать отдельный вызов.

ReadyToRun добавляет заранее скомпилированный код для ускорения старта, но сохраняет роль JIT. Native AOT компилирует приложение при publish под конкретную ОС и архитектуру; JIT при запуске нет. Это меняет допустимость генерации кода и динамической загрузки, а не синтаксис арифметики. Полная AOT-публикация требует платформенного toolchain и отдельной проверки приложения. Текущие примеры собраны как обычные JIT-приложения. Ограничения Native AOT.

Наблюдаемый пример

```
dotnet run --project examples/RuntimeLab -c Release -- 19
dotnet run --project examples/RuntimeLab -c Release -- 19-il
```

Первая команда выводит `RuntimeLab`, `42`, `True`, `7`, `7`, `1` на отдельных строках. Последняя единица — число выполнений статического конструктора Initialized. Вторая команда печатает IL метода Twice и результат его вызова:

```
[System.Runtime.CompilerServices.MethodImpl(
    System.Runtime.CompilerServices.MethodImplOptions.NoInlining)]
public static int Twice(int value) => value * 2;
```

На проверенном SDK Release-байты — `02 18 5A 2A`: `ldarg.0` кладёт аргумент на вычислительный стек IL, `ldc.i4.2` кладёт 2, `mul` снимает операнды и кладёт произведение, `ret` возвращает результат. Вычислительный стек IL — модель инструкций, а не обещание конкретного расположения в памяти процессора. Debug-сборка может содержать дополнительные инструкции.

Посмотреть JIT-код можно в отдельном PowerShell-процессе; настройки действуют только до его закрытия:

```
$env:DOTNET_JitDisasm = "Course.RuntimeLab.RuntimeExamples:Twice"
$env:DOTNET_TieredCompilation = "0"
dotnet examples/RuntimeLab/bin/Release/net10.0/RuntimeLab.dll 19
```

Найдите удвоение аргумента и возврат. Конкретные регистры и инструкции зависят от архитектуры и runtime; в отчёт задачи запишите свою среду. Это диагностический режим, не настройка приложения для production.

Статическая инициализация и версии

Явный статический конструктор выполняется CLR однократно для данного типа; для разных закрытых generic-типов статическое состояние раздельно. Инициализаторы полей работают до его тела. Не стройте взаимозависимые циклы и не выполняйте блокирующие ожидания в таком конструкторе. Ошибка инициализации обычно проявляется через TypeInitializationException, а повторный доступ не является механизмом повторной попытки. Без явного конструктора runtime имеет больше свободы в выборе момента инициализации (beforefieldinit). Статические конструкторы.

Различайте AssemblyVersion (идентичность), FileVersion (версию файла) и InformationalVersion (описание выпуска, часто с commit id). Номер NuGet-пакета — ещё одна величина. Смена версии не чинит удалённый публичный метод: двоичная совместимость требует сохранить контракт. В современной .NET зависимостями управляют приложение и контексты загрузки; правила GAC и binding redirects из книг о .NET Framework нельзя переносить автоматически.

Практика

19-A. Для RuntimeLab и ExpenseLedger.Core выпишите AssemblyName, namespace одного типа, имя модуля и наличие точки входа. Ожидается: у библиотеки нет Main, совпадение namespace не требуется. Критерий: сведения получены через Type/Assembly, а не угаданы по папке.

19-B. Замените умножение в своей копии Twice на сложение двух аргументов. Для входа 21 получите 42, сопоставьте исходник, IL и JIT-код. Критерий: в отчёте есть Release-команда, версия SDK и объяснение различия IL и машинных инструкций.

19-C. Создайте библиотеку с public и internal методами и отдельный клиент. Вызов internal должен дать ошибку доступа; затем добавьте точное имя friend assembly и повторите. Критерий: объяснено, почему одинаковый namespace не помог и почему это не граница безопасности.

Самопроверка

Почему библиотека DLL может содержать IL? Почему два одноимённых типа могут быть разными? Почему NoInlining полезен для нашего наблюдения, но не является общей оптимизацией? Как изменится эксперимент с GetMethodBody при Native AOT?

Назад: 18 · Далее: 20 · Подсказки

20. Память и время жизни

Результат: вы различаете достижимость объекта и владение ресурсом, находите удерживающую ссылку, гарантируете освобождение через using. Нужны главы 09, 12, 14 и 19. Пример — Lifetime.cs.

Где находится значение

Тип значения определяет копирование, но не универсальное правило «struct в стеке». Поле структуры внутри класса находится вместе с объектом; элементы массива структур находятся в массиве. Локальная переменная может быть размещена в регистре, стеке или преобразована компилятором. Boxing создаёт объект с копией значения. Ссылка, лежащая в локальной переменной, сама не является объектом.

Стек вызовов нужен для выполнения методов и возврата из них. Управляемая куча хранит объекты, за достижимостью которых следит GC. Полезная для диагностики схема:

```
живой publisher → делегат события → target подписчика → его данные
```

Подписчик может быть уже не нужен интерфейсу, но всё ещё достижим. Это managed leak: сборщик работает правильно, а программа удерживает лишнюю ссылку.

Что делает GC

GC проходит граф от корней: например, статических полей и живых ссылок выполняющегося кода. Недостижимые циклы сами по себе не мешают сборке. Поколения 0, 1 и 2 позволяют чаще обрабатывать молодые объекты; пережившие сборку могут продвигаться. Это эвристика, не расписание по таймеру. Большие объекты обычно попадают в LOH; ориентир по умолчанию — 85 000 байт размера объекта, а не длины строки. LOH собирается вместе с поколением 2 и обычно не уплотняется автоматически.

При перемещении объекта GC обновляет управляемые ссылки. Pinning запрещает перемещение на время использования фиксированного адреса, но длительные закрепления могут мешать уплотнению. Не вызывайте GC.Collect после каждого запроса: сначала измерьте удержание и выделения. Размер рабочего набора процесса не равен объёму живых объектов. Основы GC.

У ресурса должен быть владелец

GC не знает, когда приложению больше не нужен открытый файл или арендованный буфер. IDisposable даёт явный протокол завершения. `using` вызывает Dispose на выходе, включая return и исключение; несколько ресурсов освобождаются в обратном порядке.

Наш BufferOwner арендует массив и возвращает его в пул. Dispose не уничтожает объект в памяти: последующий доступ к Span отклоняется ObjectDisposedException, а повторный Dispose ничего не делает. Смысл его реализации:

```
// Фрагмент BufferOwner; _buffer — byte[]?, _pool — ArrayPool<byte>.
public void Dispose()
{
    if (_buffer is null) return;
    _pool.Return(_buffer, clearArray: true);
    _buffer = null;
}
```

Rent может дать массив больше запроса. Пользователь видит Span длины Length, а не весь массив. Данные аренды могут быть не нулевыми: инициализируйте их перед чтением. В нашем примере Fill делает это явно. После Return нельзя пользоваться ранее полученным Span: компилятор не отслеживает договор пула. Владелец и потребители должны договориться о времени использования.

Пример рассчитан на один поток. Идемпотентность при последовательных вызовах не означает безопасность одновременных Dispose. Синхронизация рассматривается в главе 32.

Отписка как освобождение

PulseObserver сохраняет publisher и подписывает свой метод OnPulse. В Dispose он снимает именно этот обработчик, затем обнуляет ссылку. Для лямбды пришлось бы сохранить экземпляр делегата: новая похожая лямбда не обязательно снимает прежнюю подписку.

```
dotnet run --project examples/RuntimeLab -- 20
dotnet run --project examples/RuntimeChecks
```

Демонстрация выводит `1`, `0`, `7`: одно полученное событие, ноль оставшихся обработчиков, записанное значение буфера. Повторный Raise после Dispose больше не увеличивает счётчик. Проверки также выбрасывают исключение внутри using: обработчик должен быть снят.

Это воспроизводимая проверка владения. Проверка «объект точно собран через 100 мс» ненадёжна: оптимизации, отладчик и другие ссылки меняют время жизни.

Finalizer, SafeHandle и асинхронное завершение

Finalizer `~Type()` не равен детерминированному деструктору C++. Он нужен как резерв при непосредственном владении unmanaged-ресурсом и усложняет время жизни. Не полагайтесь на его запуск перед завершением процесса и на доступность других финализируемых объектов.

Обычно обёртку ОС представляет SafeHandle: он централизует корректное освобождение дескриптора. Если класс владеет только managed-объектами IDisposable, собственного finalizer обычно не нужно. В наследуемом типе классический Dispose pattern разделяет публичный Dispose и защищённый виртуальный Dispose(bool disposing): managed-поля освобождают при disposing=true. Владение unmanaged-ресурсом в собственном finalizer требует отдельной аккуратной реализации. После нормального освобождения финализацию такого объекта подавляют GC.SuppressFinalize(this). Для sealed-обёртки из нашего примера сложная иерархия не требуется. Реализация Dispose.

IAsyncDisposable возвращает ValueTask из DisposeAsync: завершение может требовать неблокирующего I/O. `await using` ожидает это завершение; обычный using автоматически не заменяет его. Полная модель await и реализация такого владельца появятся в главе 31. Передача ресурса в метод не означает передачу владения: явно указывайте, закрывает ли метод полученный Stream, и учитывайте параметры наподобие leaveOpen.

Слабая ссылка и диагностика

WeakReference не удерживает объект в живых. Сначала вызовите TryGetTarget и используйте полученную сильную ссылку; отдельная проверка «жив ли он» не гарантирует доступность при следующем обращении. Слабая ссылка подходит для необязательного кэша, но не заменяет владение файлом и не чинит бесконечно растущий словарь ключей.

При росте памяти выясняйте: растут ли выделения, живые объекты после сборки, native-память или удержание пула? Для подписчика сначала нарисуйте путь от корня и уберите ненужную подписку. Подробные профилировщики и дампы рассматриваются в главе 35.

Практика

20-A. Подпишитесь на PulseSource внутри using, вызовите Raise дважды, затем ещё раз после выхода. Ожидаются Count=2 и SubscriberCount=0. Критерий: тот же результат при исключении внутри using.

20-B. Напишите владельца аренды на 16 байт с передаваемым тестовым пулом. Ожидаются видимая длина 16, ровно один Return при двух Dispose и запрет нового доступа после Dispose. Критерий: тестовый пул возвращает массив длины 32, чтобы обнаружить выдачу лишнего диапазона.

20-C. Создайте долгоживущий publisher и 100 временных наблюдателей. Сначала оставьте подписки, затем исправьте завершение каждой подписки. Ожидается переход числа обработчиков со 100 к 0. Критерий: объяснён граф удержания, нет GC.Collect в рабочем пути и проверка не зависит от момента финализации. WeakReference можно использовать для отдельного исследовательского опыта, не как таймер теста.

Самопроверка

Почему struct не гарантирует отсутствие выделений? Освобождает ли Dispose память самого объекта? Кто владеет массивом после Return? Почему финализатор подписчика не исправит удержание publisher? Чем идемпотентность отличается от потокобезопасности?

Назад: 19 · Далее: 21 · Подсказки

21. Атрибуты, reflection и dynamic

Результат: вы строите ограниченный валидатор по метаданным и понимаете, какие проверки переносятся с компиляции на выполнение. Нужны главы 13, 16, 19–20. Исходник — MetadataValidator.cs.

Тип, имя и метаданные

`typeof(T)` получает Type по известному типу; `value.GetType()` — фактический тип ненулевого объекта. `nameof(value)` даёт строку имени на этапе компиляции и не читает значение. Поэтому nameof не заменяет GetType, а typeof не создаёт экземпляр.

Reflection позволяет найти свойства и методы, проверить сигнатуры и вызвать их. Атрибут добавляет метаданные, но не выполняет бизнес-правило сам. Наш маркер объявляется так:

```
[AttributeUsage(AttributeTargets.Property,
    AllowMultiple = false, Inherited = true)]
public sealed class NonBlankAttribute : Attribute;
```

У AttributeUsage есть область применения, разрешение повторов и наследование. Имя атрибута при использовании можно сокращать до NonBlank. Аргументы атрибутов ограничены допустимыми константами, typeof и массивами допустимых типов; произвольный объект настроек туда не передаётся. Targets позволяют указать, что аннотируется: например, `[assembly: ...]`, `[return: ...]` или `[property: ...]` на параметре позиционной записи. Атрибуты и reflection.

Контракт небольшого валидатора

Валидируем **объявленный тип T**, а не все члены фактического подкласса. В схему входят публичные свойства экземпляра, помеченные NonBlank. Поддерживаются только string-свойства с публичным getter без индексных параметров. null, пустая строка и пробелы дают имя свойства в списке ошибок; список сортируется ordinal. Неподдерживаемый помеченный член — ошибка схемы, а не ошибка данных.

```
public sealed class ImportOptions
{
    [NonBlank]
    public string? Name { get; init; }
}
```

Алгоритм: получить свойства с BindingFlags.Public | BindingFlags.Instance, отобрать атрибут, проверить форму члена, вызвать GetValue и проверить строку. Эта явная последовательность важна: индексатор нельзя вызвать без индексов, а getter может выполнять код и выбрасывать исключение. Вызов через reflection обычно оборачивает исключение getter в TargetInvocationException; в InnerException находится причина.

```
dotnet run --project examples/RuntimeLab -- 21
dotnet run --project examples/RuntimeChecks
```

Ожидается `Name`, `0`, `GreaterThan`, `True`, `4` на отдельных строках. Первая модель имеет null, вторая — допустимое имя. Это учебная схема для собственных DTO. Не запускайте произвольные getter неизвестного плагина ради «безопасного просмотра»: reflection-вызов остаётся исполнением кода.

Создание объектов и generic-типы

`Activator.CreateInstance<T>()` подходит, если заранее известен T и доступен подходящий конструктор. В простом generic API ограничение `where T : new()` и `new T()` яснее. Для runtime-типа нужны отдельные проверки, доступен ли он, не абстрактен ли, не содержит ли незакрытые generic-параметры, есть ли нужная сигнатура конструктора.

```
// Фрагмент исследовательского JIT-приложения, не AOT-контракт.
Type open = typeof(List<>);
Type closed = open.MakeGenericType(typeof(int));
object list = Activator.CreateInstance(closed)
    ?? throw new InvalidOperationException("No instance");
Console.WriteLine(closed.IsGenericType); // True
```

MakeGenericType проверяет constraints. GetMethod только по имени может быть неоднозначен при overload: выбирайте сигнатуру. Для многократных вызовов сначала измерьте стоимость; кэш MethodInfo или заранее созданный делегат может устранить повторный поиск. При плагинах учитывайте, что кэш Type/MethodInfo может удерживать AssemblyLoadContext.

Что обещает trimming-аннотация

Trimming удаляет недостижимый код при публикации. Если нужный член определяется строкой в runtime, анализатор может не доказать, что его следует сохранить. У Validate параметр типа помечен так:

```
// Часть сигнатуры; using System.Diagnostics.CodeAnalysis.
public static IReadOnlyList<string> Validate<
    [DynamicallyAccessedMembers(
        DynamicallyAccessedMemberTypes.PublicProperties)] T>(T item)
    where T : class
```

Аннотация передаёт требование сохранить публичные свойства T по цепочке вызовов. Она не делает любой reflection автоматически совместимым с AOT. RequiresUnreferencedCode предупреждает о несовместимости с trimming; RequiresDynamicCode — о потребности создавать исполняемый код во время работы. Подавление предупреждения не добавляет отсутствующий код. Подготовка библиотек к trimming.

В этом выпуске проверено поведение в JIT-сборке. Публикация с trimming/Native AOT пока не проверена: для неё нужен отдельный маленький consumer, параметры публикации и тест его готового бинарного файла. Альтернатива для известного набора DTO — прямой валидатор или генерация исходников, как в главе 22.

dynamic, var и анонимные типы

`var` сохраняет статический тип, выведенный компилятором. `dynamic` откладывает разрешение операций до выполнения: строка поддерживает Length, но вызов Missing завершится RuntimeBinderException. Доступность членов продолжает иметь значение. dynamic не превращает объект в словарь и не делает его свойства необязательными. Для данных извне сначала выберите явный DTO либо API чтения данных. Семантика dynamic.

Анонимный тип `new { Category = "food", Count = 2 }` удобен для локальной проекции LINQ. Его свойства доступны только для чтения; форма типа определяется компилятором. Для публичного возвращаемого контракта используйте именованный тип. Возврат object с расчётом на dynamic скрывает договор от вызывающего кода.

Expression trees и информация о вызывающем коде

```
Expression<Func<int, bool>> positive = n => n > 0;
Console.WriteLine(positive.Body.NodeType); // GreaterThan
Console.WriteLine(positive.Compile(preferInterpretation: true)(3));
```

Expression хранит структуру выражения, Func — вызываемый делегат. Дерево можно анализировать и переводить в другой язык; провайдер БД поддерживает только свой набор узлов. Не всякий C#-синтаксис представим в expression tree. Compile не переводит запрос в SQL. В примере выбран интерпретатор; это не доказательство AOT-совместимости всего приложения. Деревья выражений.

CallerMemberName и CallerArgumentExpression подставляют сведения о месте вызова в необязательные параметры во время компиляции. Require из примера использует второй атрибут: при `Require(total > 0)` сообщение содержит текст `total > 0`. Это полезно для диагностики; вызывающий код может передать параметр явно, поэтому это не удостоверение личности.

Практика

21-A. Получите Name, PropertyType и наличие NonBlank у ImportOptions. Ожидаются Name, System.String и True. Критерий: отдельно объяснены typeof и GetType.

21-B. Добавьте DTO с двумя помеченными строками Code и Name. Для null/пробелов получите `Code,Name`, для двух заполненных строк — пустой список. Критерий: порядок устойчив; помеченный int и бросающий getter имеют явно различимую диагностику.

21-C. Напишите прямой валидатор той же схемы без reflection, сохранив результаты B. Критерий: набор входов общий для двух реализаций; в отчёте описаны цена изменения схемы, польза генератора и отдельные проверки, необходимые для AOT. Не называйте обычный build AOT-проверкой.

Самопроверка

Кто выполняет правило атрибута? Почему var не похож на dynamic? Почему typeof(T) и item.GetType() дают разные договоры наследования? Когда reflection вызывает пользовательский код? Что trimming-аннотация обязует сохранить?

Назад: 20 · Далее: 22 · Подсказки

22. Структура исходников и инструменты компилятора

Результат: вы организуете объявления и запускаете собственный анализатор и incremental generator с проверкой полученной сборки. Нужны главы 13, 19 и 21. Полный пример — CompilerLab.

Имена и область видимости

File-scoped namespace `namespace Course.Domain;` действует на оставшуюся часть файла и заменяет внешние фигурные скобки namespace. Не смешивайте обе формы в одном файле. Он не превращает папку в namespace автоматически.

Запись	Действие
using System.Text;	Позволяет использовать короткие имена типов namespace
global using System.Text;	Делает импорт доступным во всей компиляции
using static System.Math;	Позволяет писать Sqrt без имени типа
using Money = Course.Ledger.Core.Money;	Даёт локальный alias, не новый тип
file sealed class Helper	Ограничивает вспомогательный тип текущим файлом
extern alias Legacy;	Различает ссылки на сборки с конфликтующими именами

extern alias требует настроить alias у ProjectReference/Reference; один текст объявления не создаёт ссылку на сборку. Обращение имеет вид `Legacy::Vendor.Type`. Это инструмент интеграции несовместимых API, а не обычная замена namespace. ImplicitUsings SDK создаёт global using в obj: проверьте этот файл, если имя доступно без явного импорта.

partial связывает рукописный и сгенерированный код

Части partial-типа принадлежат одной компиляции и одному типу. Это не способ дописать закрытую библиотеку после её выпуска. В простом генераторе пользователь объявляет часть BuildInfo с вычисляемой Length, а генератор добавляет Title в другую часть.

Partial-метод с необязательной реализацией имеет ограничения: без модификатора доступа, void, без out и ряда модификаторов виртуальности. При отсутствии реализации компилятор удаляет и вызовы, включая вычисление их аргументов. Если нужен обязательный результат, объявите явный доступ и реализацию. С C# 13 есть partial properties/indexers, с C# 14 — instance constructors/events. Их определяющая и реализующая части должны согласоваться; partial не разрешён у static constructor. Правила partial-членов.

В RuntimeLab файлы Modern.cs и Modern.Implementation.cs демонстрируют property, constructor и event. Команда главы 24 проверяет совместное поведение. Выбирайте разделение ради генерации или понятной границы ответственности; дробление каждого метода по файлам не упрощает тип само по себе.

Директивы и документация

Директивы обрабатываются до обычного выполнения. `#define TRACE_INPUT` задаёт символ, а не текстовый макрос. `#if TRACE_INPUT`, `#else`, `#endif` включают разные участки исходника. Символ можно задавать в DefineConstants; DEBUG определяется конфигурацией сборки.

Директива	Когда нужна
#nullable enable/restore	Постепенное включение null-анализа
#pragma warning disable/restore CSxxxx	Узкое обоснованное подавление предупреждения
#line	Отображение сгенерированных строк на исходный документ
#error / #warning	Диагностика недопустимой конфигурации
#region / #endregion	Сворачивание в редакторе, без изменения исполнения

Не прячьте обычное ветвление приложения за #if, если решение должно приниматься по входным данным. После миграции убирайте временные подавления, иначе новый дефект окажется невидимым.

XML-комментарии `/// <summary>...` описывают API; param, returns, exception и cref связывают его с параметрами и типами. GenerateDocumentationFile создаёт XML при сборке. Документируйте правила: владение Stream, допустимость null, порядок результата, исключения. Повторение «возвращает значение» не объясняет контракт.

Syntax tree, semantic model и operations

Roslyn представляет исходник как дерево синтаксиса с токенами и trivia: пробелами и комментариями. Строка `Console.WriteLine` сама по себе не доказывает обращение к System.Console. Имя может быть alias, локальным типом или текстом внутри строки. SemanticModel связывает узлы с символами типов и методов; IOperation даёт представление операции.

Наш DomainConsoleAnalyzer сообщает COURSE001 для вызовов методов System.Console внутри namespace Course.Domain и его подпространств. Он регистрирует OperationKind.Invocation, получает TargetMethod.ContainingType и сравнивает символ с System.Console через SymbolEqualityComparer.Default.

```
// Фрагмент callback анализатора; invocation — IInvocationOperation.
bool isConsole = SymbolEqualityComparer.Default.Equals(
    invocation.TargetMethod.ContainingType, consoleSymbol);
```

Проверки отличают alias и using static от пользовательского класса Console. Course.DomainExtra не входит в Course.Domain. Сообщение указывает на место вызова. Это ограниченное архитектурное правило: оно не ищет доступ к Console.Out как свойству или косвенную передачу I/O через другой сервис.

Анализатор выдаёт диагностику, но не переписывает код. Code fix — отдельный компонент. Работа анализатора может быть конкурентной: не храните изменяемое состояние конкретной компиляции в статических полях.

Incremental generator

BannerGenerator получает один AdditionalText с именем course.txt. Сначала фильтрует входы, затем переводит SourceText в строку и собирает результат. При одном непустом значении выдаёт BuildInfo.g.cs; иначе сообщает COURSE002. Для строкового литерала используется Roslyn SymbolDisplay.FormatLiteral: кавычки, переводы строк и обратная косая черта должны сохранять значение.

Incremental API описывает цепочку преобразований. Хост может повторно использовать неизменившиеся результаты; это не обещание «генератор всегда вызывается ровно один раз». Не используйте DateTime.Now, случайные значения и чтение файлов мимо входов Roslyn. Source generator добавляет новые исходники, но не изменяет существующие. Рекомендации Roslyn.

В примере цепочка маленькая: Collect нужен для проверки единственного файла. Для большого числа независимых элементов сохраняйте поэлементную обработку, где это возможно, и используйте модели с корректным равенством, чтобы изменение одного входа не пересоздавало всё.

Запуск и границы учебного хоста

```
dotnet run --project examples/CompilerLab -c Release
```

В выводе есть `PASS CompilerLab`, сообщение об alias и строка `BuildInfo.g.cs: compiled and executed`. Хост создаёт компиляцию, запускает analyzer/generator, выполняет Emit в MemoryStream, загружает сборку в отдельный AssemblyLoadContext и читает Title/Length. Тест с кавычками и переводом строки проверяет значение, а не только наличие сгенерированного файла. Unload запрашивает выгрузку контекста; её завершение зависит от отсутствия удерживающих ссылок и GC.

Это **самостоятельный учебный хост**, а не устанавливаемый плагин IDE. Он ссылается на Microsoft.CodeAnalysis из выбранного SDK через MSBuildSDKsPath: внешние NuGet-пакеты не нужны, но совместимость связана с SDK. Для распространяемого analyzer/generator обычно создают отдельную библиотеку под поддерживаемый хостами target, фиксируют пакеты Roslyn и подключают её как analyzer. Пример подключения такой отдельной библиотеки:

```
<ItemGroup>
  <ProjectReference Include="../MyGenerator/MyGenerator.csproj"
                    OutputItemType="Analyzer"
                    ReferenceOutputAssembly="false" />
  <AdditionalFiles Include="course.txt" />
</ItemGroup>
```

Не подключайте CompilerLab.exe этим способом: он устроен как демонстрационное приложение. Установка и упаковка расширения для разных IDE в текущий практикум не входят.

Практика

22-A. В копии CompilerLab измените вход на `C# "advanced"`. Критерий: generated source компилируется, а Title в загруженной сборке совпадает с исходной строкой.

22-B. Расширьте COURSE001 запретом чтения Console.Out в Course.Domain. Входы: обычное обращение, alias, свойство одноимённого пользовательского типа и Course.Ui. Ожидается диагностика только в первых двух случаях. Критерий: анализируется символ, а не текст.

22-C. Добавьте в course.txt вторую строку с целым номером редакции 1–999. Ожидаются Title и Edition; пустой/нечисловой/нулевой номер даёт понятную диагностику без исходника. Критерий: проверьте успешный Emit, реальные значения, ошибочные входы и одинаковый результат повторного запуска на неизменных данных. Готовый генератор реализует только исходный договор.

Самопроверка

Почему namespace не равен assembly? Чем partial отличается от наследования? Почему поиск строки WriteLine даёт ложные срабатывания? Что генератор может добавить и чего не может изменить? Почему тест текста не заменяет Emit?

Назад: 21 · Далее: 23 · Подсказки

23. Ссылки, span и unsafe

Результат: вы используете диапазон памяти без лишних копий, соблюдаете время жизни и сравниваете два парсера по одному контракту. Нужны главы 05, 12–13, 19–20. Исходники — PairParser и RuntimeExamples.

Значение и ссылка на ячейку

Обычное присваивание int копирует значение. Ref local становится ещё одним именем той же ячейки. Ref return возвращает доступ к хранилищу вызывающего кода, а не копию результата.

```
int[] numbers = [2, 3];
ref int first = ref RuntimeExamples.FirstRef(numbers);
first = 8;
Console.WriteLine(numbers[0]); // 8
```

Метод FirstRef возвращает `ref values[0]`. Для пустого диапазона обращение недопустимо. `ref readonly` позволяет читать через ссылку, не записывая через неё; это не делает исходный массив неизменяемым. `in` передаёт readonly-ссылку на аргумент, но может потребовать временное значение или защитную копию при вызове изменяющего члена структуры. Не считайте in автоматическим ускорением маленьких значений.

`scoped` ограничивает возможность выхода ссылки/значения за текущую область допустимого времени жизни. В `Sum(scoped ReadOnlySpan<int> values)` метод получает данные на время вызова и возвращает число. Компилятор проверяет escape rules: нельзя вернуть span на локальный stackalloc. Это гарантия времени жизни ссылок в безопасном коде, а не счётчик владельцев. Ref locals и scoped.

Span — представление, а не владелец

Span содержит ссылку на начало и длину непрерывного диапазона. Источник может быть массивом, стековой или unmanaged-памятью. Срез Span меняет представление без копирования элементов; ToArray создаёт отдельный массив. ReadOnlySpan запрещает запись через это представление, но иной владелец всё ещё может менять источник.

Span — ref struct: его нельзя упаковать в object, захватить лямбдой или хранить полем обычного класса. Memory/ReadOnlyMemory можно хранить в объекте и переносить между асинхронными операциями; для короткой обработки берут свойство Span. Memory не продлевает аренду буфера автоматически. С C# 13 некоторые локальные ref struct разрешены в async/итераторах, но нельзя сохранять их через границу приостановки await/yield. Подробная async-модель будет в главе 31. Ограничения ref struct.

Ref struct может реализовать интерфейс, но преобразование в интерфейсную переменную потребовало бы boxing и запрещено. В примере NumberReader передаётся generic-методу с ограничением `where T : INumberReader, allows ref struct`: это разрешение принять ref struct и обязанность соблюдать ограничения его времени жизни внутри метода.

Небольшой стековый буфер и пул

```
Span<int> scratch = stackalloc int[4];
scratch.Clear();
scratch[0] = 2;
scratch[3] = 5;
Console.WriteLine(RuntimeExamples.Sum(scratch)); // 7
```

Для stackalloc с Span не нужен unsafe. До чтения память надо инициализировать. Размер стека ограничен: не выделяйте объём из непроверенного пользовательского ввода, не размещайте stackalloc в длинном

цикле. Для больших данных выбирайте массив или аренду. Правила stackalloc.

ArrayPool уменьшает частоту создания массивов, но вводит протокол Rent/Return. BufferOwner из главы 20 возвращает ровно один массив, не отдаёт запасную часть аренды и запрещает новый доступ после Dispose. Сохранённое ранее представление остаётся ответственностью потребителя. При async-вызове аренда должна жить до завершения потребления. Правила владения Memory.

Контракт парсера важнее реализации

Формат упражнения: два неотрицательных Int32 через один `;`, только ASCII-цифры. Знаки, пробелы, дроби, пустые поля, лишние разделители и переполнение суммы запрещены. При ошибке возвращаются false и sum=0. Ведущие нули допустимы.

TrySumWithSplit создаёт массив строк и подстроки. TrySum работает со срезами входного ReadOnlySpan. Оба используют одинаковую проверку числового поля; длинная сумма вычисляется в long, после чего проверяется диапазон Int32. Нельзя заменить TryParse на Parse и молча изменить обещание «не бросать на ошибочном тексте».

```
dotnet run --project examples/RuntimeLab -c Release -- 23
dotnet run --project examples/RuntimeLab -c Release -- 23-alloc
dotnet run --project examples/RuntimeChecks -c Release
```

Первый вывод: `True`, `42`, `False`, `9`, `7`. Эксперимент выделений сначала прогревает оба пути, затем считает байты текущего потока. Для 10 000 входов `12;30` контрольные суммы должны быть 420000 у обоих. В проверенной среде Split выделил 1 040 000 байт, span-вариант — 0 в измеряемом участке. Точное число не является переносимым контрактом; это не сравнение времени и не полноценный benchmark. Общие аллокации процесса и других потоков этим счётчиком не измеряются.

Указатели и границы unsafe

Unsafe-контекст разрешает операции, которые CLR не может полностью проверить на безопасность. В проекте нужен AllowUnsafeBlocks. Указатель `int*` не равен управляемой ref-ссылке: выход за диапазон или использование после конца закрепления может повредить память.

PointerSum в примере фиксирует массив на время цикла:

```
// Фрагмент unsafe-метода; values — int[], sum — int.
fixed (int* pointer = values)
{
    for (int i = 0; i < values.Length; i++)
        sum = checked(sum + pointer[i]);
}
```

Пустой массив не разыменовывается. Указатель не сохраняется после fixed. Обычный foreach здесь проще; unsafe нужен для понимания interop, а не для улучшения этого суммирования. `sizeof(T)` даёт размер unmanaged-представления допустимого типа; это не размер object в куче. Unmanaged constraint исключает поля с managed-ссылками, но не гарантирует формат сетевого протокола. Unsafe в спецификации.

Function pointer объявляется `delegate* managed<int, int>` либо `delegate* unmanaged[Cdecl]<int, int>`. В отличие от делегата он не хранит захваченный объект. Адрес метода берётся через &. Calling convention должна совпадать с вызываемой функцией. FunctionPointerResult проверяет оба вида: обычный Twice и Triple с UnmanagedCallersOnly. Пример не загружает стороннюю native-библиотеку; её жизненный цикл и ABI будут в главе 55. Указатели на функции.

Inline arrays и переинтерпретация

`[InlineArray(4)]` на структуре с одним полем элемента задаёт встроенное хранилище четырёх элементов. FourNumbers из примера преобразуется в span и суммируется без отдельного массива. Если такая структура является полем объекта, её данные находятся внутри этого объекта. Это ещё один пример того, почему struct не означает «обязательно стек».

MemoryMarshal позволяет получить байтовое представление либо переинтерпретировать допустимые диапазоны. Эти API не проверяют смысл протокола: остаются вопросы порядка байтов, выравнивания, размера, времени жизни и наличия managed-ссылок. Не сериализуйте произвольную структуру как «её байты» для хранения на годы. Для формата явно задавайте поля и endianness; работа с BinaryPrimitives появится в главе 27.

Практика

23-A. Через ref return измените первый элемент `[2,3]` на 8. Затем получите readonly-представление и измените исходный массив. Ожидается, что чтение представления видит изменение. Критерий: объяснена разница read-only и immutable.

23-B. Реализуйте оба парсера договора выше. Входы: `0;0`, `12;30`, `2147483647;0`, `2147483647;1`, `1;2;3`, строка `"\uFF11;2"` с полноширинной цифрой 1. Ожидаемые пары: true/0, true/42, true/2147483647, затем false/0. Критерий: одинаковое поведение в ru-RU, en-US и tr-TR.

23-C. Добавьте поток обработки 10 000 строк, повторно используя арендованный буфер для промежуточных данных. Критерий: результат равен простой реализации, аренда возвращена при исключении, нет чтения после Return. Отдельно сравните выделения после прогрева и запишите ограничения измерения; точное число байт не должно решать, прошёл ли тест корректности.

Самопроверка

Почему нельзя вернуть stackalloc-span? Почему Memory не является владельцем аренды? Что удерживает fixed и что он не проверяет? Почему отсутствие аллокаций не доказывает ускорение?

Назад: 22 · Далее: 24 · Подсказки

24. Версии языка и чтение старого кода

Результат: вы читаете C# разных поколений, выбираете поддерживаемую конфигурацию и обновляете код с сохранением контракта. Нужны главы 01–23. Проверяемые новшества находятся в Modern.cs.

Три независимых настройки

SDK — набор инструментов, выбранный dotnet и global.json. LangVersion ограничивает язык компилятора, TFM (например net10.0) выбирает контракт платформенных API. Установка нового SDK не добавляет API новой платформы проекту с прежним TFM. Некоторые языковые возможности требуют поддержки runtime или библиотечных типов.

В examples закреплены C# 14 и net10.0; global.json допускает обновление стабильного SDK 10.0 в пределах выбранной политики rollForward. Для воспроизведения используйте команды из examples либо tools/verify.py, который выбирает этот рабочий каталог. Значения latest/preview затрудняют повторение чужой сборки: доступный синтаксис зависит от SDK.

Книги по .NET Framework остаются полезны для базовых принципов, но старые AppDomain, GAC, конфигурация приложений и веб-стек имеют другие ограничения. Не заменяйте названия механически.

Карта поколений

Таблица — ориентиры для чтения, а не полный перечень каждого изменения.

Версия C#	Что часто встретится в проекте
1	Классы, интерфейсы, события, делегаты; негенерик-коллекции
2	Generics, nullable value types, yield, partial types
3	LINQ, лямбды, var, anonymous types, extension methods
4	dynamic, optional/named arguments, generic variance
5	async/await, caller information
6	nameof, интерполяция, null-conditional доступ, exception filters
7.x	Tuples, deconstruction, patterns, ref returns, readonly/ref struct
8	Nullable references, async streams, switch expressions, ranges
9	Records, init, top-level statements, расширенные patterns
10	Record structs, global using, file-scoped namespaces
11	Required, raw strings, list patterns, generic math, file types
12	Collection expressions, primary constructors, inline arrays
13	Params collections, ref struct interfaces, partial properties
14	Field-backed properties, extension blocks, partial events/constructors

Проверяйте точные ограничения нужной версии по истории языка. Семантика уже знакомых конструкций важнее запоминания годов выпуска.

Миграция без скрытого изменения поведения

Сначала зафиксируйте входы, результаты, исключения, порядок, побочные эффекты и владение ресурсами. Потом изменяйте одну причину за раз: SDK/TFM, nullable-аннотации, синтаксис, модель данных.

Для nullable-модернизации включите анализ на границе проекта или файла. Аннотируйте допустимый null, исправьте пути инициализации и проверьте входы. `!` подавляет предупреждение, но не проверяет объект во время работы. `required` заставляет вызывающий код инициализировать член в предусмотренных языком сценариях; оно само не валидирует пустую строку и не защищает от произвольной reflection-десериализации.

Замена class на record меняет равенство; замена цикла на LINQ может отложить выполнение; замена массива на collection expression требует подходящего целевого типа. Primary constructor обычного класса не создаёт публичные свойства автоматически. Сохранение поведения надо доказать тестами, а не короткой записью.

Что добавляет C# 13

`params ReadOnlySpan<int>` позволяет вызывать SumParams(2, 3, 4) без обязательного массива в API. Конкретные выделения зависят от формы вызова. Ref struct может реализовывать интерфейс, но не может быть упакован; generic-параметр должен разрешать `allows ref struct`. Partial properties помогают генераторам разделять контракт и реализацию.

`lock` со статическим типом System.Threading.Lock использует его специальный протокол. Преобразование к object меняет выбор механизма, поэтому тип объекта синхронизации важен. Порядок захвата блокировок и взаимные блокировки изучаются в главе 32. Другие изменения включают escape-последовательность `\e`, улучшения вывода method group и правила выбора overload. Изменения C# 13.

C# 14 на запускаемых примерах

Свойство Text в Label использует синтезируемое поле, сохраняя проверку:

```
public string Text
{
    get;
    set => field = string.IsNullOrWhiteSpace(value)
        ? throw new ArgumentException("Text must not be blank.")
        : value.Trim();
} = "untitled";
```

Старую реализацию с явным _text можно заменить такой после проверки инвариантов. Если идентификатор field уже существовал в accessor, используйте явное имя или @field, чтобы устранить неоднозначность.

Null-conditional assignment пропускает вычисление правой части при null:

```
Label? absent = null;
int calls = 0;
absent?.Text = (++calls).ToString();
Console.WriteLine(calls); // 0
```

`nameof(List<>)` возвращает List без закрытия generic-типа. Новые span-преобразования участвуют в выборе overload и выводе типов; после обновления компилятора проверяйте вызовы библиотек с близкими перегрузками. Лямбда `(ref value) => value++` может выводить тип параметра из делегата RefChange.

Extension blocks уже использованы в главе 17; они позволяют группировать extension-члены, в том числе свойства. Они не добавляют полей экземпляра в расширяемый тип. Обзор синтаксиса — C# 14.

Counter демонстрирует instance compound assignment:

```
public sealed class Counter
{
    public int Value { get; private set; }
    public void operator +=(int amount) =>
        Value = checked(Value + amount);
}
```

Здесь counter += 3 меняет существующий объект; другая ссылка alias видит Value=3. Это осознанный контракт изменяемого счётчика, который нельзя без проверки переносить на Money с семантикой значения. Старый путь через operator + мог возвращать новый объект. PartialSample отдельно демонстрирует partial constructor, property и event в двух файлах. Их общий результат проверяется вызовом, подпиской и отпиской.

```
dotnet run --project examples/RuntimeLab -c Release -- 24
dotnet run --project examples/RuntimeChecks -c Release
```

Ожидаемый вывод демонстрации: `ledger`, `List`, `0`, `3`, `5`, `9`, `7`, `6`. Это соответственно проверенное свойство, nameof, пропущенный побочный эффект, +=, ref-лямбда, partial-члены, constrained ref struct и span-вызов.

File-based приложения

Небольшую программу можно выполнить из одного .cs файла. В примере hello.cs директивы `#:property` задают настройки; `#:package` добавляет пакет, `#:sdk` выбирает SDK, `#:project` ссылается на проект. Shebang может использоваться при прямом запуске в Unix. Директивы и возможности SDK нужно сверять с установленной версией: справочник может описывать и preview.

```
dotnet run --file examples/FileBased/hello.cs
```

Ожидается `file-based C# 14`. Пример не требует пакетов и явно отключает PublishAot; публикация file-based приложений по умолчанию имеет особенности, отличные от обычного проекта. Для нескольких проектов, тестов и явных зависимостей наш курс продолжает использовать solution. File-based apps.

Практика

24-A. Для своей среды запишите dotnet --version, TFM, LangVersion и путь global.json. Критерий: объяснено, почему изменение только LangVersion не добавляет отсутствующий API платформы.

24-B. Напишите старый Label с _text и современный с field. Для `"  ledger  "` оба возвращают ledger; пустая строка бросает ArgumentException и сохраняет прежнее значение. Критерий: один набор проверок применяется к обеим реализациям, включая начальное значение.

24-C. Модернизируйте в своей копии один класс L02 и nullable-контракт его клиента. Входы и ожидаемые результаты возьмите из лабораторной: дубликат, пустой набор, неверный месяц. Критерий: прежние проверки проходят, нет массового подавления предупреждений; каждая смена равенства, порядка или владения либо исключена, либо оформлена как отдельное требование.

Допуск к BCL-блоку

Объясните сборку и JIT, найдите владельца ресурса, различите runtime reflection и compile-time generation. Покажите запрет выхода stackalloc-ссылки и тест сохранения поведения при обновлении синтаксиса. Далее: 25. Коллекции и алгоритмы BCL. Блок 25–30 объединяет форматы, HTTP, Host и CLI в лабораторной L03.

Назад: 23 · Подсказки · Ход наполнения

25. Коллекции и алгоритмы BCL

Результат: вы выбираете структуру по операциям, задаёте сравнение и проверяете порядок работы. Нужны главы 07, 12–16 и 23. Примеры нового блока находятся в Integration; первый restore загружает закреплённые пакеты.

Начните с операций

BCL — стандартная библиотека платформы. Коллекция не должна выбираться только по привычке. Сначала перечислите требования: доступ по позиции, поиск по ключу, удаление, сортированный обход, снимки, конкуренция. Запись O(n) означает, что работа растёт примерно с числом элементов; O(log n) — значительно медленнее, O(1) — не зависит от n в рассматриваемой модели.

Структура	Подходящая задача	Стоимость и ограничение
Array / List	Индексация, последовательный проход	Индекс O(1), поиск без индекса O(n)
Dictionary	Получить запись по Id	В среднем O(1); важны хеш и равенство
HashSet	Проверить уникальность	В среднем O(1); нет значений по ключам
SortedDictionary / SortedSet	Упорядоченный обход и обновления	Обычно O(log n) на поиск/вставку
SortedList	Компактный отсортированный словарь	Бинарный поиск; вставка со сдвигом O(n)
Queue / Stack	FIFO / LIFO	Порядок извлечения — часть договора
PriorityQueue	Взять наиболее срочную работу	Вставка/извлечение O(log n), минимум приоритета первым
LinkedList	Вставка рядом с известным узлом	O(1) при наличии узла; поиск узла O(n)

Ёмкость Capacity у List — зарезервированное место, Count — число элементов. Увеличение ёмкости может копировать буфер. Если размер известен, задайте разумную начальную Capacity; не выделяйте память прямо по неограниченному числу из входного файла. Выбор generic-коллекций.

Интерфейс описывает возможности

IEnumerable обещает перечисление, ICollection добавляет Count и операции коллекции, IList — индекс. IReadOnlyCollection и IReadOnlyList запрещают изменение через данный интерфейс, но не гарантируют неизменяемость объекта и не делают снимок.

List.AsReadOnly выдаёт живое представление: изменение исходного List видно потребителю. L02 требовала снимок, поэтому копировала данные. Даже immutable-контейнер не делает изменяемые объекты-элементы глубоко неизменяемыми. Возвращайте наиболее узкий интерфейс, который выражает договор, но не скрывайте стоимость: IEnumerable может означать повторное вычисление или чтение внешнего источника.

Два вида сравнения

IEqualityComparer определяет равенство и хеширование для Dictionary/HashSet. IComparer определяет порядок для сортировки и sorted-коллекций. У SortedSet сравнение, вернувшее 0, означает одинаковый элемент множества, даже если Equals самих объектов возвращает false.

```
var categories = new SortedSet<string>(
    StringComparer.OrdinalIgnoreCase) { "food", "FOOD", "travel" };
Console.WriteLine(categories.Count); // 2
```

Для технических ключей выбирайте явное ordinal-сравнение либо нормализуйте ключ на границе, как в Expense. Не меняйте поля ключа, участвующие в Equals/GetHashCode, пока ключ находится в словаре. Для отображения человеку правило культуры может быть другим. Изменение comparer меняет смысл данных: это отдельное требование, а не косметический рефакторинг.

Очередь с устойчивым порядком

PriorityQueue сама не гарантирует FIFO для одинаковых приоритетов. Чтобы два равноприоритетных задания выполнялись по порядку поступления, добавим sequence. Tuple сравнивается лексикографически: сначала Priority, затем Sequence.

```
var queue = new PriorityQueue<string, (int Priority, long Sequence)>();
queue.Enqueue("later", (2, 0));
queue.Enqueue("first", (1, 1));
queue.Enqueue("second", (1, 2));
while (queue.TryDequeue(out string? job, out _))
    Console.WriteLine(job);
```

Результат: first, second, later. Счётчик sequence принадлежит владельцу очереди; его переполнение и конкурентная выдача номеров требуют отдельной политики. Не изменяйте приоритет поля уже добавленного объекта с ожиданием автоматической перестройки очереди. Договор PriorityQueue.

Поиск, сортировка и неизменяемые данные

Array.Sort/List.Sort меняют контейнер и не обещают устойчивость равных элементов. OrderBy создаёт отложенный LINQ-запрос со стабильной сортировкой. BinarySearch корректен только на данных, отсортированных совместимым comparer. При отсутствии элемента возвращается отрицательное число; побитовое дополнение `~index` даёт позицию вставки, а не просто абсолютное значение результата.

Immutable-коллекции создают новую версию при Add/Remove и могут разделять внутренние структуры. Храните возвращённый результат: вызов Add без присваивания не меняет прежний объект. FrozenDictionary/FrozenSet рассчитаны на построение и последующие частые чтения; стоимость создания нужно окупить. Они не заменяют Dictionary, который постоянно обновляется.

ConcurrentDictionary/ConcurrentQueue предоставляют отдельные потокобезопасные операции. Последовательность «ContainsKey, затем Add» от этого не становится атомарной. Используйте подходящую единую операцию, например TryAdd. Фабрика GetOrAdd может быть вызвана несколько раз при конкуренции: не помещайте в неё необратимую оплату или отправку сообщения. Полная модель потоков появится в главе 32.

Большие числа и generic math

BigInteger хранит целое число произвольной практической длины, ограниченной памятью. Это не decimal для денежных дробей и не бесплатное расширение Int64: арифметика дорожает с ростом числа разрядов.

INumber и более узкие числовые интерфейсы позволяют выразить generic-алгоритм без dynamic. В NumberAlgorithms сумма использует T.Zero и checked-сложение. Для int проверяется переполнение, для BigInteger обычного фиксированного верхнего предела нет. Алгоритм остаётся зависимым от семантики выбранного T: float по-прежнему имеет округление и специальные значения. Generic math.

```
dotnet run --project examples/Integration/BclLab -c Release -- 25
python tools/verify_integration.py
```

После трёх имён очередь выводит `2,3` — длины исходной и новой immutable-версий, `2` — поиск FOOD в frozen-словаре, и `1267650600228229401496703205376` — 2 в степени 100.

Практика

25-A. Добавьте в очередь задачи A(1), B(1), C(0), D(2). Ожидается C,A,B,D. Критерий: порядок равных приоритетов задан sequence, а не случайно совпал.

25-B. Для категорий food, FOOD, travel постройте регистронезависимый индекс количеств. Ожидается два ключа и food=2. Затем создайте неизменяемый снимок. Критерий: последующее обновление исходного словаря не меняет значения снимка.

25-C. Создайте очередь импорта с поиском задания по Guid и выбором по приоритету. Определите договор повторного Id и отмены задания до извлечения. Критерий: тесты покрывают пустую очередь, равные приоритеты, повтор, отмену и 1000 заданий; объяснено, почему одного списка недостаточно для выбранных операций. Многопоточность пока не требуется.

Самопроверка

Когда comparer определяет уникальность? Почему read-only не равен snapshot? Что означает отрицательный BinarySearch? Когда Frozen оправдан? Почему потокобезопасная коллекция не делает всю операцию приложения потокобезопасной?

Назад: 24 · Далее: 26 · Подсказки

26. Текст, время и регулярные выражения

Результат: вы различаете байты, Unicode и элементы текста, не теряете часовой пояс и делаете проверки времени воспроизводимыми. Нужны главы 06, 21–23 и 25. Код — TextAndTime.

От байтов к тексту

Encoding переводит байты в кодовые единицы и обратно. UTF-8 относится к внешним байтам, а string в .NET представлен UTF-16. char содержит одну кодовую единицу, а не обязательно целый символ Unicode. Поэтому Length не является числом видимых символов.

Rune представляет Unicode scalar value; StringInfo работает с текстовыми элементами. Для строки `"\U0001F600"` получаются 2 UTF-16 единицы, 1 scalar и 1 элемент. У `"e\u0301"` — 2 единицы, 2 scalar, но 1 элемент. В методичке используются escape-записи, чтобы пример не зависел от наличия символа в PDF-шрифте.

Normalize(FormC) приводит канонически эквивалентные последовательности к общей форме: `"e\u0301".Normalize()` совпадает с `"\u00e9"`. Нормализация не устраняет все визуально похожие символы разных алфавитов и не заменяет проверку технического идентификатора. Категории расходов остаются ограничены ASCII. Unicode и кодировки.

```
var utf8 = new UTF8Encoding(
    encoderShouldEmitUTF8Identifier: false,
    throwOnInvalidBytes: true);
byte[] bytes = utf8.GetBytes("Привет");
string restored = utf8.GetString(bytes);
```

Строгий декодер отклоняет повреждение вместо тихой подстановки заменяющего символа. При чтении кусками многобайтовая последовательность может пересечь границу буфера: используйте Decoder или StreamReader, а не независимый GetString для каждого блока.

Regex решает задачу формы

Регулярное выражение удобно для локального формата, но не заменяет DateOnly.TryParseExact для проверки существования даты. Якоря `\A` и `\z` означают абсолютные начало и конец; `$` имеет особенности перед завершающим переводом строки. Проверка не должна принимать food с лишним переводом строки, если договор запрещает его.

```
// Внутри partial-класса; метод генерирует компилятор.
[GeneratedRegex("\\A[a-z][a-z0-9_-]{0,31}\\z",
    RegexOptions.CultureInvariant, 100)]
public static partial Regex CategoryPattern();
```

Последний аргумент задаёт timeout в миллисекундах. GeneratedRegex подходит для известного шаблона и позволяет не разбирать его заново при каждом вызове. NonBacktracking выбирает движок с другими ограничениями и линейным характером обработки; не все regex-конструкции, например backreference, им поддерживаются. Не сочетайте режимы в надежде получить все преимущества автоматически. Для сложных шаблонов ограничивайте также размер входа и обрабатывайте RegexMatchTimeoutException. Генератор Regex.

Выбирайте тип времени по смыслу

Тип	Что представляет	Чего не содержит
DateOnly	Календарную дату расхода	Время суток и зону
TimeOnly	Время на часах	Дату и зону
TimeSpan	Длительность/интервал	Календарное правило «следующий месяц»
DateTime	Дату и время с Kind	Полную идентичность часового пояса
DateTimeOffset	Время и конкретное смещение	Правила зоны на будущие даты
TimeZoneInfo	Правила зоны	Единственный момент без даты/времени

Для события, случившегося в конкретный момент, обычно храните UTC или DateTimeOffset. Для «занятие каждый понедельник в 09:00 по местному времени» нужны календарное правило и зона. Дата расхода в L03 — DateOnly: она не сдвигается на предыдущий день при переводе в UTC, потому что вообще не является моментом времени.

DateTime.SpecifyKind меняет метку Kind, но не переводит часы между поясами. Строку обмена разбирайте по явному формату и InvariantCulture; формат `O` полезен для обмена значениями времени с достаточной точностью.

Несуществующее и неоднозначное местное время

При переводе часов вперёд часть местных показаний отсутствует; при переводе назад часть встречается дважды. Нельзя молча угадывать момент. Наш ToInstant принимает DateTimeKind.Unspecified, отклоняет IsInvalidTime и IsAmbiguousTime, затем использует смещение заданной зоны. Для неоднозначного ввода можно запросить явный offset либо применить заранее документированную политику. Неоднозначное время.

Чтобы тест не зависел от установленной базы часовых поясов, TrainingZone задаёт учебные правила: UTC−5 зимой, UTC−4 летом, переходы во второе воскресенье марта и первое воскресенье ноября. Это искусственная зона для упражнения, не актуальный справочник правил реального региона.

В 2026 году 8 марта 02:30 отсутствует, 1 ноября 01:30 неоднозначно. 15 января 12:00 соответствует 17:00 UTC. Реальные идентификаторы и доступность TimeZoneInfo проверяйте на целевой ОС; обновления правил зон требуют обновлений окружения.

Время как зависимость

TimeProvider позволяет передать источник времени вместо скрытого DateTimeOffset.UtcNow. В проверках FixedClock всегда возвращает выбранный момент, поэтому граница истечения проверяется без Thread.Sleep. Договор IsExpired: истекло при `now >= created + lifetime`. Для измерения длительности обработки используйте timestamp/elapsed time, а не разность показаний настенных часов, которые могут быть скорректированы.

TimeProvider не ускоряет любой Task.Delay автоматически. Для управления таймерами нужны API, принимающие соответствующий provider; это будет важно в async-блоке.

Культура, ресурсы и случайность

CurrentCulture отвечает за форматы чисел/дат и некоторые сравнения; CurrentUICulture участвует в выборе локализованных ресурсов. BclLab содержит Messages.resx и Messages.ru.resx. ResourceManager для ru-RU находит ru, а для en-US при отсутствии отдельного ресурса использует базовый. Не используйте отображаемый перевод строки как ключ хранилища.

Guid.NewGuid даёт идентификатор; уникальность данных всё равно подтверждается правилом хранилища. Random подходит для моделирования и несекретной случайности; Random.Shared удобен для общего потокобезопасного генератора. Seed полезен в тесте, но точную последовательность между версиями реализации не считайте форматом хранения. Для секретного случайного значения используйте RandomNumberGenerator, например GetBytes. Идентификатор и секрет — разные договоры. Назначение Random.

```
dotnet run --project examples/Integration/BclLab -c Release -- 26
python tools/verify_integration.py
```

Вывод: True; 2,1,1; True; 2026-01-15T17:00:00.0000000+00:00; True; True. Проверки отдельно проверяют ресурсы, invalid/ambiguous time и точную границу истечения.

Практика

26-A. Сравните Length, Rune-count и StringInfo для `"abc"`, `"\U0001F600"`, `"e\u0301"`. Ожидаются 3/3/3, 2/1/1 и 2/2/1. Критерий: объяснены три разных единицы измерения.

26-B. Импортируйте три местных времени TrainingZone из примера. Ожидается один момент UTC и две отдельные ошибки: nonexistent и ambiguous. Критерий: результат не зависит от локальной зоны компьютера; UTC-Kind на входе отклонён.

26-C. Сделайте локализованный отчёт о сроке действия импорта: сообщения ru/en, срок 10 минут и передаваемый TimeProvider. Через 9 минут 59 секунд после создания ещё действителен, ровно через 10 минут истёк. Критерий: тесты не спят, машинные даты остаются invariant, пользовательский текст берётся из ресурсов.

Самопроверка

Почему char не всегда символ? Что нормализация не исправляет? Почему offset не равен timezone? Почему duration «месяц» нельзя выразить одним постоянным TimeSpan? Как протестировать «истёк ровно сейчас» без ожидания реального времени?

Назад: 25 · Далее: 27 · Подсказки

27. Файлы, потоки и сериализация

Результат: вы читаете ограниченный вход, отделяете формат от модели и сохраняете прежний файл при ошибке до замены. Нужны главы 09, 20–23 и 26. Код — InputFormats.

Stream описывает байты

File и Directory предоставляют операции с файлами и каталогами, Path — операции над путями. Stream представляет последовательность байтов; источником могут быть файл, память, сеть или архив. Не всякий Stream поддерживает Seek, Length и повторное чтение. Проверьте CanRead/CanSeek, когда алгоритм действительно требует этих возможностей.

Read может вернуть меньше запрошенного количества, хотя данные ещё не закончились. Ноль означает конец потока. Для фиксированного заголовка полезен ReadExactly; для неизвестного размера нужен цикл с контролем общего объёма. Буферизация уменьшает количество обращений к источнику, но FileStream и StreamReader уже имеют свои механизмы. Дополнительный BufferedStream не гарантирует ускорение. Файлы и потоки.

Path.Combine не создаёт каталог, не проверяет разрешения и не защищает от выхода за разрешённую папку. Внешний абсолютный компонент может изменить результат комбинации. В L03 пути задаёт пользователь CLI; загрузки в серверный каталог потребуют отдельного договора имён, проверки пути и учёта символических ссылок.

Небольшой мост к async

Сетевые и современные потоковые API часто возвращают Task или ValueTask. Это представление операции, которая может ещё выполняться; await получает её результат, а при сбое распространяет исключение. async позволяет использовать await внутри метода. Само слово async не создаёт отдельный поток.

CancellationToken передаёт запрос отмены. Операция должна наблюдать token; отмена не откатывает уже выполненную запись. await using ожидает DisposeAsync, когда ресурс поддерживает такой протокол. В этом блоке используем последовательные await без параллельного конвейера. Подробная модель выполнения, контексты и композиция задач — в главе 31.

```
// Фрагмент BoundedFiles.ReadPathAsync.
await using var stream = new FileStream(
    path, FileMode.Open, FileAccess.Read, FileShare.Read,
    8192, FileOptions.Asynchronous | FileOptions.SequentialScan);
return await BoundedFiles.ReadAsync(stream, maxBytes, token);
```

ReadAsync из примера проверяет каждый прочитанный блок до добавления в MemoryStream. Content-Length или длина файла — только предварительная подсказка: фактический счётчик остаётся нужен. Метод возвращает byte[], поэтому вход намеренно ограничен, по умолчанию 1 MiB. Потоковую обработку больших наборов добавим в L04. .Result и Wait в этом коде не нужны.

Контракт JSON и проверка модели

Формат L03 содержит version и records; сумму передаём строкой с invariant-точкой:

```
{
  "version": 1,
  "records": [
    {
      "id": "11111111-1111-4111-8111-111111111111",
      "date": "2026-01-10",
      "category": "food",
      "amount": "12.50"
    }
  ]
}
```

System.Text.Json преобразует внешний документ в DTO. Успешная десериализация ещё не доказывает, что дата существует, Id непустой или расход положителен. Documents.TryExpense проверяет поля, затем вызывает конструкторы Money и Expense из L02. RawExpense хранит исходные строки, чтобы неверное значение одной записи не разрушало весь отчёт.

Уровни ошибок различаются. Сломанный JSON, неизвестная версия, повторное имя свойства, неожиданное поле или число вместо строкового amount — ошибка структуры всего документа. Строка amount="1.234", некорректная категория или невозможная дата — ошибка отдельной записи. Номер ошибки — позиция в массиве records, не номер физической строки файла. System.Text.Json.

Явные настройки сериализатора

Контекст ImportJsonContext объявлен partial и помечен JsonSerializable для известных DTO. Генератор создаёт метаданные, а код использует overload с JsonTypeInfo. Это сокращает зависимость от runtime reflection, но само по себе не доказывает AOT-совместимость всего приложения с Host, HTTP и сторонними пакетами. Source generation для JSON.

Настройки примера: camelCase, чувствительность к регистру, глубина до 16, запрет неизвестных и повторных свойств. Required на Version/Records обнаруживает отсутствие поля; явный records=null проверяется отдельно. По умолчанию некоторые из этих ограничений мягче. Параметр AllowDuplicateProperties доступен в выбранной .NET 10 и проверяется тестом.

MoneyJsonConverter отдельно показывает преобразование Money в строку "12.50". Он проверяет TokenType, точность и диапазон decimal, затем создаёт значение. Конвертер выбирается через JsonSerializerOptions.Converters; нельзя считать, что наличие класса конвертера автоматически подключает его ко всем операциям приложения.

Для чтения отдельных полей пригодны JsonDocument/JsonElement; документ нужно освобождать, а элемент связан с его временем жизни, если не сделан Clone. JsonNode удобен для изменяемого дерева; Utf8JsonReader — для последовательного чтения токенов. Выбирайте DTO, когда схема известна, и явно версионируйте изменения формата.

XML, CSV и ZIP

LINQ to XML (XDocument/XElement) позволяет читать и собирать XML с экранированием. Пример использует XmlReader с запрещённым DTD, без XmlResolver и с лимитом символов. Имена с namespace сравниваются как XName; одинаковое локальное имя в разных namespace не означает одинаковый элемент. Схему и обязательные поля проверяют отдельно.

CSV нельзя корректно читать через Split(','): внутри quoted field допустимы запятая, удвоенная кавычка и перевод строки. CsvHelper 33.1.0 читает логические записи и учитывает quoting. В примере закреплены InvariantCulture, разделитель, точный заголовок и число полей; полученные строки всё равно проходят отдельную проверку предметной модели. CsvHelper.

ZIP сжимает байты, но не ограничивает объём после распаковки. OtherFormats читает единственную запись report.txt, не извлекает произвольные пути на диск и ограничивает фактически прочитанный распакованный поток. Для общего архиватора нужны ограничения числа записей, суммарного размера и пути каждой записи.

Небезопасные десериализаторы, восстанавливающие произвольные runtime-типы, не подходят для входа извне. BinaryFormatter не следует включать обходными настройками; современная платформа выводит его из употребления. Используйте явный формат и допустимые DTO. Риски BinaryFormatter.

Замена файла и двоичный формат

Нельзя сначала стереть рабочий файл, а потом пытаться записать новый. BoundedFiles.ReplaceAsync пишет временный файл рядом с назначением, закрывает и сбрасывает поток, проверяет отмену, затем выполняет File.Move с overwrite. При обычном сбое до Move прежний файл остаётся; временный удаляется в finally.

Это не общая транзакция для произвольной файловой системы и потери питания. Две независимые записи «данные» и «checkpoint» также не становятся атомарными. Поэтому L03 хранит и записи, и их Id в одном файле; взаимное исключение обеспечивается отдельной блокировкой. Для строгой устойчивости и нескольких процессов далее понадобятся транзакции БД и эксплуатационные проверки.

Если проект требует двоичного протокола, задайте порядок байтов явно: BinaryPrimitives.WriteInt32BigEndian/ReadInt32BigEndian работают с Span. MemoryMarshal.AsBytes не определяет переносимую схему файла. Длина, версия и каждое поле должны иметь согласованный формат и проверки границ.

```
dotnet run --project examples/Integration/BclLab -c Release -- 27
python tools/verify_integration.py
```

Ожидается: food,office; tea & milk; ledger — по строке на результат CSV, XML и ZIP. Тесты дополнительно моделируют короткие чтения и отменённую замену checkpoint.

Практика

27-A. Прочитайте CSV с полем `"tea,""milk"""` и XML с текстом `tea & milk`. Ожидаются строки с буквальными кавычками и амперсандом. Критерий: разбор выполняют библиотеки формата, а не цепочка Replace.

27-B. Реализуйте JSON round trip одного Expense через DTO версии 1. Проверьте неизвестную версию, duplicate property, records=null, неверный amount и повреждённый UTF-8. Критерий: структурные и построчные ошибки различены, правила Money не обходятся.

27-C. Замените существующий файл экспортом через временный файл. Имитируйте ошибку до замены и заранее отменённый token. Критерий: старые байты сохранены, временные файлы убраны, при успехе повторный разбор даёт те же расходы. Отдельно назовите, какие сценарии аварии эта проверка не доказывает.

Самопроверка

Почему короткое чтение не означает EOF? Кто владеет переданным Stream? Почему валидный JSON может содержать неверный расход? Почему два файла не образуют транзакцию? Что нужно ограничить у ZIP помимо размера архива?

Назад: 26 · Далее: 28 · Подсказки

28. HTTP и сетевые клиенты

Результат: вы получаете данные с ограничением размера и времени, различаете ошибки и проверяете повторы без внешнего сервиса. Нужны главы 20, 26–27. Код — ImportHttpClient.

Договор запроса и ответа

HTTP-запрос имеет метод, адрес, заголовки и иногда тело. Ответ содержит статус, заголовки и тело. GET читает ресурс; POST обычно инициирует действие; PUT и DELETE имеют другие договоры повторения. «Идемпотентный» означает, что повтор приводит к тому же предполагаемому эффекту, а не обязательно к тому же статусу или отсутствию журналирования.

2xx обозначает успешную обработку на уровне HTTP, 3xx — перенаправление, 4xx — проблему запроса или доступа, 5xx — сбой обработки сервером. 200 с HTML вместо ожидаемого JSON не является успешным импортом. EnsureSuccessStatusCode проверяет статус, а Content-Type, структуру и модель проверяет наш код.

Не передавайте секреты в URL: они легко попадают в журналы и историю. В образце нет аутентификации и автоматически не следуются redirects. Разрешены HTTPS и HTTP для loopback-упражнений. Сертификаты HTTPS проверяются стандартным механизмом; проверка сертификата нигде не отключается.

HttpClient и время жизни соединений

Частое создание и уничтожение клиента с собственным handler мешает повторному использованию соединений. Другой крайний случай — бесконечная жизнь соединений без учёта обновления DNS. Обычные подходы: долгоживущий HttpClient с PooledConnectionLifetime или короткоживущие клиенты IHttpClientFactory с управляемым временем жизни handler.

Dispose клиента, созданного фабрикой, не означает немедленного уничтожения общего пула. Не удерживайте typed client в singleton без оценки зависимостей и обновления handler. CookieContainer тоже связан с handler; у нашего клиента UseCookies=false. Правила HttpClient.

В ImportServices зарегистрирован именованный клиент import и typed client ImportHttpClient. Фабрика и DI подробно рассматриваются в следующей главе; сейчас достаточно понимать, что конфигурация транспорта отделена от операции загрузки.

Заголовки готовы — тело ещё читается

```
// Фрагмент DownloadAsync; deadline.Token охватывает весь метод.
using var response = await client.GetAsync(
    uri, HttpCompletionOption.ResponseHeadersRead, deadline.Token);
response.EnsureSuccessStatusCode();
await using var stream =
    await response.Content.ReadAsStreamAsync(deadline.Token);
return await BoundedFiles.ReadAsync(
    stream, options.Value.MaxBytes, deadline.Token);
```

Режим ResponseHeadersRead возвращает управление после получения заголовков. Это позволяет проверять размер и читать поток, но время чтения тела нужно ограничивать отдельно. DownloadAsync создаёт связанный CancellationTokenSource и задаёт deadline на весь метод, включая повторы и чтение тела. Сам HttpClient.Timeout отключён, чтобы не создавать ещё один плохо различимый срок поверх этого договора.

Content-Length проверяется заранее, если он известен; счётчик ReadAsync нужен даже при его отсутствии. Автоматическая распаковка отключена. Для клиента с gzip пришлось бы дополнительно ограничивать декомпрессированный поток и учитывать его стоимость.

Отмена, timeout и классификация ошибок

Событие	Поведение образца
Пользователь отменил операцию	OperationCanceledException, CLI-код 130
Истёк собственный HTTP-deadline	TimeoutException, CLI-код 5
Ошибка сети или неприемлемый статус	Ошибка получения, CLI-код 5
Неподходящий Content-Type или слишком большое тело	Ошибка входа, CLI-код 3
JSON повреждён после успешной загрузки	Ошибка документа, CLI-код 3

Отмена — совместный протокол. Передавайте один token вниз по вызовам, не заменяйте его default при чтении тела. Не ловите OperationCanceledException как обычный повод повторить запрос. Ошибка, возникшая после записи состояния, требует понимать границу фиксации; повторный запуск L03 рассчитан на распознавание уже принятых Id.

Ограниченный повтор

Microsoft.Extensions.Http.Resilience 10.11.0 предоставляет интеграцию с HttpClient и использует Polly. В образце добавлен один standard resilience handler: не более одной повторной попытки, задержка с jitter и отключение повторов unsafe-методов.

```
// Фрагмент настройки AddStandardResilienceHandler.
options.Retry.MaxRetryAttempts = 1;
options.Retry.Delay = TimeSpan.FromMilliseconds(100);
options.Retry.UseJitter = true;
options.Retry.DisableForUnsafeHttpMethods();
```

Стандартный handler включает также rate limiter, timeouts и circuit breaker. В нашем договоре попытка ограничена 5 секундами, pipeline — 15 секундами; общий deadline DownloadAsync может прервать операцию раньше и покрывает чтение тела. На задержку может влиять Retry-After. Нельзя считать «две попытки по 5 секунд» единственным выражением времени всего импорта. HTTP resilience.

Не повторяйте каждый 4xx: неверный URL или токен обычно не исправляются ожиданием. Не повторяйте POST, если сервер не предоставил безопасный договор, например idempotency key. Не складывайте несколько retry-handler: число запросов перемножится. Backoff уменьшает частоту повторов, jitter рассеивает их во времени; circuit breaker временно останавливает обращения к регулярно падающей зависимости. Это разные механизмы, а не разные названия одного retry.

Проверка настоящего транспорта

IntegrationChecks поднимает TcpListener на 127.0.0.1 и свободном порту. Сервер выдаёт заранее заданные ответы, затем освобождается через await using. Он нужен только для тестов и не является шаблоном production HTTP-сервера.

Проверяются 503→200 и ровно два GET, 404 без повтора, POST без повтора при 503, слишком большое тело с Content-Length и без него, неверный тип содержимого, redirect, задержка тела после заголовков и пользовательская отмена. Отдельная проверка CLI использует локальный HTTP-сервер Python.

```
python tools/verify_integration.py
```

Тесты не обращаются к публичному сервису; интернет нужен первому restore пакетов. Чтение JSON и изменение checkpoint проверяются отдельно от транспорта, поэтому сетевой сбой не должен создавать частично принятые расходы.

Слои ниже и другие протоколы

DNS разрешает имя в адреса; TCP даёт поток байтов, TLS защищает соединение и проверяет стороны согласно настройкам. HttpClient работает поверх этих слоёв и сам управляет их деталями. Для постоянного обмена сообщениями может понадобиться ClientWebSocket: тогда нужно учитывать фрагментацию сообщений, закрытие и ограничения размера. Socket/TcpClient нужны при собственном протоколе, где приложение само задаёт framing и границы сообщений. Ни одна операция Receive не обязана вернуть «ровно одно сообщение приложения».

Практика

28-A. Получите версию 1 JSON с локального сервера через ResponseHeadersRead. Критерий: response и поток освобождаются, слишком большое тело отвергается даже без Content-Length.

28-B. Настройте сервер сначала на 503, потом на 200; отдельно на постоянный 404. Ожидаются две и одна попытка соответственно. Критерий: счётчик запросов проверяется сервером, а не выводом клиента; повреждённый JSON после 200 не повторяется автоматически.

28-C. Задержите тело после заголовков и отмените запрос token; затем повторите опыт с deadline. Ожидаются различимые отмена и timeout, checkpoint остаётся прежним. Критерий: тест завершает сервер и освобождает соединения; POST при 503 не повторяется.

Самопроверка

Что не проверяет EnsureSuccessStatusCode? Почему headers timeout не равен сроку чтения тела? Кто владеет handler клиента фабрики? Почему retry может сделать сбой тяжелее? Чем отличается идемпотентность GET от наличия уникального Id у локального расхода?

Назад: 27 · Далее: 29 · Подсказки

29. Host, DI, настройки и журналы

Результат: вы собираете импортёр из зависимостей, проверяете конфигурацию при старте и завершаете его с освобождением ресурсов. Нужны главы 11, 18, 20 и 27–28. Точка сборки приложения — ImportCli/Program.cs.

Где создаются объекты

В L02 LedgerService получал IExpenseStore через конструктор. Это уже dependency injection: сервис использует предоставленную зависимость, а не создаёт её сам. Контейнер DI автоматизирует создание графа объектов, но не заменяет проектирование интерфейсов.

Generic Host объединяет контейнер, конфигурацию, журналирование и управление жизнью приложения. Build создаёт host, StartAsync запускает его службы, StopAsync запрашивает корректное завершение. Dispose освобождает принадлежащие host ресурсы. Для долгоживущей службы удобен RunAsync; наш CLI выполняет одну операцию между Start и Stop. Generic Host.

Composition root — место регистрации зависимостей. Бизнес-правила не должны обращаться к IServiceProvider за произвольным сервисом: конструктор лучше показывает потребности объекта. В текущем решении Core не зависит ни от Host, ни от Serilog, ни от HTTP.

Три времени жизни

Регистрация	Когда создаётся экземпляр	Типичная ошибка
Transient	При каждом разрешении	Считать, что все потребители разделяют одно состояние
Scoped	Один в пределах scope	Получить из root provider и удерживать до конца приложения
Singleton	Один для контейнера	Захватить scoped-зависимость или изменяемое состояние без синхронизации

Singleton, который держит scoped-сервис, создаёт captive dependency. Например, объект обработки одной операции начинает жить весь срок приложения. ValidateScopes и ValidateOnBuild помогают обнаружить такие ошибки, но не доказывают потокобезопасность и корректность всех фабрик.

Контейнер освобождает созданные им disposable-объекты согласно scope. Не вызывайте Dispose чужой внедрённой зависимости ради «очистки после метода». Экземпляр, созданный вручную и переданный в контейнер, имеет отдельный договор владения. Рекомендации DI.

```
// Фрагмент регистрации и выполнения в ImportCli.
builder.Services.AddSingleton(TimeProvider.System);
builder.Services.AddImportHttp();
builder.Services.AddScoped<ImportRunner>();

using var scope = host.Services.CreateScope();
var runner = scope.ServiceProvider.GetRequiredService<ImportRunner>();
```

Scope относится к одному запуску импорта. В BackgroundService, зарегистрированном как singleton, создавайте scope на операцию через IServiceScopeFactory, а не внедряйте scoped Runner напрямую. Подробный фоновый конвейер появится в L04.

Настройки как проверяемый контракт

Host по умолчанию объединяет источники конфигурации. Более поздний источник может переопределить ключ предыдущего. В образце ContentRootPath равен AppContext.BaseDirectory: appsettings.json ищется рядом с собранным приложением и копируется туда при build.

Дополнительно подключён префикс CS_IMPORT_. Переменная `CS_IMPORT_Import__MaxBytes` переопределяет `Import:MaxBytes`; двойное подчёркивание задаёт вложенность. Аргументы CLI разбирает System.CommandLine, они не передаются повторно в Host как произвольные настройки.

```
builder.Services.AddOptions<ImportSettings>()
    .Bind(builder.Configuration.GetSection("Import"))
    .Validate(settings => settings.IsValid(), "Invalid import limits.")
    .ValidateOnStart();
```

Проверяются MaxBytes от 128 до 16 MiB, MaxRecords от 1 до 100000, TimeoutSeconds от 1 до 120. Неверное значение приводит к отказу старта до чтения входа. Значение по умолчанию не должно молча исправлять отрицательный пользовательский лимит. Options pattern.

IOptions представляет настройки для обычного получения значения; IOptionsSnapshot рассчитан на scope, IOptionsMonitor — на наблюдение обновлений. Перезагрузка не делает изменение всех настроек одной транзакцией. Для одного запуска импортёра подходит стабильная конфигурация, а не меняющийся в середине batch лимит.

Секреты и окружение

User secrets предназначены для локальной разработки, хранятся вне репозитория и не являются шифрованным production-хранилищем. Инициализируйте их в собственной копии проекта, когда появится секретный параметр, и подключайте только нужному окружению. В текущем импортёре секретов нет; не добавляйте фиктивные токены в appsettings.

Переменные среды тоже могут быть доступны процессам и инструментам диагностики. Выбирайте production secret store по среде развёртывания, передавайте минимально необходимые значения и исключайте их из журналов. Конфигурация описывает параметры, а не должна содержать произвольный исполняемый код.

Структурированные журналы

```
logger.LogInformation(
    "Import finished: {Added} added, {Failed} rejected",
    report.Added, report.Failed);
```

Added и Failed остаются именованными полями. При интерполяции строки значения заранее смешиваются с текстом, и провайдеру сложнее искать события по полям. Используйте устойчивые шаблоны, уровни и EventId там, где это помогает сопровождению.

BeginScope добавляет контекст batch. Serilog.Extensions.Hosting подключает Serilog к ILogger, а Serilog.Sinks.Console направляет диагностические события в stderr. В stdout остаётся машинный JSON-отчёт либо явно выбранная таблица. В примере нет логирования содержимого входных записей, тела HTTP-ответа или полного URL.

Enrich.FromLogContext переносит свойства scope в событие; outputTemplate определяет, что видно в консоли. В журнале выводится BatchId, чтобы различать операции. Более сложные sinks и централизованное хранение рассматриваются позже. Serilog с Host.

Старт, завершение и границы проверки

Команда создаёт host, проверяет Options, связывает token команды с ApplicationStopping, создаёт scope и выполняет Runner. В finally вызывается StopAsync с отдельным сроком 5 секунд: уже отменённый рабочий token не должен немедленно отменять саму очистку.

Если источник ещё читается, отмена препятствует изменению checkpoint. Если файл уже заменён, корректное восстановление — повторный запуск с дедупликацией. Файл состояния не является транзакцией вместе с выводом в терминал.

IntegrationChecks проверяет время жизни объектов DI и отказ при captive dependency. CLI-проверки передают неверные и корректные значения через environment. Логи и коды завершения проверяются отдельно от бизнес-расчёта. Установка как Windows Service/systemd unit относится к разделу поставки; текущий пример — консольный Host.

```
python tools/verify_integration.py
```

Практика

29-A. Зарегистрируйте три счётчика экземпляров: transient, scoped, singleton. Разрешите их дважды внутри scope и затем во втором scope. Критерий: transient различаются, scoped совпадает только внутри scope, singleton общий.

29-B. Переопределите MaxBytes через CS_IMPORT_Import__MaxBytes. Для 0 и abc ожидается код 2 до изменения checkpoint; для допустимого числа загрузка работает. Критерий: appsettings остаётся неизменным, тест восстанавливает окружение после себя.

29-C. Перенесите запуск в собственную службу IHostedService с одной операцией импорта. Критерий: scoped-сервис создаётся внутри scope, остановка передаёт отмену, диагностические события имеют общий BatchId, ресурсы освобождаются при исключении. Запуск службы ОС и параллельная обработка пока не требуются.

Самопроверка

Чем DI отличается от контейнера? Кто освобождает scoped-объект? Почему singleton не означает потокобезопасность? Когда проверяются Options? Почему рабочий token и срок остановки должны различаться?

Назад: 28 · Далее: 30 · Подсказки

30. Практикум утилиты импорта

Результат: вы поставляете CLI, которая читает файл или HTTP, принимает правильные записи, сохраняет checkpoint и повторяется без дублирования данных. Нужны главы 19–29 и L02. Полный договор — лабораторная L03.

Сборка уже изученных частей

```
System.CommandLine → Generic Host → ImportRunner
                                      ├→ InputFormats → ExpenseLedger.Core
                                      ├→ ImportHttpClient → HttpClient/resilience
                                      └→ checkpoint + export
```

Core по-прежнему отвечает за Money/Expense. InputFormats переводит внешнее представление в DTO, а затем в доменную модель. ImportEngine принимает решения над данными в памяти. Runner координирует I/O и владение ресурсами. Program связывает команды, DI и код завершения.

В готовом примере импортируется JSON версии 1. CSV, XML и ZIP изучены в главе 27 и остаются упражнениями на расширение; их поддержка в CLI не заявляется.

Аргументы как публичный интерфейс

System.CommandLine 2.0.13 разбирает команды import/export, опции, обязательные аргументы и --help. У import должен быть ровно один источник: --file или --url. --state задаёт checkpoint, --pretty включает человекочитаемую таблицу. У export обязательны --state и --output.

```
// Упрощённый фрагмент регистрации, типы System.CommandLine.
var state = new Option<string>("--state")
{
    Required = true,
    Description = "Checkpoint and accepted records"
};
var import = new Command("import") { state };
```

SetAction получает ParseResult и token; GetValue читает типизированную опцию. Дополнительное правило «ровно один источник» проверяется до чтения данных. Не пишите второй самостоятельный парсер args для тех же аргументов. API стабильной версии отличается от старых beta-примеров с SetHandler: ориентируйтесь на закреплённый пакет и работающий Program. System.CommandLine.

Частичный успех и приоритет ошибок

Сначала проверяется весь документ: лимит байтов, JSON-схема, версия и число записей. Структурная ошибка завершает операцию без изменения checkpoint. Затем записи идут по порядку; неправильная получает код и позицию в errors.

Для корректного Expense:

1. Новый Id добавляется в план.
2. Уже известный Id с теми же нормализованными полями считается duplicate.
3. Такой же Id с другими полями даёт id_conflict; прежняя запись сохраняется.

Следовательно, повторное появление Id внутри одного файла учитывается так же, как Id из прежнего checkpoint. Проверка поля предшествует дедупликации: ошибочная запись не становится «хорошим дубликатом» только из-за совпавшего Id.

Checkpoint — подтверждённое состояние

Checkpoint содержит принятую модель, а его Id одновременно служат историей дедупликации. Так не приходится синхронизировать два файла «данные» и «обработанные идентификаторы». Перед загрузкой состояния Runner берёт эксклюзивную блокировку state.json.lock и держит её до завершения фиксации. Конкурирующий запуск завершается ошибкой I/O. Пустой lock-файл остаётся на диске намеренно: важен открытый handle, а не само существование файла.

Изменения сначала вычисляются в памяти. Если превышен лимит итогового состояния, ничего не сохраняется. При наличии новых записей создаётся временный файл и заменяется checkpoint. Если добавлений нет, состояние не переписывается. Это делает повторный запуск проверяемым даже по байтам.

Ошибка записи не должна печатать отчёт об успешной фиксации. Сбой после замены и до печати отчёта возможен: при повторном запуске принятые Id станут duplicates. Это восстанавливаемый договор, а не обещание exactly-once исполнения всех побочных эффектов. В L05 файл будет заменён хранилищем с транзакциями.

Запуск на готовых данных

Команды выполняются из корня репозитория. work предназначен для ваших результатов.

```
$importArgs = @("import", "--file", "labs/03-import-tool/data/mixed.json",
    "--state", "work/l03/state.json")
dotnet run --project examples/Integration/ImportCli -- @importArgs
$exportArgs = @("export", "--state", "work/l03/state.json",
    "--output", "work/l03/export.json")
dotnet run --project examples/Integration/ImportCli -- @exportArgs
```

Первый запуск на отсутствующем state добавляет две записи, находит один duplicate и три ошибочные записи. Код завершения 4 означает частичный успех, а не повреждение checkpoint. Повтор той же команды добавляет 0, находит 3 duplicates и снова сообщает те же три ошибки: неправильные записи не считаются обработанными успешно.

```
{
  "added": 2,
  "duplicates": 1,
  "failed": 3,
  "total": 2,
  "errors": [
    {"record": 4, "code": "invalid_date"},
    {"record": 5, "code": "invalid_amount"},
    {"record": 6, "code": "id_conflict"}
  ]
}
```

JSON в stdout может быть записан одной строкой; сравнивайте структуру, а не отступы. Сообщения ILogger/Serilog идут в stderr. Export проверяет checkpoint и создаёт такой же формат версии 1, пригодный для нового импорта.

UI не должен менять договор импорта

--pretty использует Spectre.Console 0.57.2 для таблицы счётчиков. Коды завершения, записи состояния и решение об ошибках остаются прежними. Для внешнего текста используется Text, а не интерпретация строки как markup. Машинные сценарии должны выбирать обычный JSON-вывод. Spectre.Console Table.

Пока CLI обрабатывает один ограниченный документ в памяти. Нет SQL, авторизации, daemon-режима, расписания и параллельной обработки. Это осознанная граница L03; последующие работы добавят именно эти ответственности.

Практика

30-A. Запустите mixed.json дважды в собственной папке. Ожидаются счётчики 2/1/3/2 и 0/3/3/2, а checkpoint между запусками одинаков по байтам. Критерий: сохранены коды завершения и объяснены все шесть решений по записям.

30-B. Реализуйте весь договор L03 в собственных проектах и экспортируйте состояние. Критерий: импорт экспорта не добавляет дубликатов; повреждённый checkpoint не перезаписывается; stdout остаётся пригодным для JSON-парсера даже при включённом журналировании.

30-C. Добавьте --dry-run с отчётом о предполагаемых изменениях. Для mixed.json ожидаются обычные счётчики, но ни новый, ни существующий checkpoint не изменяется. Критерий: сетевые и форматные ошибки сохраняют классификацию, повтор dry-run воспроизводим, обычный импорт по-прежнему проходит прежние проверки. Готовый CLI эту опцию не реализует.

Допуск к асинхронному блоку

Сдайте L03, объясните точку фиксации и судьбу ресурса при отмене. Покажите тест HTTP без публичного сервера и независимость Core от Host. Далее главы 31–35 добавят композицию задач, конкурентность, ограниченный конвейер и измерения. Состояние наполнения находится в программе.

Назад: 29 · Подсказки · Далее: 31 · Ход наполнения

31. Task и async await

Результат: вы составляете несколько асинхронных операций, наблюдаете их ошибки и отмену, освобождаете ресурсы и объясняете, где нужен поток. Нужны главы 14–15, 20, 27–30. Запускаемый пример — AsyncLab, режим 31; проверки — AsyncChecks.

Операция, поток и ожидание

Task представляет завершение операции: успех, ошибку или отмену. Он не является потоком. Для чтения сокета ОС может сообщить о готовности позже, а поток тем временем обслужит другую работу. Для расчёта хеша процессор действительно выполняет инструкции; async не уменьшает их число.

Нагрузка	Начальный выбор	Что ограничивать
I/O: HTTP, файл, БД	Асинхронный API и await	Одновременные запросы, байты, deadline
CPU: расчёт, преобразование	Синхронный алгоритм; при необходимости Task.Run/Parallel	Занятые ядра и память
UI с долгим CPU-расчётом	Task.Run с кооперативной отменой	Частоту запуска, передачу результата в UI

Не оборачивайте каждый ReadAsync в Task.Run: API уже умеет ожидать без блокировки. На сервере Task.Run для каждой операции не создаёт дополнительной вычислительной мощности: тот же ThreadPool получает ещё одну задачу. Конкурентность означает перекрытие операций во времени, параллелизм — одновременное выполнение на нескольких исполнителях.

Что компилятор делает с await

Обычный вызов async-метода начинает его синхронную часть на вызывающем потоке. Если ожидаемая операция уже завершена, await сразу получает результат. Иначе метод сохраняет состояние, регистрирует продолжение и возвращает управление вызывающему. После завершения операции продолжение возобновляет метод. Локальные переменные, нужные после await, становятся частью состояния. Новый поток при каждом await не создаётся. Модель async.

```
// Самостоятельный фрагмент внутри async-метода.
var ready = new TaskCompletionSource<int>(
    TaskCreationOptions.RunContinuationsAsynchronously);
Task<int> pending = ready.Task;
Console.WriteLine(pending.IsCompleted); // False
ready.SetResult(42);
Console.WriteLine(await pending);       // 42
```

TaskCompletionSource позволяет адаптировать callback/event API: владелец завершает задачу через TrySetResult, TrySetException или TrySetCanceled. Нужно снять подписку и согласовать гонку «результат против отмены». RunContinuationsAsynchronously уменьшает риск исполнения чужого продолжения прямо внутри SetResult и удерживаемой блокировки; это не делает окружающий код потокобезопасным.

await работает по шаблону: GetAwaiter(), IsCompleted, регистрация продолжения и GetResult(). Поэтому допустимы Task, ValueTask и другие awaitable-типы. Собственный awaiter — специальная техника; для приложения почти всегда достаточно стандартных типов.

Исключения и возвращаемый тип

В async Task-методе исключение, в том числе до первого await, сохраняется в возвращённой задаче. Оно наблюдается при await. Обычный метод, который возвращает Task, но не объявлен async, может выбросить исключение прямо при вызове. Синхронная обёртка удобна для немедленной проверки аргументов:

```
// Метод класса; LoadCoreAsync — его асинхронная реализация.
public Task<string> LoadAsync(string path, CancellationToken token)
{
    ArgumentException.ThrowIfNullOrWhiteSpace(path);
    return LoadCoreAsync(path, token);
}
```

Используйте Task для операции без значения, `Task<T>` для результата. async void оставляйте только обработчикам событий, чей делегат требует void: вызывающий не сможет await такую операцию и обработать её исключение как ошибку Task. В обработчике нужна собственная граница ошибок. Отбрасывание задачи через `_ = SaveAsync()` не решает ни владение, ни журналирование, ни остановку.

WhenAll и WhenAny

Последовательные await выполняют шаги последовательно. Чтобы перекрыть независимые I/O-операции, сначала вызовите обе, затем await Task.WhenAll. WhenAll завершается после всех участников; результаты `Task<T>[]` возвращаются в порядке входных задач. Ошибка одной не отменяет остальные. При нескольких сбоях await пробрасывает одно исключение, а Exception агрегирующей задачи содержит все. Договор WhenAll.

```
// tasks — небольшой заранее ограниченный массив Task.
Task all = Task.WhenAll(tasks);
try { await all; }
catch
{
    if (all.Exception is { } aggregate)
        foreach (Exception error in aggregate.Flatten().InnerExceptions)
            Console.Error.WriteLine(error.GetType().Name);
    throw;
}
```

В ObserveAll каждый вызов обёрнут в отдельный async-метод. Результат хранит Value, Error и Cancelled, поэтому сохраняются также синхронные ошибки фабрик. Для трёх операций — результат 7, FormatException и IOException — возвращаются три исхода в исходном порядке. Это удобный договор частичного успеха. Отмена в нём представлена данными и не выбрасывается наружу.

WhenAny возвращает первую завершённую **задачу**, а не её значение. После `Task winner = await Task.WhenAny(...)` нужно `await winner`, чтобы получить значение или исключение. Проигравшие продолжают работать. Если они больше не нужны, отмените их через принадлежащий вам CTS и всё равно дождитесь завершения. WhenAll над миллионом созданных задач не является ограниченным конвейером; к этому вернёмся в главе 33.

Отмена и предел времени

CancellationTokenSource принадлежит инициатору операции, CancellationToken передаётся вниз. Cancel — просьба завершиться, а не принудительное убийство потока. I/O должен получить token; CPU-цикл должен периодически вызывать ThrowIfCancellationRequested. CTS освобождает владелец.

```
// Фрагмент границы операции; callerToken приходит от вызывающего.
using var deadline = CancellationTokenSource.CreateLinkedTokenSource(callerToken);
deadline.CancelAfter(TimeSpan.FromSeconds(5));
try
{
    await DownloadAndReadBodyAsync(deadline.Token);
}
catch (OperationCanceledException) when (!callerToken.IsCancellationRequested)
{
    throw new TimeoutException("Deadline expired.");
}
```

Такой перевод ошибки корректен, если DownloadAndReadBodyAsync использует переданный token и не применяет независимую отмену с другим смыслом. При одновременных причинах здесь приоритет у callerToken. Лимит охватывает отправку, повторы и чтение тела; это уже реализовано в HTTP-клиенте L03.

Task.WaitAsync(token) отменяет **ожидание**, но не исходную операцию. Не освобождайте её Stream/CTS, пока она ещё использует их. При остановке группы нужен общий token, переданный каждой операции, и последующий join через WhenAll. Выход по тайм-ауту не откатывает уже сохранённые данные.

Контекст продолжения и deadlock

UI обычно имеет SynchronizationContext, возвращающий продолжение на UI-поток. Если этот поток блокируется через .Result или .Wait(), а операция ждёт продолжение на нём же, возникает deadlock. Исправление — async до границы события/Main и await вместо синхронного ожидания. GetAwaiter().GetResult() меняет форму пробрасывания исключения, но по-прежнему блокирует поток.

ConfigureAwait(false) сообщает конкретному await, что возвращение в захваченный контекст не требуется. Это полезно в библиотеке, которая не зависит от UI; после такого await нельзя считать себя на UI-потоке. Оно не заставляет код обязательно перейти на другой поток и не отключает ExecutionContext/AsyncLocal. В консольном приложении и ASP.NET Core по умолчанию нет UI-подобного контекста, но блокирующие ожидания всё равно могут истощать ThreadPool. Разбор ConfigureAwait.

Проверка курса использует управляемый QueueContext: продолжение ожидает ручной обработки очереди. Она воспроизводит причину зависания без намеренного вечного deadlock процесса.

ValueTask, async streams и освобождение

`ValueTask<T>` может содержать готовое значение без отдельного Task, ссылку на Task или повторно используемый источник завершения. Это оптимизация для измеренного горячего пути, а не универсальная замена Task. По общему договору экземпляр ожидают один раз. Если нужно повторное ожидание или WhenAll, однократно вызовите AsTask и используйте полученный Task. Не читайте Result незавершённого ValueTask. Ограничения ValueTask.

`IAsyncEnumerable<T>` отдаёт элементы по мере готовности через await foreach. MoveNextAsync возвращает `ValueTask<bool>`; DisposeAsync освобождает перечислитель. В async-итераторе `[EnumeratorCancellation]` связывает параметр token с WithCancellation у потребителя. Само перечисление не запускает обработку элементов параллельно.

```
// rows — IAsyncEnumerable<byte[]>; ProcessAsync обрабатывает одну строку.
await foreach (byte[] row in rows.WithCancellation(token))
    await ProcessAsync(row, token);
```

await using ожидает IAsyncDisposable.DisposeAsync даже при исключении или досрочном выходе. Не заменяйте его синхронным ожиданием DisposeAsync. Span и другие ref struct не могут сохраняться через точку приостановки: держите Memory/массив, а Span получайте внутри синхронного участка. Асинхронные потоки.

Практика

31-A. Выполните `dotnet run --project examples/Integration/AsyncLab -- 31`. Ожидаются `success=1;errors=2`, число 0 и `disposed=True`. Перед запуском объясните порядок; затем замените break на полное перечисление и получите 0, 1, 2. Критерий: finally перечислителя выполняется ровно один раз в обоих случаях.

31-B. Создайте загрузчик трёх небольших HTTP-документов с общим token и результатом для каждого URL. Подмените транспорт: 200 с JSON, 404, исключение I/O. Ожидаются один успех и две отдельные ошибки в порядке входа; публичный сервер для теста не нужен. Ограничьте пакет тремя запросами. Критерий: дождались всех задач и не потеряли ошибку, возникшую до возврата Task.

31-C. Добавьте deadline и отмену пользователя, затем смоделируйте зависшее чтение тела. Ожидаются различимые timeout/cancelled и завершение всех начатых операций до освобождения транспорта. Критерий: тест управляется сигналами, а не надеждой, что произвольный Sleep успеет создать нужную гонку.

Контроль

Почему два последовательных await не создают конкурентности? Кто наблюдает исключение проигравшего WhenAny? Можно ли освободить файл после отмены только WaitAsync? Почему ConfigureAwait(false) не заменяет lock?

Назад: 30 · Далее: 32 · Подсказки

32. Потоки и синхронизация

Результат: вы воспроизводите гонку, выбираете примитив синхронизации и обосновываете владение изменяемым состоянием. Нужны главы 12, 20, 25 и 31. Пример — SharedState, режим AsyncLab 32.

От потока к общему состоянию

Thread — отдельный исполнитель с собственным стеком. ThreadPool переиспользует потоки и обслуживает задачи, таймеры и продолжения. Task.Run обычно отправляет работу в этот пул. Создавать Thread для каждого запроса дорого; отдельный поток оправдан для специального длительного блокирующего цикла или API с привязкой к потоку.

Объекты в куче могут быть доступны нескольким потокам. Даже если чтение/запись поля int атомарны, `value++` содержит чтение, вычисление и запись. Два исполнителя могут прочитать одно значение и записать одинаковый результат: одно обновление потеряется.

```
Поток A: прочитать 0 ──────────────────→ записать 1
Поток B:             прочитать 0 ──────────────────→ записать 1
Итог: 1 вместо 2
```

LostUpdate в примере использует Barrier: оба специально созданных потока сначала читают, затем оба проходят барьер и записывают. Результат воспроизводим без случайного расписания. Барьер служит постановке эксперимента; он не исправляет read-modify-write.

lock защищает инвариант

Защищайте всю последовательность, которая должна наблюдаться как единое изменение. Например, перевод между счетами меняет два остатка; два независимых атомарных присваивания не делают перевод атомарным. Чтение согласованного снимка должно пользоваться тем же протоколом.

```
// Члены класса; System.Threading.Lock доступен в .NET 9+ / C# 13+.
private readonly Lock gate = new();
private int count;

public void Add()
{
    lock (gate) count++;
}

public int Count
{
    get { lock (gate) return count; }
}
```

Если статический тип выражения — System.Threading.Lock, компилятор использует EnterScope/Dispose. Для обычного object lock основан на Monitor.Enter/Exit с try/finally. Не приводите Lock к object: это меняет механизм синхронизации и может нарушить общий протокол. Внутри lock нельзя await. Семантика lock.

Объект блокировки должен быть приватным и стабильным: не this, не typeof(T), не строковый литерал. Удерживайте блокировку коротко. I/O, пользовательский callback и событие внутри неё могут надолго остановить других или войти обратно в объект в момент нарушенного инварианта.

Видимость памяти и атомарность

Компилятор и процессор могут менять порядок независимых обращений к памяти. Без синхронизации другой поток не обязан наблюдать связанные изменения в ожидаемом порядке. Вход и выход из lock задают правила публикации; Interlocked и Volatile имеют собственные гарантии. Потокобезопасность нельзя вывести только из того, что тест несколько раз прошёл.

Средство	Типичная задача	Что оно не обещает
Interlocked.Increment/Exchange/CompareExchange	Атомарная операция над одним значением	Транзакцию над несколькими полями
Volatile.Read/Write, volatile	Упорядоченное чтение/публикация отдельного поля	Атомарность ++, согласованный снимок графа
lock / Monitor	Короткое синхронное изменение инварианта	Ожидание с await
SemaphoreSlim	Ограничение числа входов; WaitAsync	Привязку разрешения к потоку, автоматический Release
Mutex	Исключение между потоками/процессами ОС	Удобное асинхронное ожидание

CompareExchange сравнивает старое значение и при совпадении заменяет его. Цикл CAS должен повторять вычисление при конфликте, не выполнять необратимые эффекты до успешной замены. Volatile.Write флага после подготовки данных и соответствующее Volatile.Read позволяют задать публикацию, но изменяемые опубликованные данные всё ещё требуют отдельного протокола. Наивный spin-loop расходует ядро; для ожидания предпочитайте сигналы или задачи. Volatile, модель памяти runtime.

Ожидание без удержания потока

```
// semaphore создан владельцем как new SemaphoreSlim(4, 4).
await semaphore.WaitAsync(token);
try
{
    await SendAsync(token);
}
finally
{
    semaphore.Release();
}
```

try начинается **после** успешного WaitAsync. Если ожидание отменено до входа, разрешение не получено и Release делать нельзя. Освобождать сам SemaphoreSlim можно, когда все пользователи завершились. Лимит в 4 не ограничивает миллион заранее созданных ожидающих Task: для ограничения очереди нужна схема следующей главы.

Monitor.Wait освобождает монитор и блокирует поток до сигнала с повторным захватом. Условие проверяют в while под тем же монитором; Pulse/PulseAll лишь сигнализирует, что состояние могло измениться. Это протокол condition variable, а не сохранённое сообщение. Для асинхронного обмена удобнее Channel. Mutex имеет владение потоком; при AbandonedMutexException защищённое состояние могло остаться повреждённым.

Deadlock, порядок и повторный вход

Если A держит lock X и ждёт Y, а B держит Y и ждёт X, обе операции остановятся. Задайте единый порядок захвата, например по неизменяемому Id ресурса; все пути обязаны его соблюдать. Другая стратегия — один владелец состояния и передача ему сообщений. Тайм-аут помогает обнаружить ожидание, но не исправляет неверную модель владения.

Monitor и Lock допускают повторный вход того же потока. Это не означает безопасность reentrancy: callback может прочитать объект посередине изменения. Сначала восстановите инвариант, выйдите из блокировки, затем уведомляйте внешних подписчиков. Метод, безопасный для двух потоков, не обязательно безопасен для повторного вызова из самого себя.

ConcurrentDictionary обеспечивает атомарность отдельных операций, например TryAdd. Пара ContainsKey + присваивание не становится общей атомарной операцией. Фабрики GetOrAdd/AddOrUpdate могут вызываться несколько раз и вне внутренней блокировки; не помещайте в них отправку платежа или другое действие, которое нельзя повторить. Неизменяемый снимок убирает гонки чтения самого снимка, но публикация новой ссылки остаётся отдельной операцией.

AsyncLocal и данные запроса

ThreadLocal привязан к физическому потоку: после await работа может продолжиться на другом. AsyncLocal хранит значение в логическом ExecutionContext; оно обычно проходит через await и Task.Run. Так переносят correlation id, культуру и контекст диагностики. Изменение значения в дочернем async-потоке не является способом вернуть значение родителю.

`AsyncLocal<List<T>>` не клонирует список: несколько контекстов могут держать ссылку на один изменяемый объект. Предпочитайте небольшие неизменяемые значения, ограничивайте область через scope и восстанавливайте прежнее значение в finally. ConfigureAwait(false) не подавляет этот перенос контекста.

Практика

32-A. Запустите режим 32. Ожидаются `lost=1`, `atomic=4000`, `locked=4000`. Нарисуйте порядок двух чтений и двух записей в LostUpdate. Критерий: объяснена разница между атомарной записью int и атомарным увеличением.

32-B. Создайте счёт с балансом 100.00 и потокобезопасным TryWithdraw. Две операции одновременно снимают 70.00: ровно одна успешна, итог 30.00. Критерий: проверка достаточности и изменение входят в одну критическую секцию; чтение Balance согласовано с записью; исход проверки не зависит от победившего потока.

32-C. Добавьте TransferTo для двух счетов и встречные переводы по 1.00. У каждого счёта изначально 100.00; после 100 встречных переводов остатки снова 100.00 и 100.00. Проверьте перевод самому себе и недостаток денег. Критерий: единый порядок блокировок, сохранение общей суммы, отсутствие callbacks под lock и завершение теста за заданный предел времени.

Контроль

Почему volatile не исправит LostUpdate? Когда можно Release после отмены? Почему ConcurrentDictionary не позволяет бездумно отправлять HTTP внутри GetOrAdd? Кто владеет состоянием импортёра, если обработчиков несколько?

Назад: 31 · Далее: 33 · Подсказки

33. Параллельные конвейеры

Результат: вы ограничиваете число активных операций, очередь и накопление результатов, сохраняете порядок и завершаете всех участников при сбое. Нужны главы 25, 27 и 31–32. Основной пример — ImportPipeline.

Выбрать уровень управления

Средство	Когда подходит	Что нужно решить отдельно
Task.WhenAll	Небольшой известный набор независимых операций	Отмену группы и размер набора
Parallel.ForEachAsync	Одинаковая обработка большого источника с лимитом	Хранение результатов и порядок
PLINQ / Parallel.For	CPU-вычисления над данными в памяти	Побочные эффекты и стоимость разделения
Channel	Несколько стадий с очередями	Владельцев completion, ошибок и состояния
TPL Dataflow	Граф преобразований и маршрутизации	Лимиты блоков, связь ошибок и завершения

Parallel.ForEachAsync принимает MaxDegreeOfParallelism и token каждой итерации. При лимите 4 одновременно исполняется не более четырёх тел; это не ограничивает размер коллекции, в которую вы складываете все ответы. Результаты могут завершиться в любом порядке. В AsyncLab 33 каждая итерация пишет в свой индекс массива: общего счётчика и конкурирующих Add нет. Parallel.ForEachAsync.

PLINQ делит CPU-запрос между исполнителями. AsOrdered сохраняет логический порядок результата ценой координации; выполнение Select всё равно может идти параллельно. Не меняйте общий List из Select. Не передавайте async-лямбду в синхронный Parallel.ForEach: можно получить async void и потерять ожидание. Для маленькой коллекции обычный цикл нередко быстрее из-за отсутствия накладных расходов.

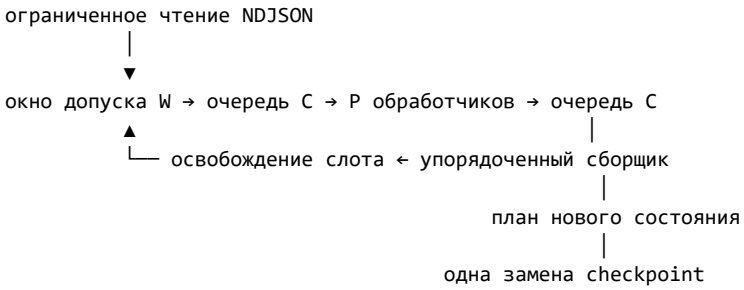
Backpressure — ожидание свободного места

Channel объединяет асинхронные записи и чтения. У bounded channel есть ёмкость. FullMode.Wait заставляет WriteAsync ждать, когда потребитель освободит место: это backpressure. DropOldest/DropNewest/DropWrite допустимы для некоторых потоков телеметрии, но не для финансовых записей. TryWrite возвращает false, если запись сейчас невозможна; игнорировать этот результат нельзя. Channels.

```
// Фрагмент создания очереди; Channel входит в .NET 10.
var queue = Channel.CreateBounded<byte[]>(
    new BoundedChannelOptions(8)
    {
        SingleWriter = true,
        FullMode = BoundedChannelFullMode.Wait
    });
```

SingleWriter/SingleReader — обещания вызывающего о числе одновременных участников, а не блокировки, которые исправят неверное использование. AllowSynchronousContinuations оставляем по умолчанию false, чтобы запись не заставляла неожиданно выполнять код читателя на том же стеке. Reader.ReadAllAsync завершает перечисление после закрытия writer и чтения накопленных элементов. Канал в памяти не является долговечной очередью: при падении процесса его содержимое теряется.

Схема L04



NDJSON содержит один JSON-объект на физической строке. Такой источник можно читать частями, не загружая весь документ. Схема состояния остаётся JSON версии 1 из L03. BoundedLines ограничивает байты строки, всего входа и число строк **во время чтения**. Проверка длины после неограниченного ReadLineAsync происходила бы слишком поздно: огромная строка уже заняла бы память.

Producer присваивает строкам номера. Обработчики независимо разбирают и проверяют строку. Collector единолично меняет словарь расходов и применяет результаты в порядке номеров. Поэтому при конфликте Id выигрывает первая корректная строка входа, а не самый быстрый worker. Domain Expense неизменяем, а изменяемый Dictionary не передаётся обработчикам.

Почему двух ограниченных очередей недостаточно

Представьте, что строка 1 обрабатывается минуту, а строки 2–100000 — быстро. Collector читает их, освобождает очередь и складывает в pending, ожидая 1. Обе очереди имеют размер 8, но pending растёт почти до размера файла.

В примере SemaphoreSlim выдаёт максимум W разрешений. Producer получает разрешение до записи задания; Collector возвращает его только после **упорядоченного применения** результата. Если строка 1 задержалась, дальше W строк не уйдёт. Capacity ограничивает отдельную очередь, Window — весь участок от допуска до применения. Workers ограничивает вычислителей.

При максимальной длине строки L, ограничении состояния S и первых E ошибках память имеет порядок O(W × L + S + E), плюс фиксированные буферы, P задач и представление сохраняемого состояния. Читатель может заранее иметь ещё одну строку и свой буфер на 4096 байт. Это оценка структур, а не точное число байтов GC: есть объекты, строки, таблицы и временная сериализация. Число исходных строк N не создаёт N задач и не добавляет N записей в журнал ошибок.

Малое W может снизить параллелизм, а большое — увеличить память и время ожидания. При W=1 результат остаётся верным и работа становится последовательной. Медленная ранняя запись задерживает применение последующих: это цена детерминированного порядка.

Ошибки, completion и отмена

У каждой очереди должен быть понятный владелец закрытия. Producer закрывает jobs в finally. Координатор ждёт **всех** workers и затем закрывает results. Один worker не должен закрывать общую очередь, пока остальные ещё пишут. Collector читает её до конца и проверяет отсутствие незавершённого pending.

Различаются два класса ошибок:

- Ошибка записи — данные результата: invalid_json, invalid_amount, id_conflict и т. п. Она учитывается в отчёте и не отменяет остальные записи.
- Ошибка инфраструктуры или нарушение лимита — сбой всей операции. Первый такой сбой сохраняется, общий CTS отменяет заблокированные чтения, записи и обработчики. Все участники

ожидаются через WhenAll; затем исходная ошибка пробрасывается наружу.

Если бы код только ждал WhenAll, упавший consumer мог бы оставить producer навсегда заблокированным на заполненной очереди. Если бы он сразу выбросил исключение, оставшиеся задачи могли бы продолжить использовать уже освобождённые Stream и SemaphoreSlim. Нужны обе части: отменить и дождаться. При одновременном сбое и внешней отмене этот пример отдаёт приоритет сохранённой инфраструктурной ошибке.

Для CPU-парсинга запускается фиксированное число Task.Run, а не задача для каждой строки. Не создавайте Task.Run с уже отменённым token для стадии, чей finally обязан закрыть очередь: делегат может вообще не начаться. В примере отмена передаётся внутрь участника.

Повтор, poison record и завершение потока

IRowProcessor должен быть безопасен для параллельного вызова и повторения. Только явно обозначенный TransientWorkException повторяется до Attempts, с задержкой и тем же token. После исчерпания попыток строка получает retry_exhausted. JSON-парсер не выбрасывает эту ошибку: неправильные данные повторять бессмысленно; проверки используют управляемую подмену обработчика. Живой сервис должен добавить backoff/jitter и учитывать суммарный deadline из главы 28.

Нормальный EOF закрывает вход и позволяет очередям опустеть. Внешняя отмена в L04 прерывает текущую работу и отменяет весь ещё не сохранённый план. Это разные протоколы завершения. Для drain при остановке daemon нужны два сигнала: прекратить приём новых заданий, затем до отдельного срока дождаться принятых; после срока — abort. Не используйте один уже отменённый token для обоих шагов.

Периодическая работа и TPL Dataflow

PeriodicTimer позволяет одному циклу ждать тиков через WaitForNextTickAsync(token). Если тело заняло дольше периода, тики объединяются, а не создают неограниченную очередь запусков. Одновременно ждать следующий тик должен один потребитель. await тела внутри цикла предотвращает наложение запусков; обычный callback Timer может запускаться повторно до завершения предыдущего. PeriodicTimer.

TPL Dataflow предлагает TransformBlock, ActionBlock и связи между ними. BoundedCapacity ограничивает принятые сообщения, MaxDegreeOfParallelism — работу блока, EnsureOrdered — порядок выходов. SendAsync нужен, когда следует ждать место; Post может отказать. PropagateCompletion передаёт завершение вниз по связи, но не заменяет общий протокол отмены всего графа. Это обзор альтернативы; L04 реализована на Channels и отдельного Dataflow-приложения не содержит. TPL Dataflow.

Практика

33-A. Выполните AsyncLab 33. Ожидаются две строки `0,1,4,9`. Измените лимит Parallel.ForEachAsync с 2 на 1: значения должны сохраниться. Критерий: объяснено, почему порядок массива определяется индексом, а не очередностью завершения.

33-B. Реализуйте producer/consumer для 1000 чисел: C=2, P=3, результат — квадрат каждого числа. Проверьте ровно 1000 результатов, отсутствие повторов/пропусков и остановку при ошибке consumer. Критерий: нет Task на каждый элемент; исключение не оставляет producer в ожидании заполненной очереди.

33-C. Добавьте восстановление порядка и окно W=4; удерживайте результат 1 управляемым сигналом, дайте 2–4 завершиться. До освобождения 1 нельзя допустить 5; после сигнала все 1000 чисел проходят. Критерий: тест проверяет предел по счётчику допуска, отмену при заполненных очередях и join участников.

Назад: 32 · Далее: 34 · Подсказки

34. Измерение и оптимизация

Результат: вы находите затратный участок по наблюдениям, меняете его и подтверждаете сохранение результата. Нужны главы 19–20, 23, 25 и 31–33. Примеры — AmountAlgorithms, PerfLab и нагрузочный эксперимент.

Сначала вопрос, затем инструмент

«Медленно» нужно превратить в измеримое утверждение: запуск CLI занимает 2 секунды, p95 HTTP-ответа превышает 300 мс, либо импорт не успевает за поступлением записей. Сравнивайте одинаковые данные, результат, режим сборки, runtime и ограничения нагрузки. Изменение алгоритма, которое ускоряет программу за счёт пропуска проверки, не является эквивалентной оптимизацией.

Метрика	Что означает	Частая ошибка
Latency	Время отдельной операции	Забыть время ожидания в очереди
Throughput	Завершённых операций за интервал	Считать начатые вместо завершённых
p50/p95/p99	Квантили распределения задержек	Выводить p99 по трём запросам
Allocated bytes	Выделения управляемой памяти	Путать с размером живой кучи
Working set	Физическая память процесса	Приписать весь рост утечке объектов
CPU time / wall time	Работа процессора / прошедшее время	Назвать любое ожидание загрузкой CPU

Например, при последовательной регистрации 1000 задержек p95 — значение, ниже которого примерно 95% наблюдений; нужно указать способ расчёта и учитывать незавершённые/отменённые запросы. Система под перегрузкой может иметь быстрые успешные ответы и огромную очередь отклонённых: одно среднее скроет проблему.

Release, прогрев и воспроизводимость

Debug, подключённый debugger, первый JIT, загрузка сборок, tiered compilation, GC и файловый кэш влияют на измерение. Для устойчивого сравнения используйте Release, несколько итераций и описание среды. Прогрев измеряет установившийся режим; cold start надо измерять отдельно в новом процессе. Stopwatch подходит для длительности целого сценария, а не для одного слишком короткого вызова без контроля шума.

Не используйте GC.GetAllocatedBytesForCurrentThread для асинхронного метода с переходом между потоками: разность может относиться к разным исполнителям. Рабочие метрики и профили должны соответствовать вопросу. Не добавляйте GC.Collect в обычный код приложения «для уменьшения памяти» без подтверждённой причины.

BenchmarkDotNet

В PerfLab закреплён BenchmarkDotNet **0.15.8**. Он создаёт отдельный процесс, подбирает число повторений и отделяет прогрев от измерений. MemoryDiagnoser показывает выделения. GlobalSetup подготавливает данные вне измеряемого участка; Benchmark возвращает результат, чтобы измерялась полезная работа. Baseline задаёт основу сравнения, Params — размеры входа. Запуск BenchmarkDotNet.

Здесь сравнивается извлечение последнего поля уже проверенной строки `2026-01-10;food;12.50`: Split создаёт массив и подстроки; Span берёт срез существующей строки. Оба варианта используют decimal и invariant culture. Это не замена полного парсера L01: формат и бизнес-ограничения считаются проверенными до горячего участка.

```
# Из корня; предварительно соберите Integration.slnx в Release.
Push-Location examples/Integration
try {
    $benchArgs = @("--filter", "*AmountBenchmarks*", "--job", "short",
        "--artifacts", "../../.artifacts/benchmarks-05")
    dotnet run --project PerfLab -c Release --no-build -- @benchArgs
} finally { Pop-Location }
```

ShortRun удобен для проверки процедуры, но недостаточен для точных обещаний скорости. Для серьёзного сравнения уберите `--job short`, выполните несколько независимых запусков и сохраните полные отчёты. Время измерений не должно быть порогом обычного CI-теста.

Реальный опыт этого выпуска

22 октября 2026: Windows 11 x64, i5-11400H, SDK 10.0.402, runtime 10.0.13, Release, Workstation GC; ShortRun: один запуск, три прогрева и три измерения. Одной операцией считается сумма **всего массива**, а не одной строки.

Метод	Строк	Mean, мкс	Error, мкс	Выделения на операцию
Split	100	9.322	10.860	16000 B
Span	100	4.228	3.919	Не обнаружены
Split	1000	93.621	64.332	160000 B
Span	1000	46.267	1.544	Не обнаружены

Error — половина 99.9% доверительного интервала BenchmarkDotNet, а не максимальная ошибка программы. Интервалы короткого опыта широкие; эти цифры не доказывают универсальное ускорение в два раза. Наблюдение по выделениям объясняется устройством алгоритмов. Равенство сумм отдельно проверено для пустого входа, граничных сумм и трёх культур. Сохранённый CSV.

От процесса к участку кода

dotnet-counters показывает динамику runtime-метрик: CPU, heap, GC, ThreadPool, исключения. dotnet-trace собирает события и выборки стеков; профиль помогает найти кандидата на оптимизацию. dotnet-dump сохраняет состояние для анализа удерживаемых объектов и стеков; это снимок, поэтому он сам по себе не измеряет длительность метода. Counters, trace, dump.

В этом выпуске инструменты версии **10.0.750501** установлены только в .artifacts/diagnostics. Следующие команды PowerShell выполняются из корня после сборки примеров:

```
$nuget = "examples/Integration/NuGet.Config"
$installArgs = @("dotnet-trace", "--version", "10.0.750501",
    "--tool-path", ".artifacts/diagnostics", "--configfile", $nuget)
dotnet tool install @installArgs
$trace = ".artifacts/diagnostics/dotnet-trace"
$demo = "examples/Integration/AsyncLab/bin/Release/net10.0/AsyncLab.dll"
$collectArgs = @("collect", "--profile", "dotnet-sampled-thread-time",
    "--output", ".artifacts/split.nettrace", "--", "dotnet", $demo,
    "34", "split", "5")
& $trace @collectArgs
& $trace report .artifacts/split.nettrace topN -n 8
```

Если инструмент уже установлен по этому пути, повторять install не нужно. В версии 10 прежнее имя cpu-sampling не подходит для обычного collect на Windows; список проверяется через list-profiles. dotnet-sampled-thread-time оценивает время по выборкам управляемых стеков, включая ожидания: его проценты нельзя автоматически называть процентами CPU.

В собранном профиле SplitInternal занял 19.78% exclusive, SplitSum — 64.11% inclusive. Также видны разбор decimal и ожидание GC. Inclusive включает вызванные методы, exclusive относится к самому кадру. Это объясняет выбор кандидата Split для отдельного измерения. Сохранённый topN.

Для counters установите аналогично dotnet-counters той же версии и запустите:

```
$counters = ".artifacts/diagnostics/dotnet-counters"
$counterArgs = @("collect", "--format", "csv", "--output",
    ".artifacts/span-counters", "--counters", "System.Runtime",
    "--", "dotnet", $demo, "34", "span", "5")
& $counters @counterArgs
```

Сбор CSV проверен. Для живого процесса можно использовать monitor --process-id с его PID. Для dump после аналогичной установки: `dotnet-dump collect --process-id <PID>` и `dotnet-dump analyze <путь>`; внутри анализатора начните с help, clrstack, dumpheap -stat, затем gcroot конкретного удерживаемого объекта. Угловые скобки здесь — места подстановки, не буквальные аргументы. Снятие и анализ dump в этом выпуске не выполнялись. Dump может содержать входные данные и секреты процесса; храните его как диагностический артефакт с ограниченным доступом.

Что исправлять по симптомам

При высоком CPU ищите горячие стеки и алгоритмическую сложность. При частых GC и большом allocation rate ищите временные строки, boxing, замыкания и материализацию LINQ. При низком CPU, растущем числе потоков и очереди ThreadPool проверьте блокировки .Result/Wait и синхронное I/O: добавление потоков может только отсрочить starvation. Диагностика starvation.

ArrayPool требует возврата массива в finally и запрета использования после возврата. Span уменьшает копирование, но не владеет памятью. SIMD обрабатывает несколько чисел за инструкцию, однако требует подходящего алгоритма, обработки остатка и учёта поддержки оборудования. Не применяйте их до обнаружения соответствующего узкого места. Начните с удаления повторной работы и выбора подходящей структуры данных; только затем меняйте горячий участок.

Практика

34-A. Запустите PerfLab и объясните Mean, Error, Allocated для Count=100 и 1000. Критерий: записаны среда и команда, проверено равенство сумм; нет требования совпасть с таблицей по времени.

34-B. Соберите профиль режима `34 split 5`, найдите SplitInternal и сравните со `34 span 5`. Ожидается исчезновение разделения строки из горячего пути. Критерий: различены inclusive/exclusive, выборки стеков и точное CPU-время; ускорение подтверждается отдельным benchmark, а не одним процентом.

34-C. Выполните `python tools/measure_pipeline.py`: по 10000 записей, один прогрев и три запуска при P=1 и P=4, C=8, W=16. Скрипт проверяет отчёт и пишет время всего процесса в .artifacts/pipeline-load.json. Критерий: сравниваются медиана, разброс и правильность; холодный запуск каждого процесса и файловый кэш указаны как часть опыта. Три наблюдения не выдаются за надёжный p99 или тест долговременной стабильности.

Назад: 33 · Далее: 35 · Подсказки

35. Практикум фонового обработчика

Результат: вы соединяете ограниченный конвейер с Generic Host, сохраняете совместимое состояние и останавливаете обработку без оставшихся задач. Нужны главы 29–34 и L03. Договор — L04, реализация — PipelineWorker.

От библиотеки к приложению

PipelineCore отвечает за чтение строк, независимую обработку и упорядоченное применение результатов. FileImportJob владеет файлами, блокировкой и фиксацией. ImportWorker запускает одну работу внутри Host. Program разбирает аргументы и собирает зависимости. Domain Core остаётся независимой от потоков, Host и файлов: правила корректного Expense не меняются из-за добавления конкурентности.

```
Program / System.CommandLine
          |
       Generic Host
          |
       ImportWorker (BackgroundService, singleton)
          | scope одной работы
       FileImportJob → ImportPipeline → JsonRowProcessor
          |                               |
     checkpoint                      Expense
```

Этот Host выполняет один файл и завершается. Это удобная законченная учебная единица с жизненным циклом BackgroundService; бесконечный daemon и установка службы ОС остаются расширениями. Несколько workers внутри конвейера не означают несколько конкурентных владельцев checkpoint.

BackgroundService и scope

ExecuteAsync представляет время жизни фоновой операции. В **.NET 10** весь этот метод запускается в фоне; нельзя полагаться на синхронный участок до первого await как на блокирующую инициализацию Host. Если готовность должна предшествовать запуску зависимостей, задайте явную фазу StartAsync или lifecycle hook. Изменение .NET 10.

Hosted service регистрируется как singleton. Scoped-зависимости нужно разрешать в созданном scope, а не захватывать в его конструкторе. У каждой логической работы должен быть собственный scope.

```
// Фрагмент ExecuteAsync; scopes — IServiceScopeFactory.
await using var scope = scopes.CreateAsyncScope();
var job = scope.ServiceProvider.GetRequiredService<FileImportJob>();
PipelineReport report = await job.ExecuteAsync(stoppingToken);
```

await using дождётся асинхронного освобождения сервисов scope. JsonRowProcessor зарегистрирован singleton, потому что он не хранит изменяемого состояния отдельной записи и допускает конкурентные вызовы. При добавлении изменяемого парсера или клиента проверьте этот договор заново. Scope в фоновой службе.

Фиксация и повторный запуск

FileImportJob открывает state.json.lock с FileShare.None и удерживает handle на время чтения, вычисления и замены состояния. Другой процесс, соблюдающий этот договор, получает ошибку I/O. Затем проверяется старый JSON checkpoint из L03. Повреждённое состояние никогда не считается пустым.

В процессе чтения конвейер строит ограниченный по MaxRecords кандидат в памяти. Корректные новые записи добавляются, эквивалентные Id считаются duplicates, конфликт сохраняет прежнее значение. При нормальном EOF и завершении всех стадий кандидат сериализуется и заменяет checkpoint через временный файл. Если добавлений нет, файл не переписывается. Ошибка строки не запрещает сохранить

другие правильные строки; ошибка I/O, превышение лимита или отмена до фиксации запрещает всю замену.

```
прочитать прежнее состояние → обработать файл → дождаться всех
            → проверить отмену → временный файл → заменить checkpoint
```

После замены файла нельзя обещать откат из-за поздней отмены. Если процесс погиб между заменой и печатью отчёта, повторный запуск найдёт уже принятые Id и не добавит их повторно. Сетевые эффекты внутри processor таким способом не откатываются; пример не выполняет их. Для долговечного приёма заданий и согласования с БД понадобятся транзакции, idempotency key и outbox в следующих блоках.

Остановка и ошибки

При EOF очереди естественно опустошаются. При Host.StopAsync stoppingToken отменяется, конвейер будит ожидающих и дожидается участников, после чего scope и файлы освобождаются. ShutdownTimeout Host установлен в 10 секунд. Это предел ожидания, а не принудительный останов произвольного чужого кода; IRowProcessor обязан соблюдать token.

ImportWorker классифицирует ожидаемые исходы, записывает JobOutcome и вызывает StopApplication для завершения конечной работы. JobCompletion использует TaskCompletionSource с асинхронными продолжениями. Program читает результат после RunAsync; отчёт появляется только после успешной фиксации. Необработанная ошибка фоновой службы по умолчанию останавливает Host; Ignore не перезапускает упавший цикл.

Код	Значение	stdout
0	Все строки приняты или признаны дубликатами	JSON-отчёт
4	Есть отклонённые строки, остальные обработаны	JSON-отчёт
2	Неверные аргументы/лимиты	Пусто
3	Ошибка I/O, checkpoint или лимита входа/состояния	Пусто
130	Отмена до завершения работы	Пусто
1	Неожиданная ошибка реализации	Пусто

Диагностика ILogger идёт в stderr; входные строки и полные исключения с потенциальными данными не печатаются. В отчёте сохраняются первые 20 ошибок по порядку строк, общий failed и признак errorsTruncated. Retried считает дополнительные попытки, а не исходные вызовы. По обычным JSON-данным он равен нулю.

Проверка управляемых отказов

PipelineChecks удерживает первую строку сигналом, позволяет следующим завершиться и проверяет окно допуска. Другие сценарии отменяют заполненные очереди, бросают IOException из worker, ломают чтение, исчерпывают retry и останавливают настоящий Host через StopAsync. В каждом случае проверяется, что активных обработчиков не осталось и старый checkpoint сохранился. Произвольный Sleep не используется для угадывания порядка событий; тайм-аут теста лишь обнаруживает зависание.

```
$jobArgs = @("--file", "labs/04-background-pipeline/data/mixed.ndjson",
    "--state", "work/l04/state.json", "--workers", "4",
    "--capacity", "8", "--window", "16")
dotnet run --project examples/Integration/PipelineWorker -- @jobArgs
python tools/verify_pipeline.py
```

На первом запуске ожидаются processed=6, added=2, duplicates=1, failed=3, total=2. При повторе — added=0, duplicates=3, остальные счётчики сохраняются; checkpoint одинаков по байтам. При P=1, C=1, W=1 решения должны быть такими же, как при 4/8/16.

Нагрузочный опыт и пределы результата

В проверенной среде главы 34 импорт 10000 уникальных строк дал медиану 0.343 с для P=1 и 0.330 с для P=4, при C=8 и W=16. Это примерно 29117 и 30276 записей/с с учётом запуска процесса, Host и записи файла. Использованы один прогрев и три наблюдения на вариант. Исходные измерения.

Небольшая разница не обосновывает универсальный выбор четырёх потоков: парсинг короткий, есть последовательный collector и стоимость запуска/файла. Увеличение P может повысить накладные расходы. Этот опыт проверяет процедуру и сохранение результата, а не долговременную устойчивость сервиса. Ограничения памяти обоснованы устройством очередей/окна и лимитами; точный пик RSS отдельно не измерялся.

Практика

35-A. Импортируйте mixed.ndjson дважды, затем повторите в новой папке с 1/1/1. Ожидаются одинаковые решения по строкам и одинаковые checkpoint. Критерий: сохранены stdout, stderr и exit code; вывод журнала не ломает JSON.

35-B. Реализуйте договор L04 самостоятельно. Добавьте управляемый процессор, который дважды выдаёт TransientWorkException и на третий раз возвращает Expense. Ожидаются added=1, retried=2; постоянно ошибающийся процессор получает retry_exhausted после трёх попыток. Критерий: повтор ограничен и отменяем; обычная неправильная сумма не повторяется.

35-C. Расширьте приложение до очереди нескольких файлов с drain/abort. После запрета приёма новые файлы отклоняются, принятые завершаются до срока остановки; после срока текущий незавершённый план отменяется. Критерий: отдельные сигналы приёма и abort, scope на работу, последовательная фиксация общего checkpoint и проверка повторного запуска. Готовый пример выполняет одну работу и этот daemon-режим не реализует.

Назад: 34 · Лабораторная L04 · Подсказки · Следующий блок в плане

L01. Консольный журнал расходов

Реализуйте программу ExpenseLedger по этому контракту. Предварительно пройдите главы 01–09 и разбор проекта. Цель — связать язык и BCL в работающую утилиту. Ориентир: 6–10 часов, включая самостоятельные проверки и исправления.

Входные данные

Принимается ровно один аргумент: путь к файлу. Файл — UTF-8, необязательный UTF-8 BOM допустим. UTF-16 и повреждённый UTF-8 отклоняются. Строки заканчиваются LF или CRLF; последняя может не иметь перевода строки. Пустые строки и строки из пробелов пропускаются. Номер ошибочной строки считается по физическим строкам файла, включая пропущенные. Заголовка и комментариев в формате нет.

Каждая непустая строка имеет ровно три поля:

```
дата;категория;сумма
2026-01-10;food;12.50
```

Пробелы по краям каждого поля удаляются перед проверкой. Внутренние пробелы не удаляются.

Поле	Правило
Дата	Ровно `yyyy-MM-dd`, настоящая календарная дата в диапазоне DateOnly
Категория	1–32 ASCII-символа; первый — буква; остальные — буквы, цифры, `_`, `-`
Сумма	Только ASCII-цифры и необязательная точка; целая часть обязательна; после точки 1–2 цифры
Диапазон суммы	Больше 0 и не больше 999999.99
Число записей	Не больше 100000 непустых корректных строк

Разрешены `1`, `1.5`, `1.50`, `001.50`; запрещены `+1`, `-1`, `0`, `.5`, `1.`, `1,50`, `1e2`, `1.234`, `NaN`, разделители тысяч. Регистры категорий не различаются; в результате выводится нижний ASCII-регистр. Повторные строки считаются отдельными расходами: правило дедупликации здесь не задано.

Лимит записей позволяет держать первый учебный вариант в памяти. Это формат небольшой локальной выгрузки; потоковый импорт и лимиты байтов изучаются в следующих блоках.

Результат

Сначала выведите count и total, затем категории в порядке StringComparer.Ordinal. Суммы всегда имеют точку и два знака после неё. Культура компьютера не влияет на результат. Успешный stderr пуст; порядок строк определён контрактом.

Для valid.txt:

```
count=3
total=50.00
food=20.00
travel=30.00
```

Для пустого файла:

```
count=0
total=0.00
```

Ошибки и коды завершения

Код	Ситуация	Требование к stderr
0	Успех	Пуст
2	Не один аргумент	Начинается с `usage:`
3	Ошибка пути, открытия или чтения	Начинается с `io:`
4	Ошибка данных или кодировки	`line N: причина` либо `encoding: причина`

При любом ненулевом коде stdout пуст. Для ошибки данных достаточно сообщить первую; номер строки обязателен. Текст причины после префикса может быть русским или английским. При повреждении кодировки номер строки не требуется, поскольку буферизация чтения усложняет его определение.

У invalid.txt ошибочна дата в строке 2. Код — 4. Строка 1 не должна приводить к частичному успешному отчёту.

Требования к реализации

- Суммы представлены decimal; при приёме данных точность проверяется, а не молча округляется.
- Есть отдельный тип расхода и методы разбора без чтения Console или файла.
- После успешного создания расход удовлетворяет правилам категории и суммы.
- Файл освобождается при успехе, ошибке данных и ошибке чтения.
- Для группировки используется Dictionary, для порядка вывода — явная сортировка.
- Нет подавления nullable-предупреждений с помощью ! без объяснённого инварианта.
- При максимальном количестве записей расчёт остаётся в допустимом диапазоне decimal.

Минимальные сценарии приёмки

Вход или изменение	Ожидание
Основной файл	Точный отчёт выше
Пустой файл и только пробелы	count=0, total=0.00
`food` и `FOOD`	Одна категория
Две одинаковые строки	Два расхода
Сумма 0.01 и 999999.99	Обе принимаются
0, −1, 1000000, 1.234, 1,50	Код 4, номер строки
2024-02-29 / 2026-02-29	Первая дата допустима, вторая нет
Пустая категория, 33 символа, буква вне ASCII	Код 4
Четвёртое поле	Код 4
Пустая строка перед ошибочной	Учитывается в номере ошибки

Вход или изменение	Ожидание
100000 / 100001 записей	Успех / код 4
UTF-8 BOM / неверный байт UTF-8	Успех / код 4
Несуществующий файл	Код 3
Нет аргумента / два аргумента	Код 2
Разные системные культуры	Одинаковый отчёт

Как проверить и сдать

Соберите свою программу в Release. Выполните:

```
python tools/verify.py --ledger-dll work/ExpenseLedger/bin/Release/net10.0/ExpenseLedger.dll
```

Без параметра скрипт проверяет готовые примеры курса и их сборку. Подготовьте исходники, команду запуска и запись трёх найденных вами дефектов с причиной. Минимум для зачёта: контракт выполнен полностью, все проверки пройдены, код объясним без образца.

Оценка из 10: корректность 4; граничные случаи и ошибки 2; декомпозиция и инварианты 2; воспроизводимость и ясность 2. Даже при хорошей сумме баллов ошибка денежного расчёта, частичный отчёт при сбое или неосвобождённый ресурс требуют исправления.

Дополнительное изменение

После зачёта добавьте фильтр месяца из задачи 10-C. Отдельно запишите контракт, проверьте пустой результат и ошибочную строку вне выбранного месяца. Сравните архитектуру: удалось ли добавить фильтр, не переписывая парсер суммы?

L02. Библиотека правил и отчётов

Постройте второй этап ExpenseLedger после глав 11–18. Ориентир: 8–12 часов с проверками. Работа развивает L01: одни правила должны обслуживать разные способы ввода и хранения.

Структура решения

Создайте библиотеку Core с моделью, Result, интерфейсом хранения и сервисом. Отдельная библиотека Memory реализует хранилище. Консольный клиент демонстрирует работу, а отдельный проект проверок вызывает библиотеку напрямую.

Core не зависит от клиента или Memory. Он не читает файлы и не выводит сообщения в Console. Парсинг текстового формата L01 остаётся на границе приложения. Готовые проекты находятся в examples; самостоятельное решение размещайте в work/L02.

Договор Money и Expense

Money — readonly record struct для одной условной денежной единицы. Он хранит неотрицательный decimal, точно выражаемый сотыми. Допустимы 0, 0.01, 1.2300m; недопустимы −1 и 1.001. Default равен нулю. Оператор + сохраняет правила и выбрасывает OverflowException при переполнении.

Expense — неизменяемый record class с Guid Id, DateOnly Date, string Category и Money Amount. Id не равен Guid.Empty. Сумма больше 0 и не выше 999999.99. Категория после Trim содержит 1–32 ASCII-символа: первая буква, остальные буквы, цифры, дефис или подчёркивание. Хранится нижний регистр. Невалидный аргумент конструктора вызывает подходящий ArgumentException или его производный тип. Null-категория явно отклоняется.

Записи с одинаковыми данными равны; идентичность для хранилища определяется только Id. Нельзя изменить сумму или категорию существующего Expense, обходя его конструктор.

Договор Result

`Result<T> where T : notnull` создаётся только через Success(value) и Failure(error). При успехе IsSuccess=true, Error=null и Value содержит значение, включая 0 для int. При отказе IsSuccess=false, Error — непустой стабильный код; чтение Value бросает InvalidOperationException. Success с null и Failure с пустым/пробельным кодом отклоняются.

Договор хранения

```
public interface IExpenseStore
{
    bool TryAdd(Expense expense);
    IReadOnlyList<Expense> Snapshot();
}
```

Первое добавление Id возвращает true. Повторный Id возвращает false и сохраняет прежний объект, даже если новая запись имеет другую сумму. Отдельные Id с одинаковыми остальными данными допустимы.

Snapshot возвращает отдельный снимок. Последующие добавления его не меняют. Через возвращённый объект нельзя изменить содержимое хранилища. Контракт не определяет порядок строк и не обещает потокобезопасность.

Договор сервиса

LedgerService получает IExpenseStore через конструктор. Add возвращает Success(expense) либо Failure("duplicate_id"). Added возникает один раз после успешного добавления и не возникает при дубликате.

ByCategory(year, month) возвращает read-only список CategoryTotal(Category, Total, Count). Включаются записи только данного месяца и года. Категории отсортированы Ordinal; итоги и количества точны. Пустой выбор даёт пустой список. Year должен подходить DateOnly, month лежать в 1…12. Неверные аргументы отклоняются даже на пустом хранилище.

Готовый отчёт не меняется после добавления новых записей. Вызов события синхронный. Если подписчик бросит исключение, оно выйдет из Add, но расход уже сохранён. Проверьте это явно: повтор операции не должен создать вторую запись.

Набор данных

Используйте четыре разных непустых Guid и следующие записи:

Дата	Категория	Сумма
2026-01-01	food	12.50
2026-01-01	travel	30.00
2026-01-01	FOOD	7.50
2026-02-01	food	1.00

За январь: food=20.00, Count=2; travel=30.00, Count=1. За февраль: food=1.00, Count=1. За март — пусто. Число уведомлений после четырёх добавлений — 4, после попытки дубликата — всё ещё 4.

Проверки приёмки

1. Default Money равен Money(0), а две равные суммы ведут себя как один ключ HashSet.
2. Отрицательная сумма и дробь 1.001 отклоняются; 1.2300 не округляется и допустима.
3. Expense запрещает нулевой Id, нулевую сумму, невалидную категорию и сумму выше лимита.
4. Повторный Id с изменёнными данными не перезаписывает исходную запись.
5. Снимок не меняется после Add. Не должно существовать обходного изменения через cast к List/IList.
6. Отчёт учитывает год и месяц, объединяет регистр, имеет явный порядок и сохраняется после новых Add.
7. Событие вызывается только при новом Id; после отписки обработчик не вызывается.
8. Бросающий обработчик не откатывает запись. Повторный Add возвращает duplicate_id.
9. Проверки работают при ru-RU, en-US и tr-TR; машинное представление отчёта остаётся одинаковым.
10. Тот же сервис и набор контрактных проверок работают со второй реализацией хранения на List.

Запуск и сдача

```
dotnet run --project examples/TypesAndQueries -- 18
dotnet run --project examples/CoreChecks
```

CoreChecks проверяет поставляемое решение. Для проверки своего решения создайте отдельный тестовый проект, ссылающийся на ваши Core/Memory, и перенесите контрактные сценарии; имена API приведены выше. Не заменяйте собственные ссылки на библиотеки готовыми только ради зелёной проверки.

Сдайте исходники, схему зависимостей и список проверок. Объясните один дефект, который выявила подмена хранилища или проверка снимка. Оценка из 10: контракты и инварианты 4; отчёт и ошибки 2; направление зависимостей 2; воспроизводимые проверки и объяснение 2. Нарушение денежной точности, уникальности Id или независимости снимка требует исправления до зачёта.

Дополнительно выполните 18-C: фильтр категории с явной нормализацией и сохранением исходных контрактов.

L03. Воспроизводимая утилита импорта

Цель: объединить BCL, форматы, HTTP, Host и CLI, сохранив правила ExpenseLedger.Core. Нужны главы 19–30 и L02. Образец — ImportCli.

Что нужно сдать

Самостоятельную solution с библиотекой правил, адаптером формата, CLI и проверками. Один запуск принимает один JSON-документ из файла или HTTP. Результат — checkpoint с принятыми записями, отчёт с ошибками каждой отклонённой записи и код завершения. Экспорт checkpoint должен быть пригоден для повторного импорта.

Команды и конфигурация

```
import (--file PATH | --url URL) --state PATH [--pretty]
export --state PATH --output PATH
--help
```

Пути не должны совпадать: вход/состояние и состояние/экспорт различаются. Для HTTP разрешены HTTPS и loopback HTTP; redirects не следуются. URL с user info отвергается. Реальная проверка TLS остаётся включённой.

appsettings задаёт Import:MaxBytes=1048576, MaxRecords=10000 и TimeoutSeconds=15. Префикс CS_IMPORT_ с двойным подчёркиванием позволяет переопределить настройки, например CS_IMPORT_Import__MaxRecords. Диапазоны: 128–16777216 байт, 1–100000 записей, 1–120 секунд. Неверная конфигурация должна обнаруживаться до изменения состояния.

MaxBytes ограничивает вход и итоговый сериализованный checkpoint; из-за отступов канонический экспорт может оказаться больше компактного входа. MaxRecords ограничивает и входной массив, и совокупное состояние. Превышение итогового лимита — ошибка операции, а не разрешение молча обрезать данные.

Формат документа

UTF-8, допустим начальный UTF-8 BOM. Верхний объект содержит ровно поля version и records. version — целое 1, records — массив. Отсутствующие обязательные поля, records=null, повторные имена свойств, неизвестные свойства, слишком глубокий JSON и неверные JSON-типы дают ошибку всего документа. Комментарии и trailing comma не разрешены.

Элемент records — объект с полями id, date, category, amount, либо null. Поля записи строковые; отсутствующие и null-поля проходят в построчную валидацию. Число вместо строкового amount — структурная ошибка. null-элемент даёт invalid_id.

Поле	Проверка
id	Guid.TryParseExact с форматом D, значение не Guid.Empty
date	Существующая DateOnly в yyyy-MM-dd, InvariantCulture
amount	ASCII-цифры, необязательная точка и 1–2 дробных знака; 0.01–999999.99
category	Правила L02: trim, ASCII-буква, затем буквы/цифры/_/-, длина 1–32, lowercase

amount не принимает знак, пробелы, запятую и exponent; ведущие нули допустимы. Внешняя числовая строка ограничена 32 символами. При нескольких неверных полях сообщается первая ошибка в порядке id,

date, amount, category. Номер record в отчёте начинается с 1 и относится к позиции в массиве.

Дедупликация и конфликт

Сравнение выполняется после преобразования в Expense. FOOD и food, 12.5 и 12.50 дают одинаковые нормализованные значения.

Новый корректный Id добавляется. Известный Id с равными датой, категорией и Money считается duplicate. Известный Id с другими данными даёт id_conflict и не перезаписывается. Правило действует и внутри текущего входа, и относительно прежнего состояния.

Checkpoint содержит только корректные записи с уникальными Id, версию 1, нормализованные категории и amount с двумя дробными знаками. Порядок экспорта определяется сортировкой Guid через comparer .NET, а не исходной позицией.

Фиксация и повторный запуск

Сначала прочитайте и проверьте внешний документ, затем возьмите блокировку состояния, прочитайте и проверьте прежний checkpoint, вычислите изменения и сохраните новый файл. Файл .lock остаётся после освобождения; наличие файла само по себе не означает занятую блокировку. Одновременный второй писатель должен завершиться ошибкой, а не потерять изменения первого.

Если есть ошибки отдельных записей, корректные новые записи всё равно сохраняются. При структурной ошибке, неверном checkpoint, превышении итогового лимита, сбое получения или отмене до фиксации прежнее состояние остаётся. При отсутствии добавлений checkpoint не переписывается; для пустого первого импорта файл данных не создаётся.

Временный файл создаётся рядом с назначением и удаляется при обычном отказе. Успешная замена предшествует публикации отчёта. Повтор после потери ответа распознаёт уже принятые Id. Контракт не обещает устойчивость к отключению питания и атомарность между файлом, stdout и логами. Это учебное файловое хранилище; проверка транзакций БД относится к L05.

HTTP и ресурсы

Чтение response body ограничивается даже без Content-Length. Проверяется application/json. Response, поток, token source, scope и host освобождаются владельцами. Разрешена не более чем одна повторная попытка GET при временной ошибке; 404 и ошибки формата не должны превращаться в бесконечный retry.

Пользовательская отмена и deadline различаются. Общий deadline включает чтение тела после заголовков. Проверки поднимают сервер только на loopback и выбирают свободный порт.

Вывод и коды завершения

Код	Значение	stdout
0	Успех без ошибок записей, успешный export или help	JSON-отчёт, exported либо справка
2	Неверные аргументы или конфигурация	Пусто при ошибке
3	Файл, структура, лимиты, состояние или запись результата	Пусто
4	Частичный успех с ошибками записей	Полный JSON-отчёт
5	Ошибка HTTP/сети или сетевой deadline	Пусто
130	Отмена	Пусто до завершения операции

Обычный отчёт содержит added, duplicates, failed, total и errors. Сумма added+duplicates+failed равна числу элементов входного records. total — число принятых записей в итоговом состоянии, включая прежние. В errors только record и code; содержимое отклонённого текста не требуется.

Диагностика и структурированные логи идут в stderr. --pretty меняет stdout на таблицу Spectre.Console и список ошибок, сохраняя коды и состояние.

Контрольный набор

Используйте mixed.json и отсутствующий checkpoint.

Запись	Решение
1	Добавить food=12.50
2	Добавить travel=30.00
3	Duplicate записи 1 после нормализации
4	invalid_date
5	invalid_amount
6	id_conflict с записью 1

Первый отчёт совпадает с expected-first.json: 2/1/3/2, код 4. Повторный отчёт: 0/3/3/2, те же ошибки и те же байты состояния. Экспорт содержит две записи; его импорт в то же состояние даёт 0/2/0/2 и код 0. valid.json содержит только две корректные записи.

Проверка сдачи

```
python tools/verify_integration.py
```

Скрипт проверяет поставляемый образец; он не является универсальным автогрейдером чужой структуры. Для своего решения перенесите тот же набор контрактных сценариев и укажите команду запуска.

Обязательны обычный и пустой ввод, неверные поля, duplicate/conflict, повреждённый документ и checkpoint, лимиты, повторный запуск, экспорт, сбой до замены файла, отмена, два конкурирующих владельца блокировки, GET 503→200, 404 без retry и медленное тело. Тесты не должны зависеть от публичного API, реального ожидания истечения даты или порядка Dictionary.

На защите объясните владение каждым disposable-объектом, направления зависимостей, границу фиксации, стоимость обработки в памяти и отличие идемпотентности от exactly-once. Расширение --dry-run из 30-C выполняется самостоятельно.

Глава 30 · Подсказки

L04. Ограниченный фоновый конвейер

Цель: перейти от одного документа L03 к потоку записей, сохранив правила расходов, детерминированный результат, отмену и воспроизводимый запуск. Предварительно изучите главы 31–35 и сдайте L03. Готовую реализацию открывайте после собственной попытки.

Что сдавать

Решение содержит библиотеку конвейера, Host-приложение, проверки, инструкцию запуска и протокол измерений. Укажите, кто закрывает очереди, кто меняет состояние, когда возможна фиксация и что происходит при отмене. Не достаточно показать только успешный запуск: нужны управляемые сценарии отказов.

Формат входа

Вход — локальный файл NDJSON: на каждой физической строке один JSON-объект с полями id, date, category, amount. Это отдельный потоковый формат; объект с version/records из L03 не является NDJSON-входом. Пример одной строки:

```
{"id":"11111111-1111-1111-1111-111111111111","date":"2026-01-10","category":"food","amount":"12.50"}
```

Правила Id, даты, категории и суммы совпадают с L03. Нормализация категории предшествует сравнению. Запрещены неизвестные/повторные JSON-свойства, комментарии и неправильные типы. Файл использует UTF-8; допускаются BOM в начале, LF/CRLF и последняя строка без LF. Перевод строки внутри JSON-строки должен быть escape-последовательностью, а не физическим LF.

Пустая физическая строка — invalid_json и учитывается в processed/failed. Пустой файл содержит ноль строк. Один завершающий LF не создаёт дополнительную строку; два LF подряд создают пустую строку. Один BOM без данных считается одной пустой строкой. Неправильный JSON, включая повреждённый UTF-8 внутри строки, даёт invalid_json только для этой строки. JSON null даёт invalid_id. Структурно правильная строка проверяется в порядке Id, date, amount, category.

Ограничения готового приложения

Ресурс	Значение по умолчанию	Допустимое изменение
Workers P	4	CLI --workers, 1–32
Capacity C каждой очереди	8	CLI --capacity, 1–1024
Window W допущенных строк	16	CLI --window, 1–4096
Байты физической строки	4096	До удаления CR/BOM, без LF
Байты всего входа	16 MiB	Включая BOM и разделители
Число физических строк	100000	Включая ошибочные
Итоговое состояние	10000 расходов, 16 MiB JSON	Превышение отменяет фиксацию
Подробные ошибки в отчёте	Первые 20	failed считает все
Попытки временной ошибки	3	Задержка между попытками 50 мс

Остальные лимиты настраиваются через PipelineOptions при использовании библиотеки, но не вынесены в CLI. Для библиотеки Validate задаёт конечные допустимые диапазоны. Размер строки проверяется при чтении; накопление полного входного файла запрещено. Состояние ограничено, но хранится в памяти; это

не обработчик произвольно большой базы расходов.

Стадии и порядок

Producer читает ограниченные строки, получает разрешение окна и записывает задание с номером. P workers проверяют строки. Collector получает результаты через вторую ограниченную очередь и применяет их строго в исходном порядке. Разрешение окна освобождается после применения. Обе очереди используют FullMode.Wait, без потери сообщений при заполнении.

Решение для корректной записи:

1. Новый Id добавляется в кандидат состояния.
2. Совпадающие Id и нормализованные поля дают duplicate.
3. Совпадающий Id с другими полями даёт id_conflict; прежняя запись сохраняется.

Поэтому конфликт внутри файла решается порядком входа, а не скоростью потоков. Неправильные поля проверяются до дедупликации. Коллектор — единственный владелец изменяемого Dictionary. Каждый допущенный элемент даёт ровно один результат при нормальном завершении.

Сбой и повтор

Ошибочная строка не прерывает файл. Только TransientWorkException допускает повтор; после трёх неудач результат — retry_exhausted, retried=2 для этой строки. Настоящий JsonRowProcessor не использует повторы: они проверяются через подмену IRowProcessor. Подмена обязана быть потокобезопасной, идемпотентной и поддерживать CancellationToken.

Ошибка I/O, неожиданное исключение обработчика, превышение лимита или внешняя отмена останавливает операцию. Заблокированные участники должны проснуться; вызывающий получает исход только после завершения всех стадий. При инфраструктурном сбое кандидат не сохраняется. Первый сохранённый инфраструктурный сбой имеет приоритет над сопутствующей отменой.

При нормальном EOF очереди опустошаются. Host.StopAsync выполняет abort текущей работы, а не drain с обязательным сохранением. ShutdownTimeout=10 секунд не может принудительно остановить процессор, игнорирующий token; такой процессор нарушает договор.

Checkpoint

Формат — JSON версии 1, совместимый с экспортом L03. Сначала проверяется прежнее состояние; повреждение, неверная запись и повторный Id в checkpoint — фатальная ошибка. На всю работу удерживается state.json.lock через FileShare.None. Lock-файл остаётся на диске, а исключительность определяется открытым handle. Все участники записи должны соблюдать этот протокол.

Фиксация выполняется один раз после полного чтения и завершения стадий: временный файл в каталоге checkpoint, flush, закрытие и замена. При нуле добавлений checkpoint не переписывается. Вход не должен совпадать с checkpoint или его lock-файлом; проверка сравнивает нормализованные пути. Alias через hard link/symlink не определяется: используйте отдельные обычные файлы в своём work. Потеря питания и сетевые файловые системы не входят в гарантию примера.

Отмена до замены сохраняет прежние байты. Сбой после замены, но до печати отчёта не отменяет запись; повторный запуск распознаёт сохранённые расходы как duplicates. Очереди не долговечны, а exactly-once для внешних побочных эффектов не заявляется.

Проверяемый пример

```
$jobArgs = @("--file", "labs/04-background-pipeline/data/mixed.ndjson",
    "--state", "work/l04/state.json", "--workers", "4",
    "--capacity", "8", "--window", "16")
dotnet run --project examples/Integration/PipelineWorker -- @jobArgs
```

mixed.ndjson содержит шесть строк. На отсутствующем checkpoint ожидается expected-first.json: processed=6, added=2, duplicates=1, failed=3, total=2, retried=0. Ошибки: строка 4 invalid_date, 5 invalid_amount, 6 id_conflict; errorsTruncated=false. Код завершения 4 означает частичный успех с сохранением двух расходов. Повтор даёт added=0, duplicates=3 при тех же остальных полях и байтах checkpoint.

stdout содержит только JSON при кодах 0/4; диагностические сообщения идут в stderr. При других кодах stdout пуст: 2 — аргументы, 3 — данные/I/O/лимит, 130 — отмена, 1 — непредвиденная ошибка реализации. Длительность записывается в журнал, но не в сравниваемый JSON, чтобы отчёт оставался воспроизводимым.

Приёмка

- Пустой файл, BOM, CRLF, короткие чтения и отсутствие последнего LF обработаны по договору.
- При P/C/W=1/1/1, 2/1/3, 4/8/16 получаются одинаковые отчёты и состояние.
- Удержание строки 1 не позволяет допустить более W строк; число активных processor не превышает P.
- Нормальный EOF не теряет записи; ошибка consumer будит producer с заполненной очередью.
- Отмена завершает все processors до возврата, checkpoint не меняется, временные файлы не остаются.
- Три неудачные попытки дают одну ошибку retry_exhausted; отмена прерывает задержку повтора.
- Неправильный checkpoint и занятый lock не перезаписываются; повторный импорт идемпотентен.
- Проверки времени используют сигналы для порядка и конечный timeout для обнаружения зависания.
- Нагрузочный опыт сохраняет результат, параметры, время и ограничения интерпретации.

Команды проверки: `python tools/verify_pipeline.py`; измерения после Release-сборки: `python tools/measure_pipeline.py`. Встроенные проверки не оценивают решение учащегося автоматически: при переносе контрактных сценариев подключите свою реализацию и сохраняйте независимые ожидаемые результаты. Пример, проверки и benchmark находятся в Integration.

После сдачи переходите к блоку 36–39: тестовые фреймворки, проектирование и командная работа. Его состояние отражено в программе.

Подсказки и разбор контрольных вопросов

Здесь намеренно даны направления решения и объяснения, а не готовые программы для всех задач. Открывайте соответствующий пункт после самостоятельной попытки. Полные учебные примеры находятся в examples, эталон L01 — в ExpenseLedger.

Главы 01 и 02

01-B: `--no-build` запускает уже собранный результат. Изменённый исходник сам не становится новой сборкой. 01-C: DLL находится в publish-каталоге и запускается подходящим runtime; файл исходника для этого не нужен.

02-B: неизвестное имя обнаруживает компилятор; целочисленное деление на ноль при переменной-делителе — среда выполнения; замена умножения сложением может быть полностью допустимым кодом. 02-C: условие breakpoint проверяется в точке остановки, где total уже должен быть вычислен.

SDK включает инструменты сборки, runtime исполняет код. Namespace не устанавливает пакет. Область видимости локального bonus заканчивается вместе с блоком. Диагностика nullable — не замена проверке внешних данных.

Глава 03

03-A: `7 / 3` даёт int 2; `7 / 3m` даёт decimal с конечным приближением к 2⅓; `-7 % 3` равно −1; cast 2.9 к int даёт 2. 03-B: 125.50 × 4 = 502.00; после скидки остаётся 85%, результат 426.70. 03-C: сначала приведите один множитель к long. Присваивание готового переполненного int в long не восстанавливает потерянное значение. checked должен охватывать именно умножение.

`var` выбирает один статический тип при компиляции. Decimal тоже не является бесконечно точной дробью. Правило округления принадлежит предметной задаче; его нельзя заменять произвольным приведением.

Глава 04

04-A: проверьте отрицательное n до цикла. При n=0 цикл от 1 до n не выполняется. 04-B: количество увеличивается только после успешного TryParse. 04-C: храните число последовательных ошибок; сбрасывайте после принятого значения. Выход через break заканчивает ближайший цикл, через return — весь метод. EOF приходит как null, Enter без текста — пустая строка.

Глава 05

05-A: метод должен возвращать число; форматирование делается отдельно. 05-B: заранее присвойте out-параметру 0. Это упрощает все пути отказа. 05-C: рекурсия имеет базовый случай n=0. Невалидный вход отклоняйте до первого рекурсивного вызова. 20! помещается в long, 21! — нет. Цикл не создаёт цепочку рекурсивных вызовов.

Обычный параметр копирует своё значение. Если значение — ссылка, копируется ссылка на общий объект. Ref позволяет менять переменную вызывающего кода, а out обязан получить значение от метода.

Глава 06

06-A: сначала различите null и обычную строку, затем проверьте IsNullOrWhiteSpace. 06-B: NumberStyles.AllowDecimalPoint совместно с InvariantCulture не разрешает запятую или знак. 06-C: проверка диапазона и количества цифр выполняется дополнительно к TryParse; проверяйте ASCII-цифры явно, если таков договор формата.

Для контрольных вопросов: Length считает кодовые единицы UTF-16; `!` меняет анализ компилятора, но не объект во время выполнения. Ordinal определяет сравнение текста, CultureInfo — в том числе форматирование чисел. Эти инструменты решают разные задачи.

Глава 07

07-A: минимум можно инициализировать первым элементом только после проверки непустого массива. Либо используйте int? и явно представьте отсутствие результата. 07-B: comparer задаётся при создании Dictionary. Числовой итог 50.01 не зависит от порядка перечисления. 07-C: Queue вернёт add, report, quit; Stack начнёт с quit, и обработчик может сразу закончиться.

Срез массива создаёт новый контейнер. Если элементы — ссылки, сами объекты остаются общими. Словарь обеспечивает поиск по ключу; требуемый порядок отчёта задавайте сортировкой.

Глава 08

08-A: `Expense second = first;` копирует ссылку. `new Expense(...)` создаёт новый экземпляр. 08-B: get-only свойства и проверка конструктора не позволяют последующим присваиванием нарушить правило. 08-C: Basket содержит список и выдаёт только нужные операции. Внешний код не должен иметь возможность добавить недопустимый элемент в обход этих операций.

Required требует инициализации по правилам компилятора, но не обещает непустую строку. Readonly не делает изменяемый объект глубоко неизменяемым.

Глава 09

09-B: проверяйте число аргументов до открытия файла. Печатайте результат после завершения чтения. 09-C: `new UTF8Encoding(false, true)` включает исключение при неверных последовательностях байтов. Включение автоматического определения BOM может принять UTF-16; для строгого UTF-8 его отключают.

Using вызывает Dispose при обычном выходе и раскручивании исключения. Throw без аргумента сохраняет исходную цепочку диагностики. Между Exists и открытием файловая система может измениться. Ленивая последовательность способна выбросить исключение при MoveNext внутри foreach.

Глава 10 и L01

Сначала зафиксируйте формат и тестовые строки. Разбор суммы должен иметь один источник правил. Дата проверяется календарным типом, а не только разделением строки на части. Полностью прочитанный корректный набор отделяется от вывода — тогда ошибка строки 2 не выдаст отчёт строки 1.

Для 10-C сначала валидируйте весь вход, потом отбирайте месяц. Иначе фильтр случайно скрывает ошибочные записи и нарушает контракт. Не меняйте денежный расчёт при добавлении фильтра.

Эталонное решение — один из возможных вариантов. После сравнения закройте его и повторно реализуйте самый трудный метод по собственным тестам.

Глава 11

11-A: new выбирается по статическому типу; virtual/override связывает вызов с фактическим объектом. 11-B: храните строки в отдельном списке тестового writer и сравните их с ожидаемым набором. 11-C: потребитель чтения должен зависеть только от возможностей чтения. Если объект обещает возможность записи через общий контракт, постоянный отказ в потомке требует пересмотра модели.

Глава 12

12-A: record генерирует равенство данных; ReferenceEquals по-прежнему проверяет идентичность ссылок. 12-B: проверьте значение через decimal.Round(amount, 2) == amount, не число знаков в текстовом представлении. Нулевой default Money должен быть допустим. Ограничение положительности расхода проверяйте в Expense. 12-C: with копирует ссылки на вложенные объекты. Новый List копирует контейнер, но не изменяемые элементы-классы.

Равные объекты обязаны иметь равные хеши, но одинаковый хеш не доказывает равенство. Уникальность расхода определяется Id, даже когда record сравнивает все данные.

Глава 13

13-A: `IComparable<T>` обещает CompareTo; для отдельного правила сравнения можно передавать `IComparer<T>`. 13-B: Success(0) нельзя путать с ошибкой. Закрытый конструктор и фабрики ограничивают комбинации состояния. 13-C: producer string безопасен там, где нужен producer object; consumer object — там, где нужен consumer string. Для `List<string>` запись произвольного object была бы небезопасной, поэтому такого преобразования нет.

Глава 14

14-A: условие меняется делегатом, а цикл остаётся одним и тем же. 14-B: цикл for захватывает одну переменную i; локальная captured в теле создаёт отдельную переменную на итерацию. 14-C: сохраните экземпляр делегата, чтобы отписаться. Удаление переменной клиента не убирает ссылку из события долгоживущего издателя. Исключение подписчика не отменяет уже выполненное изменение.

Глава 15

15-A: finally итератора выполняется при Dispose перечислителя, в том числе через break в foreach. 15-B: вынесите проверку в обычный внешний метод, возвращающий локальный итератор. 15-C: создайте счётчик снаружи генератора. После материализации повторно обходится новый контейнер, а не первоначальное вычисление. Отдельно проверьте освобождение ресурса, если перебор прерван.

Глава 16

16-A: Where отбирает чётные, Select умножает, OrderBy задаёт порядок результата. 16-B: GroupBy сохраняет все расходы внутри категории; DistinctBy(category) потерял бы часть сумм. 16-C: ToArray фиксирует набор ссылок/значений на момент перечисления, но не делает глубокую копию элементов. Any и второй foreach могут запускать два обхода. Нельзя помещать обязательные побочные действия в селектор и рассчитывать, что любой терминальный метод его вызовет.

Глава 17

17-A: int-pattern не совпадает с long, даже если число помещается в int. 17-B: list pattern проверяет форму массива, а TryParseExact — смысл строки месяца. 17-C: extension вызывается как статический метод; private-члены Money ему недоступны. Extension property IsZero работает на C# 14. Для более старого компилятора нужен обычный метод.

Глава 18 и L02

Snapshot должен копировать контейнер и не выдавать его для изменения. Один IReadOnlyList над внутренним List не создаёт снимок и не мешает попытке приведения к изменяемому интерфейсу. В образце используется Array.AsReadOnly над новым массивом неизменяемых Expense.

Неправильный месяц проверяйте до запроса к хранилищу: пустой вход не должен скрывать ошибку аргументов. Ожидаемый дубликат возвращайте через Result. Ошибка подписчика распространяется после записи; следующая попытка того же Id должна вернуть duplicate_id. Для 18-C нормализуйте фильтр один раз, до сравнения с уже нормализованной моделью.

Глава 19

19-A: начните с typeof выбранного типа, затем Assembly, ManifestModule и EntryPoint. Имя namespace принадлежит типу, а не каталогу DLL. 19-B: IL работает с вычислительным стеком; JIT может заменить умножение на эквивалентную инструкцию и использовать регистры. Не требуйте буквального совпадения с исходной записью. 19-C: проверяйте AssemblyName клиента. InternalsVisibleTo не ограничивается одним методом; не добавляйте его, если достаточно публичного контракта.

Явный статический конструктор даёт однократную инициализацию, но не повторные попытки после сбоя. Native AOT меняет доступность IL и динамического кода: GetMethodBody не универсальный инспектор.

Глава 20

20-A: using должен охватывать весь период подписки. Проверяйте количество обработчиков после нормального выхода и исключения, а не только значение Count внутри блока. 20-B: храните запрошенную длину отдельно от массива. Тестовый пул должен считать Return; проверка повторного Dispose выявит двойной возврат даже без реального ArrayPool. 20-C: корень удерживает publisher, publisher — делегат, делегат — подписчика. Finalizer не спасает достижимый подписчик. Снимите подписку там, где заканчивается владение.

ReadOnlySpan не продлевает аренду и не запрещает владельцу изменить массив. Dispose меняет договор использования ресурса; GC освобождает недостижимую управляемую память.

Глава 21

21-A: используйте GetProperties и GetCustomAttribute, а не поиск слова NonBlank в исходнике. 21-B: сначала проверяйте форму схемы, потом значения. Ordinal-сортировка имён отделяет договор результата от неопределённого порядка отражённых членов. 21-C: прямой валидатор знает свойства на этапе компиляции. Генератор может создавать аналогичные прямые обращения автоматически; работа генератора и работа приложения — разные этапы.

Для AOT нужна публикация и запуск результата, а не только отсутствие ошибок обычного build. TargetInvocationException у getter не означает, что ошибка произошла в проверке пробелов. CallerArgumentExpression подставляется компилятором и может быть переопределён вызывающим кодом.

Глава 22

22-A: сравнивайте значение Title после Emit и загрузки сборки. Экранированная запись исходника и значение строки выглядят по-разному, но должны представлять одни данные. 22-B: чтение свойства представлено IPropertyReferenceOperation. Проверяйте символ владельца свойства, учитывайте namespace и отсутствие диагностики для чужого Console. 22-C: сначала разберите дополнительный файл в простую модель (title, edition), затем генерируйте. Генератор обязан объяснить ошибку ввода и не выдавать полусобранный класс.

Partial-части принадлежат одной компиляции. Analyzer диагностирует существующие операции, generator добавляет исходник. Инкрементальность зависит от входов и равенства промежуточных моделей.

Глава 23

23-A: readonly ограничивает действия через конкретную ссылку, но не других владельцев. 23-B: проверьте ASCII-цифры и NumberStyles.None с InvariantCulture; сумму вычисляйте в long. Второй разделитель останется в правом срезе и должен сделать поле недопустимым. 23-C: оборачивайте аренду в try/finally либо владельца using, ограничивайте длину запроса. Сначала докажите одинаковый результат на обычных и ошибочных данных, потом измеряйте выделения.

Ref return допустим для элемента живущего у вызывающего кода массива, но не локального stackalloc. Fixed сохраняет адрес на время блока, однако указатель сам не проверяет индекс. Показатель выделений текущего потока не является временем выполнения или общим расходом памяти.

Глава 24

24-A: запускайте dotnet --version из разных каталогов и учитывайте поиск global.json вверх по дереву. 24-B: в setter вычислите и проверьте новое значение до присваивания; при исключении старое останется. Сравните также начальное значение, чтобы миграция не изменила поведение объекта до первого set. 24-C: обновляйте по одному аспекту. Record меняет равенство, LINQ может менять момент выполнения, nullable-аннотации уточняют статический договор, но не заменяют проверки входа.

В примере null-conditional assignment пропускает правую часть полностью. Instance operator += у Counter изменяет объект, видимый через alias; это существенная семантика. Новый синтаксис считается освоенным, когда вы можете назвать сохраняемый контракт и проверить его.

Глава 25

25-A: используйте tuple из приоритета и последовательного номера; не полагайтесь на FIFO встроенной PriorityQueue при равных приоритетах. 25-B: задайте OrdinalIgnoreCase при построении индекса и снимка. AsReadOnly над исходным изменяемым контейнером не создаёт независимой версии. 25-C: словарь отвечает за Id, очередь — за выбор следующего. При ленивой отмене извлечение пропускает неактивные записи; не забывайте о накоплении таких записей и политике очистки.

BinarySearch требует того же правила порядка, что сортировка; позиция вставки равна ~result. GetOrAdd не гарантирует единственный вызов фабрики, поэтому фабрика должна допускать повтор.

Глава 26

26-A: EnumerateRunes считает scalar values, StringInfo — текстовые элементы; char — UTF-16 единицу. Используйте escape-записи, чтобы редактор не нормализовал исходный текст незаметно. 26-B: задайте Unspecified для местного показания и передайте TrainingZone явно. IsInvalidTime и IsAmbiguousTime проверяются до вычисления offset. 26-C: подставьте FixedClock, проверьте now == deadline отдельно от now < deadline. Для языка ресурсов нужен CurrentUICulture, для машинного формата — InvariantCulture.

DateTime.SpecifyKind меняет описание значения, но не переводит время. Нормализация Unicode не превращает все похожие буквы разных алфавитов в один символ.

Глава 27

27-A: CSV quoting и XML escaping относятся к синтаксису формата; проверьте фактическую строку после разбора. Количество строк файла не равно количеству CSV-записей. 27-B: сначала проверьте envelope, затем каждую RawExpense. Numeric JSON token и строка с неверной денежной точностью находятся на разных уровнях договора L03. 27-C: временный файл должен быть в том же каталоге, а рабочий файл нельзя удалять заранее. Для отмены передайте уже отменённый token и сравните старые байты.

ReadAsync может вернуть короткий блок; продолжайте до нуля. Source generation создаёт метаданные сериализатора, но не проверяет предметные инварианты конструктора за вас.

Глава 28

28-A: Content-Length может отсутствовать. Ограничивайте сумму реально прочитанных байтов, а response освобождайте даже при раннем отказе по заголовкам. 28-B: настройте один retry-handler и счётчик на сервере. 503→200 даёт два запроса, постоянный 404 — один. JSON-проверка происходит после транспортного pipeline. 28-C: сервер сначала отправляет заголовки, затем задерживает тело. Это выявляет ошибку, когда timeout покрывает только получение response. Для caller cancellation проверяйте token вызывающего кода.

Не заменяйте тест сервера обращением к публичному сайту: его задержки и ошибки не управляются тестом. Отмена не является rollback уже совершённой записи.

Глава 29

29-A: сравнивайте ReferenceEquals внутри одного scope и между scope. Singleton принадлежит контейнеру, а не обязательно всему процессу при наличии нескольких контейнеров. 29-B: appsettings лежит рядом с собранной программой. Переменная с префиксом CS_IMPORT_ подключается позже базовых источников и переопределяет ключ Import:MaxBytes. 29-C: IHostedService обычно живёт всё время host. Внедрите IServiceScopeFactory, создавайте scope на операцию, передавайте token остановки вниз и ожидайте завершения работы.

Не разрешайте scoped-сервис из root provider и не внедряйте его в singleton напрямую. Structured logging сохраняет значения полями, а scope добавляет общий контекст batch.

Глава 30 и L03

30-A: третий объект после нормализации совпадает с первым; шестой конфликтует по сумме. После первого запуска первые три становятся duplicates, ошибочные записи снова дают ошибки. 30-B: история Id и принятые данные находятся в одном checkpoint. Проверяйте его полностью до расчёта нового состояния; повреждение не должно создавать «пустое начальное состояние». 30-C: dry-run должен строить тот же план, но не вызывать фиксацию. Явно определите, берёт ли он блокировку для согласованного чтения существующего checkpoint.

Ошибка записи не должна сопровождаться stdout-отчётом об успехе. Потеря ответа после фиксации разрешается повторным импортом, а не догадкой по логам. Не удаляйте lock-файл после закрытия handle: другой процесс уже мог открыть его, и создание нового файла по тому же имени нарушило бы взаимное исключение.

Глава 31

31-A: break освобождает async-перечислитель; его finally выполняется и без полного обхода. 31-B: создавайте задачи через async-обёртку, которая перехватывает также синхронное исключение фабрики. Храните исход каждого URL; общий WhenAll ждёт завершения всех обёрток. 31-C: token нужен самой операции, включая чтение тела; WaitAsync отменяет только ожидание. После запроса отмены дождитесь задач и только потом освобождайте общий транспорт.

Обычный вызов async-метода выполняет синхронный префикс; await незавершённой операции возвращает управление. WhenAny не отменяет проигравших и не пробрасывает ошибку победителя вместо второго await. ConfigureAwait(false) относится к контексту продолжения, а не к защите общего состояния.

Глава 32

32-A: оба потока читают 0 до барьера и записывают 1 после него. Volatile не объединяет эти действия. 32-B: проверка balance >= amount и уменьшение баланса должны быть под одним приватным lock. Проверка допускает любого победителя, но ровно один успешный исход и баланс 30.00. 32-C: сначала обработайте перевод самому себе, затем захватите два lock в общем порядке Id. Проверка недостатка средств выполняется уже внутри защищённой секции.

Если WaitAsync отменён, разрешение не получено. Release требуется только после успешного входа. Callback GetOrAdd может исполниться несколько раз; побочный эффект требует другого договора. Один владелец изменяемого словаря в L04 избавляет workers от блокировок вокруг бизнес-решений.

Глава 33

33-A: каждый результат помещается в заранее закреплённый индекс; завершение итераций может быть любым. 33-B: создание P worker-задач ограничивает работу; bounded channel ограничивает ожидающие элементы. Упавший consumer должен отменить общий token, иначе producer может остаться в WriteAsync навсегда. 33-C: pending тоже является очередью. Возвращайте разрешение W после применения по порядку, а не после любого завершившегося worker; тогда медленная первая запись останавливает допуск новых.

Нормальный EOF означает completion с дочитыванием; отмена означает прекращение незавершённой работы. Закрывать results может координатор после всех workers, но не первый завершившийся worker.

Глава 34

34-A: одна benchmark-операция суммирует Count строк; Allocated относится ко всей этой операции. Не делайте вывод о фиксированном коэффициенте ускорения по ShortRun с широким Error. 34-B: inclusive включает дочерние вызовы, exclusive — только собственный кадр. Сэмплы потоков показывают также ожидания; доля стека не равна автоматически доле CPU. 34-C: сравнивайте одинаковую работу и отдельные новые checkpoint. Три запуска дают лишь небольшой опыт; разницу порядка нескольких процентов легко объяснить шумом, стартом процесса или файловым кэшем.

Глава 35 и L04

35-A: применение в исходном порядке сохраняет выбор первой корректной записи при конфликте. 35-B: сначала считайте исходный вызов, затем максимум две повторные попытки. Только явно временная ошибка повторяется; неправильная сумма сразу становится ошибкой строки. 35-C: token прекращения приёма и token принудительной отмены текущих работ имеют разные роли. Однофайловый готовый пример использует abort; daemon требует отдельного протокола drain.

Checkpoint меняется после полного завершения стадий. Сбой до замены оставляет старые байты; потеря ответа после замены разрешается повтором и дедупликацией. Очередь Channels не восстанавливается после падения, поэтому источник NDJSON должен оставаться доступен. Проверяйте не только исключение снаружи, но и active=0 после join: иначе фоновые задачи ещё работают.

Карта покрытия языка C#

Карта помогает проверять полноту программы. Номер означает запланированное место темы, а статус показывает наличие учебного текста. **Написаны главы 01–35**; строка «план» не означает, что тему уже можно изучить здесь. Сверка состава выполнена с официальным справочником и описанием C# 14.

Область	Где изучается	Состояние
Лексика, литералы, комментарии, имена, scope	02–03	Базовый материал написан
Числовые типы, bool/char/string, conversions, checked	03, 06	Базовый материал написан
Операции, precedence, short-circuit, биты и сдвиги	03	Введение написано
if/switch, циклы, break/continue/return, goto	04, 17	Написано
Методы, static, overload, local functions, recursion, params	05, 13, 24	Основы написаны
ref/out/in, ref return, ref readonly, scoped	05, 23	Написано
Null, `Nullable<T>`, аннотации и анализ	06, 13, 21, 24	Основы написаны
Массивы, indices/ranges, collection expressions	07, 23–25	Основы написаны
class, constructors, fields, properties, accessibility	08, 11–12, 24	Основы написаны
init, required, readonly, object initializer	08, 12	Введение написано
Исключения, filters, using и IDisposable	09, 20, 31	Основы написаны
Наследование, override/new, abstract, sealed, interfaces	11	Написано
struct/readonly struct, records, primary constructors, with	12, 24	Написано
object, boxing/unboxing, enum/Flags, tuples, equality	12, 20	Написано
Generics, constraints, variance, generic math	13	Написано; ref-ограничения углубляются в 23
delegate, lambda, closures, events	14	Написано
foreach pattern, IEnumerable, yield, iterator disposal	15, 31	Написано; есть проверки async-перечисления
LINQ method/query syntax, IQueryable	16	Написано; работа с БД далее в 42
Patterns, extension members, operators, indexers	17, 24	Написано
Assemblies, IL, metadata, CLR, JIT/AOT	01, 19	Написано; AOT-публикация далее
GC, finalization, managed/native lifetime	20	Написано; async-disposal и проверки в 31
Attributes, reflection, dynamic, anonymous types, expressions	21	Написано

Область	Где изучается	Состояние
namespaces, aliases, global/static using, extern alias, file types	02, 22	Написано
partial, preprocessor, XML docs, Roslyn и source generators	22	Написано; есть учебный хост с проверками
Span/Memory, ref struct, unsafe, fixed, stackalloc, pointers	23	Написано
Function pointers, calling conventions, inline arrays, sizeof	23, 55	Основы написаны; полный interop в 55
C# 13–14 и совместимость версий	24	Написано; конкурентность Lock проверена в 32
async/await, awaitables, Task/ValueTask, async streams	31	Написано и проверено
lock, memory model, volatile, Interlocked	32	Написано; Lock/Interlocked и гонки проверены
Interop, extern methods, marshaling, native ownership	55	План

Что находится за пределами синтаксиса

Работа с JSON, датами, HTTP, DI и журналами написана в 25–30 и проверяется в L03. Ограниченная конкурентность, Channels, BackgroundService и измерения написаны в 31–35 и проверяются в L04. Проверки, проектирование и инструменты — в 36–39; SQL и persistence — в 40–43; веб и сервисы — в 44–49; UI, игры и вычисления — в 50–54. Поставка и эксплуатация — в 55–60.

Таким образом, «знать язык» и «работать в C#-проекте» проверяются отдельно. Редкие средства, например COM или unsafe, нужны не каждой роли, но для них задан явный маршрут. Карта охватывает семейства возможностей; точные ограничения грамматики и комбинации модификаторов проверяются по языковой спецификации и компилятору.

Как дополнять карту

При добавлении новой главы обновите её статус и проверьте, что тема имеет объяснение, пример, самостоятельное упражнение и критерий проверки. Если материал только упомянут в списке, статус остаётся «план». Новая глава обязательно входит в book.json и общий PDF.

Источники и версии

База курса — C# 14 и .NET 10. Дата сверки: 22 октября 2026. Учебные объяснения и задания написаны самостоятельно; книги используются как дополнительное чтение. Содержимое папки Books не включается в PDF курса.

Официальные источники

Назначение	Источник
Синтаксис и семантика	C# language reference
Нормативная модель и feature specifications	C# specification
Версия языка	C# 14
Поддержка SDK/runtime	.NET support policy
Работа с проектами	.NET CLI
Стандартная библиотека	.NET API browser
Nullable-анализ	Nullable reference types
Сравнение строк	String best practices
Веб-разработка	ASP.NET Core
ORM	Entity Framework Core
Объектная модель	Интерфейсы, records
Generic-контракты	Ограничения типов
Отложенное выполнение	Yield, LINQ
Современный синтаксис	Patterns, extension members

Справочник языка обновляется и может упоминать preview-возможности; в курс их включают только с явной пометкой. Существование API в документации не подтверждает его доступность в выбранном TFM. Запускаемые примеры компилируются под net10.0 и LangVersion 14.0.

Книги в репозитории

Книга	Файл в Books	Для чего использовать
Adam Freeman, Pro ASP.NET Core 6, 9th edition, 2022	`Adam_Freeman_Pro_ASP_NET_Core_6_Develop_Cloud_Ready_Web_Applications.pdf`	HTTP pipeline, MVC, Razor Pages, Blazor; главы 44–51
Xiaodi Yan, Web API Development with ASP.NET Core 8, 2024	`Xiaodi_Yan_Web_API_Development_with_ASP_NET_Core_8_Learn_techniques.pdf`	Проектирование и проверка API; главы 44–49
Jeffrey Richter, CLR via C#, русское издание	`Мастер_класс_Джеффри_Рихтер_CLR_via_C#_Программирование_на_платформе.pdf`	Типы, CLR, ресурсы и конкурентность; главы 11–24, 31–32

В первых двух файлах сверены титульные страницы. У файла Рихтера начальные страницы представлены без извлекаемого текста; номер издания здесь не утверждается. Вместо ненадёжных номеров PDF-страниц ищите соответствующую тему в оглавлении своего издания. Для начала курса книги не

обязательны.

В старых книгах встречаются API и настройки эпохи .NET Framework, ASP.NET Core 6 или 8. Изучайте объяснение задачи, затем проверяйте современный способ по документации. Например, рекомендации о hosting model, nullable, пакетах и версиях SDK нельзя переносить механически.

Внешние библиотеки

Для блока 25–30 проверены стабильные версии из nuget.org, приведённые ниже. Транзитивный граф закреплён в packages.lock.json; воспроизведение использует locked restore. Примеры 01–24 по-прежнему не используют внешние NuGet-пакеты. CompilerLab использует Roslyn из выбранного SDK (проверен 10.0.402); это учебный хост, связанный с его версией.

Инструменты книги

PDF собирается из Markdown с помощью markdown-it-py и ReportLab. Кириллические TrueType-шрифты встраиваются в PDF. Проверка структуры использует pypdf. Основа структуры и сборщика адаптирована из соседнего курса SystemVerilog пользователя; учебный текст и примеры здесь относятся к C#.

Дополнение для глав 19–24

Тема	Первичный источник
Сборки и загрузка	Assemblies
Достижимость и поколения	GC fundamentals
Владение ресурсами	Dispose
Trimming	Подготовка библиотек
Генерация исходников	Roslyn cookbook
Время жизни буферов	Memory/Span guidelines
История совместимости	C# versions
Запуск одного исходника	File-based apps

В справочнике unsafe уже встречаются правила preview C# 15; примеры курса используют только C# 14. Поведение проверено компилятором и тестами, а возможность AOT-публикации отдельно не заявляется.

Источники и пакеты глав 25–30

Пакет / область	Проверенная версия	Официальный источник
CsvHelper	33.1.0	Документация
Microsoft.Extensions.Hosting	10.0.13	Generic Host
Microsoft.Extensions.Http.Resilience	10.11.0	HTTP resilience
System.CommandLine	2.0.13	Обзор API
Serilog.Extensions.Hosting	10.0.0	Репозиторий
Serilog.Sinks.Console	6.1.1	Репозиторий sink
Spectre.Console	0.57.2	Table

Пакет / область	Проверенная версия	Официальный источник
System.Text.Json	В составе .NET 10	Source generation
Unicode	В составе .NET 10	Character encoding
Generic math	В составе .NET 10	Числовые интерфейсы

Доступность конкретных версий сверена с официальным NuGet flat-container index, затем выполнены restore, сборка и контрактные проверки. Номера пакетов Host и Resilience имеют разные циклы выпуска. Полный список прямых зависимостей и команды — в примерах интеграций.

Источники и измерения глав 31–35

Область	Первичный источник
Task и композиция	Async, WhenAll
Контексты продолжений	ConfigureAwait FAQ
Синхронизация	lock, модель памяти
Очереди и потоки данных	Channels, Dataflow
Жизненный цикл	BackgroundService в .NET 10, scope
BenchmarkDotNet 0.15.8	Руководство
dotnet diagnostics 10.0.750501	counters, trace, dump

Пакеты закреплены в csproj/lock-файлах; диагностические инструменты устанавливаются локально по точной версии. BenchmarkDotNet, trace и counters действительно запускались; для dump проверены установка и версия, снятие/анализ дампа не выполнялись. Числа измерений относятся только к описанной среде и нагрузке. Сохранённые результаты связаны с главой 34.

Программа практикума C# и .NET

Программа ведёт от первой строки кода к самостоятельной разработке и сопровождению приложения. Базовая среда — **C# 14, .NET 10 LTS**, проверка актуальности: **22 октября 2026**. Версия языка и поддержка платформы проверяются отдельно.

Как читать программу

Ниже сначала перечислен весь маршрут, включая темы будущих выпусков. **Наполнены главы 01–35**; главы 36–60 пока имеют статус **план**. Их наличие здесь не означает готовность учебного текста. Оглавление написанных глав и порядок наполнения показывают фактическое состояние.

Общее ядро: 01–49 и 55–60; прикладная ветка: одна или несколько из 50–54. Знание синтаксиса само по себе не заменяет SQL, HTTP, тестирование и эксплуатацию: эти темы включены в маршрут. Для первого трудоустройства нужно глубоко пройти выбранную ветку, а остальные знать на уровне назначения и ограничений.

Оценки времени включают самостоятельное решение и исправления, а не только чтение. Весь маршрут с одной веткой потребует нескольких сотен часов. При опыте другого языка разрешается проходить входные проверки вместо повторного чтения; пропускать практику и контрольные работы нельзя. Скорость определяется прохождением проверок.

Два обязательных способа изучения

1. **Репозиторий как самостоятельный учебник:** тематические каталоги, оглавление, переходы между главами, ссылки на задания и запускаемые примеры. Каждую главу можно читать отдельно в Markdown.
2. **Весь учебный материал как один PDF:** все написанные главы, лабораторные и приложения собираются в `dist/csharp.pdf` по единому порядку `book.json`. По мере наполнения в эту же сборку войдут все 60 глав. PDF содержит оглавление, закладки, номера страниц, кириллицу, читаемый код, таблицы и внутренние ссылки.

Исходник текста один — Markdown в репозитории; отдельная вручную поддерживаемая версия PDF не создаётся. Команда сборки: `python tools/book.py build`. Глава считается готовой только после включения в оглавление, проверки ссылок, сборки общего PDF и проверки читаемости её страниц. Лабораторные условия тоже входят в PDF; исходный код проектов остаётся запускаемым в репозитории. Инструкция — в README.

Состав каждой главы

1. Результат и необходимые предыдущие знания.
2. Объяснение модели, затем синтаксис и небольшой разобранный пример.
3. Типичные ошибки, граничные случаи и способы диагностики.
4. Практика A — воспроизведение; B — самостоятельное решение; C — изменение требований.
5. Проверяемые условия готовности, вопросы и подсказки отдельно от задания.
6. Ссылки на первоисточники и связь со сквозным проектом.

Задачи имеют обозначения вида `03-A`; лабораторные — `L01`. В начале допускаются ручные сценарии; после главы 36 для нового поведения обязательны автоматические проверки. Готовый пример открывают после самостоятельной попытки. Ошибочный код всегда явно помечается как упражнение или

контрпример.

I. Первая самостоятельная программа

01–10; вход без опыта программирования; ориентир 30–45 часов.

01. C# и .NET, рабочая среда

Статус: написано.

1. Язык, компилятор Roslyn, IL, CLR, JIT и библиотеки.
2. SDK и runtime.
3. .NET и .NET Framework.
4. установка SDK и редактора.
5. CLI, csproj, slnx, TFM.
6. build, run, restore, publish.

Практический результат: Собрать и запустить приложение из терминала и отладчика.

02. Синтаксис и диагностика

Статус: написано.

1. Токены, идентификаторы, ключевые слова, литералы и комментарии.
2. выражение и инструкция.
3. блоки и область видимости.
4. top-level statements и Main.
5. using и пространства имён.
6. ошибки компиляции, исключения и логические ошибки.
7. breakpoint, step, watch, call stack.

Практический результат: Исправить три разных вида ошибок и объяснить каждый диагноз.

03. Типы, числа и операции

Статус: написано.

1. bool, char, целые знаковые и беззнаковые типы, nint/nuint, float/double/decimal.
2. литералы и суффиксы.
3. var, const, default.
4. преобразования.
5. checked/unchecked.
6. арифметика, сравнения, логика, биты и сдвиги.
7. приоритет и порядок вычисления.

Практический результат: Рассчитать стоимость заказа и проверить округление и границы.

04. Ветвления и циклы

Статус: написано.

1. if/else.
2. switch statement и expression.
3. for, while, do, foreach.
4. break, continue, return.
5. короткое замыкание.
6. интервалы и инвариант цикла.
7. EOF.
8. трассировка и граничные случаи.
9. область применения goto.

Практический результат: Сделать интерактивный сумматор с устойчивым вводом.

05. Методы и декомпозиция

Статус: написано.

1. Параметры и аргументы.
2. возвращаемые значения и void.
3. static.
4. перегрузки.
5. именованные и необязательные аргументы.
6. params.
7. локальные функции.
8. передача по значению, ref/out/in.
9. рекурсия и стек.
10. чистые вычисления и побочные эффекты.

Практический результат: Разделить расчёт, разбор данных и вывод; реализовать Try-метод.

06. Строки, null и культура

Статус: написано.

1. string и UTF-16.
2. char, Rune и текстовый элемент.
3. неизменяемость.
4. escape, verbatim, raw и interpolation.
5. StringBuilder.
6. сравнение Ordinal и культурное сравнение.

7. Parse/TryParse, CultureInfo.
8. nullable value/reference types, ?. и ??, ??= и !.

Практический результат: Определить строгий формат суммы и корректно обработать пустой ввод.

07. Массивы и базовые коллекции

Статус: написано.

1. Одномерные, прямоугольные и ступенчатые массивы.
2. индексы и диапазоны.
3. копирование и общие ссылки.
4. List, Dictionary, HashSet, Queue, Stack.
5. коллекционные выражения.
6. TryGetValue.
7. сложность O(n) и O(1).
8. перебор и изменение.

Практический результат: Сгруппировать расходы по категориям без LINQ.

08. Классы и инварианты

Статус: написано.

1. class, объект и ссылка.
2. поля, свойства, методы и конструкторы.
3. this.
4. public/private/internal.
5. readonly и static.
6. авто-свойства, get/set/init.
7. object initializer.
8. required.
9. namespace и файл.
10. инкапсуляция и композиция.

Практический результат: Создать объект расхода, который нельзя оставить в недопустимом состоянии.

09. Ошибки, ресурсы и первые файлы

Статус: написано.

1. Exception.
2. try/catch/finally, throw и throw ex.
3. catch filters.
4. Try-паттерн.

5. IDisposable и using.
6. Path, File, StreamReader/Writer, UTF-8.
7. относительные пути и рабочая директория.
8. обработка I/O на границе приложения.

Практический результат: Прочитать текстовый файл, сообщить о повреждённой строке и закрыть ресурс.

10. Практикум ExpenseLedger CLI

Статус: написано.

1. Контракт входного файла.
2. разбор строк.
3. модель предметной области.
4. список расходов и агрегаты.
5. протокол ошибок и exit codes.
6. детерминированный вывод.
7. ручные проверки, автоматические сценарии и ревью.

Практический результат: Поставить консольный отчёт по расходам с проверяемыми критериями приёмки.

II. Система типов и выразительный код

11–18; нужны 01–10; ориентир 35–50 часов.

11. Наследование и интерфейсы

Статус: написано.

1. Базовый класс, base, abstract, virtual/override, sealed, new hiding.
2. полиморфизм.
3. интерфейсы и явная реализация.
4. default interface members.
5. static abstract/virtual members.
6. ковариантный результат.
7. композиция и подстановка.

Практический результат: Заменять хранилище расходов через интерфейс без изменения расчёта.

12. Структуры, записи и равенство

Статус: написано.

1. struct, readonly struct, record class/struct.
2. primary constructors.
3. value/reference semantics.

4. boxing/unboxing.
5. with и поверхностное копирование.
6. Equals, GetHashCode, IEquatable, операторы.
7. enum и Flags.
8. ValueTuple и деконструкция.
9. ключи словаря.

Практический результат: Создать тип Money, определить равенство и проверить ключи коллекций.

13. Обобщения и контракты типов

Статус: написано.

1. Generic типы и методы.
2. вывод типов.
3. where class/struct/notnull/unmanaged/new/base/interface/default.
4. allows ref struct.
5. variance in/out.
6. generic static state.
7. ограничения и static abstract для generic math.
8. nullable T.
9. обобщения и отражение.

Практический результат: Реализовать `Result<T>` и типобезопасный алгоритм без dynamic.

14. Делегаты, лямбды и события

Статус: написано.

1. delegate, Action, Func, Predicate.
2. method group.
3. multicast и исключения.
4. анонимные функции.
5. замыкания и lifetime.
6. static lambdas.
7. event, подписка и отписка.
8. `EventHandler<T>`.
9. утечки через события.
10. async void в обработчиках.

Практический результат: Создать уведомления об изменениях и доказать освобождение подписчика.

15. Перечисление и итераторы

Статус: написано.

1. IEnumerable/IEnumerator и generic варианты.
2. foreach pattern.
3. yield return/break.
4. отложенное выполнение.
5. повторное перечисление.
6. освобождение итератора.
7. lazy и eager.
8. бесконечные последовательности.
9. основы IAsyncEnumerable.

Практический результат: Сделать потоковый импорт большого файла без загрузки всего содержимого.

16. LINQ в памяти и в запросах

Статус: написано.

1. Where/Select/SelectMany.
2. OrderBy/ThenBy.
3. GroupBy, Join/GroupJoin.
4. Any/All, First/Single и Default.
5. Aggregate, Count, Sum.
6. DistinctBy, Chunk.
7. query syntax.
8. materialization.
9. IEnumerable против IQueryable.
10. expression tree и перевод запроса.

Практический результат: Написать отчёты и найти ошибку из-за повторного перечисления.

17. Шаблоны и расширение синтаксиса

Статус: написано.

1. is/as и pattern matching.
2. type, constant, property, positional, relational, logical, list и slice patterns.
3. guards и исчерпываемость.
4. extension methods/members.
5. перегрузка операторов и преобразований.
6. индексаторы.

7. deconstruct.
8. target-typed new.

Практический результат: Реализовать классификацию операций без хрупкой цепочки if.

18. Практикум библиотеки предметной области

Статус: написано.

1. Разделение CLI и class library.
2. агрегаты и правила.
3. интерфейсы на границах.
4. равенство.
5. коллекции только для чтения.
6. документирование API.
7. совместимость.
8. ошибки как результат и исключения.

Практический результат: Вынести ExpenseLedger.Core и подтвердить независимость от UI и диска.

III. Полная карта языка и CLR

19–24; нужны 11–18; ориентир 25–40 часов.

19. Сборки и модель исполнения

Статус: написано.

1. PE, IL и metadata.
2. тип, assembly, module, namespace.
3. CTS/CLS.
4. загрузка типов и статическая инициализация.
5. JIT, tiered compilation, ReadyToRun, Native AOT.
6. public/internal и InternalsVisibleTo.
7. сборки и версии.

Практический результат: Проследить путь вызова от исходника до IL и машинного кода.

20. Память и время жизни

Статус: написано.

1. Стек вызовов, managed heap и ссылки.
2. копирование значений.
3. GC roots, поколения, LOH и pinning.
4. finalizer и Dispose pattern.

5. WeakReference, SafeHandle.
6. IDisposable/IAsyncDisposable.
7. managed leaks.
8. владение ресурсом.

Практический результат: Найти удерживаемый объект и исправить время жизни ресурса.

21. Атрибуты, reflection и dynamic

Статус: написано.

1. Атрибуты и targets.
2. typeof/GetType/nameof.
3. metadata inspection.
4. Activator, generics и reflection.
5. dynamic и runtime binder.
6. anonymous types.
7. expression trees.
8. trimming annotations.
9. рефлексия при AOT.
10. CallerMemberName/CallerArgumentExpression.

Практический результат: Сделать валидатор по метаданным и описать ограничения AOT.

22. Структура исходников и инструменты компилятора

Статус: написано.

1. file-scoped namespaces и file-local types.
2. global/static/alias using и extern alias.
3. partial типы, методы, свойства, события, конструкторы.
4. #if/#define/#nullable/#pragma/#line/#error/#warning/#region.
5. XML docs.
6. analyzers, Roslyn syntax/semantic model, incremental source generators.

Практический результат: Добавить анализатор правила и генератор кода с тестами.

23. Ссылки, span и unsafe

Статус: написано.

1. ref locals/returns, ref readonly, scoped.
2. ref struct и escape analysis.
3. Span/ReadOnlySpan, Memory/ReadOnlyMemory.
4. stackalloc, ArrayPool и ownership.

5. pointers, fixed, sizeof, function pointers, unmanaged calling conventions.
6. inline arrays.
7. MemoryMarshal и границы безопасности.

Практический результат: Сравнить выделения памяти в двух парсерах и проверить корректность.

24. Версии языка и чтение старого кода

Статус: написано.

1. История возможностей C# 1–14.
2. LangVersion, TFM и SDK.
3. nullable migration.
4. records, required, collection expressions и primary constructors.
5. C# 13 params collections, Lock, ref struct interfaces.
6. C# 14 field, extension blocks, null-conditional assignment, unbound nameof, span conversions, lambda modifiers, partial members, compound operators.
7. file-based apps.
8. stable против preview.

Практический результат: Перенести старый фрагмент на современный стиль, сохранив поведение.

IV. Библиотеки .NET и внешние данные

25–30; нужны 11–18, обзор 19–20; ориентир 25–40 часов.

25. Коллекции и алгоритмы BCL

Статус: написано.

1. ICollection/IList/IReadOnlyList.
2. Dictionary, SortedDictionary, SortedSet, PriorityQueue.
3. Immutable и Frozen.
4. Concurrent collections.
5. comparer и equality comparer.
6. поиск, сортировка, двоичный поиск.
7. capacity.
8. BigInteger и generic math.

Практический результат: Выбрать структуры для очереди работ и доказать порядок обработки.

26. Текст, время и регулярные выражения

Статус: написано.

1. Encoding и Unicode normalization.
2. Rune/StringInfo.

3. Regex, timeout, NonBacktracking и GeneratedRegex.
4. DateTime/DateTimeOffset/DateOnly/TimeOnly/TimeSpan.
5. TimeZoneInfo и DST.
6. TimeProvider.
7. CultureInfo.
8. ресурсы локализации.
9. Guid, Random и криптографический RNG.

Практический результат: Импортировать даты с часовыми поясами и протестировать переход времени.

27. Файлы, потоки и сериализация

Статус: написано.

1. File/Directory/Path, Stream, BufferedStream, async I/O.
2. JSON и System.Text.Json, converters и source generation.
3. XML и LINQ to XML.
4. CSV и CsvHelper.
5. ZIP.
6. format versions, размер входа, повреждение данных, замена файла.
7. запрет небезопасной десериализации.

Практический результат: Сделать версионируемый импорт/экспорт с проверкой испорченных данных.

28. HTTP и сетевые клиенты

Статус: написано.

1. HTTP методы, статусы и заголовки.
2. HttpClient/handler lifetime.
3. IHttpClientFactory.
4. JSON, streams, timeout/cancellation.
5. retries/backoff/jitter и идемпотентность.
6. Microsoft.Extensions.Http.Resilience и Polly.
7. sockets, DNS, TLS, WebSocket обзор.

Практический результат: Написать клиент с отменой, ограниченным повтором и тестовым сервером.

29. Host, DI, настройки и журналы

Статус: написано.

1. Generic Host.
2. dependency injection.
3. transient/scoped/singleton.

4. captive dependency.
5. Options и validation.
6. appsettings, environment variables и user secrets.
7. ILogger, structured logging и Serilog.
8. scope.
9. startup/shutdown.

Практический результат: Запустить импорт как службу с корректным завершением и диагностикой.

30. Практикум утилиты импорта

Статус: написано.

1. Аргументы CLI и System.CommandLine.
2. Spectre.Console как необязательный UI.
3. конфигурация.
4. файлы и HTTP.
5. checkpoint.
6. обработка частично ошибочного набора.
7. отчёт об импорте.
8. повторный запуск.

Практический результат: Поставить воспроизводимый импортёр с отдельными ошибками каждой записи.

V. Асинхронность, потоки и производительность

31–35; нужны 14–15, 20 и 27–29; ориентир 30–45 часов.

31. Task и async await

Статус: написано.

1. I/O-bound и CPU-bound.
2. `Task`/`Task<T>`, state machine, awaitable pattern.
3. исключения до/после await.
4. WhenAll/WhenAny.
5. CancellationTokenSource и deadline.
6. async all the way.
7. SynchronizationContext, ConfigureAwait.
8. deadlock .Result.
9. ValueTask.
10. async streams и await using.

Практический результат: Скачать набор данных с отменой и сохранением всех ошибок.

32. Потоки и синхронизация

Статус: написано.

1. Thread/ThreadPool.
2. shared mutable state.
3. race и memory visibility.
4. lock и System.Threading.Lock.
5. Monitor, Interlocked, Volatile, SemaphoreSlim, Mutex.
6. deadlock и порядок захвата.
7. thread safety и reentrancy.
8. AsyncLocal.

Практический результат: Воспроизвести потерю обновления и исправить её с объяснением.

33. Параллельные конвейеры

Статус: написано.

1. Parallel.ForEachAsync и PLINQ.
2. bounded concurrency.
3. Channels и backpressure.
4. producer/consumer.
5. TPL Dataflow обзор.
6. cancellation propagation.
7. completion и graceful shutdown.
8. IAsyncEnumerable.
9. периодические задачи.

Практический результат: Создать ограниченный конвейер импорта и доказать отсутствие потери записей.

34. Измерение и оптимизация

Статус: написано.

1. Release и warmup.
2. BenchmarkDotNet.
3. время, allocations, throughput и percentiles.
4. dotnet-counters/trace/dump.
5. GC и threadpool starvation.
6. boxing, closure, LINQ hot path.
7. pools, spans, SIMD обзор.

8. измерение до/после.

Практический результат: Найти узкое место по профилю и подтвердить улучшение без изменения результата.

35. Практикум фонового обработчика

Статус: написано.

1. BackgroundService.
2. конкурентные задания.
3. очередь.
4. лимиты.
5. отмена.
6. retries и poison message.
7. поведение при падении.
8. нагрузочный эксперимент.
9. диагностика.

Практический результат: Обработать поток операций с лимитом памяти и управляемым завершением.

VI. Инженерная работа и качество

36–39; нужны 18 и 31; ориентир 25–40 часов.

36. Автоматические тесты

Статус: план.

1. xUnit как основной пример.
2. NUnit/MSTest как альтернативы.
3. Arrange/Act/Assert.
4. parameterized tests.
5. unit/integration/contract/e2e.
6. fakes и NSubstitute.
7. FsCheck обзор.
8. coverage и mutation testing.
9. тестирование времени и async.
10. Testcontainers.

Практический результат: Проверить бизнес-правила, сбои сети и реальные ограничения БД.

37. Проектирование и изменение кода

Статус: план.

1. Cohesion/coupling, SOLID с контрпримерами.

2. composition.
3. DI boundaries.
4. layered architecture и vertical slices.
5. Strategy, Factory, Decorator, Adapter, Observer.
6. CQRS по необходимости.
7. refactoring.
8. API contracts, errors, immutability.
9. code review.

Практический результат: Изменить бизнес-правило без дублирования и большого каскада изменений.

38. Git, сборка и NuGet

Статус: план.

1. Branch, commit, diff, merge и PR.
2. csproj/MSBuild/Directory.Build.props.
3. NuGet и central package management.
4. lock file, restore.
5. transitive dependencies и аудит.
6. analyzers/.editorconfig/dotnet format.
7. SDK pinning.
8. CI.
9. pack и SemVer.

Практический результат: Собрать библиотеку и проверить её потребление из другого проекта.

39. Практикум сопровождения

Статус: план.

1. Воспроизведение бага.
2. минимальный failing test.
3. отладчик и логи.
4. совместимость.
5. миграция API.
6. README и ADR.
7. review checklist.
8. измеримое определение готовности.

Практический результат: Принять чужой код, исправить дефект и подготовить проверяемое изменение.

VII. Данные и базы данных

40–43; нужны 16, 27, 31 и 36; ориентир 25–40 часов.

40. SQL и реляционная модель

Статус: план.

1. Таблицы, ключи, constraints, NULL и трёхзначная логика.
2. SELECT/JOIN/GROUP BY.
3. индексы и query plan.
4. normalization.
5. transactions, isolation и concurrency.
6. SQLite и PostgreSQL/SQL Server.
7. migrations.

Практический результат: Спроектировать схему расходов и обосновать индексы.

41. ADO.NET и Dapper

Статус: план.

1. Connection/command/reader и pooling.
2. parameters.
3. transactions.
4. async и cancellation.
5. mapping.
6. Dapper Query/Execute.
7. SQL injection.
8. streaming vs buffering.
9. provider-specific types.

Практический результат: Сделать параметризованный репозиторий и проверить rollback.

42. Entity Framework Core

Статус: план.

1. DbContext lifetime.
2. entities и DTO.
3. mapping, navigation и relationships.
4. tracking/AsNoTracking.
5. projection.
6. eager/explicit/lazy loading и N+1.
7. LINQ translation.

8. migrations.
9. concurrency tokens.
10. transactions и bulk operations.

Практический результат: Заменить файловое хранилище БД и устранить N+1.

43. Практикум надёжного хранения

Статус: план.

1. Unique constraints.
2. optimistic concurrency.
3. keyset pagination.
4. batch import.
5. migrations в CI и развертывании.
6. Testcontainers.
7. backup/restore.
8. query logging.
9. идемпотентность.

Практический результат: Доказать отсутствие дублей и потери обновлений при конкурирующих запросах.

VIII. ASP.NET Core и сервисы

44–49; нужны 28–29, 31, 36 и 40–43; ориентир 40–65 часов.

44. HTTP API на ASP.NET Core

Статус: план.

1. Kestrel и request pipeline.
2. middleware order.
3. routing.
4. minimal APIs и controllers.
5. binding/validation.
6. DTO.
7. ProblemDetails.
8. status codes.
9. OpenAPI.
10. pagination и versioning.

Практический результат: Опубликовать локальный API с проверяемым HTTP-контрактом.

45. MVC, Razor Pages и браузер

Статус: план.

1. MVC и server rendering.
2. Razor syntax.
3. forms, model binding и validation.
4. layout/partials.
5. cookies, sessions, static assets.
6. HTML/CSS/JS минимум.
7. antiforgery и XSS.
8. различие API и браузерного приложения.

Практический результат: Создать страницу списка и форму редактирования с обработкой ошибок.

46. Идентификация и защита API

Статус: план.

1. Authentication/authorization.
2. Identity.
3. cookies, bearer, OAuth2/OIDC.
4. claims, roles, policies и ownership.
5. password hashing через стандартные средства.
6. CORS/CSRF/XSS.
7. HTTPS.
8. rate limiting.
9. секреты и журналирование.
10. threat modeling.

Практический результат: Добавить пользователей и доказать изоляцию их данных негативными тестами.

47. Кэш, фоновые задачи и наблюдаемость

Статус: план.

1. IMemoryCache/HybridCache, distributed cache и Redis.
2. invalidation и stampede.
3. BackgroundService, Quartz/Hangfire обзор.
4. health checks.
5. structured logs.
6. metrics/tracing и OpenTelemetry.
7. correlation.

8. service configuration.

Практический результат: Добавить кэш с корректной инвалидацией и найти медленный запрос по trace.

48. Связь между сервисами

Статус: план.

1. gRPC и Protobuf.
2. SignalR.
3. очереди и RabbitMQ/MassTransit обзор.
4. delivery semantics.
5. outbox/inbox.
6. duplicate handling.
7. retry storms.
8. idempotency key.
9. eventual consistency.
10. modular monolith и границы микросервисов.

Практический результат: Обработать повторно доставленное сообщение без повторного эффекта.

49. Практикум ExpenseLedger API

Статус: план.

1. API, БД, identity, импорт и фоновые задачи.
2. integration tests через WebApplicationFactory.
3. OpenAPI.
4. observability.
5. load test.
6. отказ внешнего сервиса.
7. инструкции запуска.

Практический результат: Передать сервис другому разработчику с тестами и сценарием восстановления.

IX. Прикладные специализации

50–54; выбирается одна ветка после общего ядра; ориентир 25–60 часов на ветку.

50. Windows desktop и кроссплатформенный desktop

Статус: план.

1. WinForms и WPF.
2. XAML, data binding, commands, INotifyPropertyChanged.
3. MVVM и CommunityToolkit.Mvvm.

4. dispatcher/UI thread.
5. async commands.
6. Avalonia обзор.
7. installer и updates.
8. тестируемая view model.

Практический результат: Создать desktop-клиент API с отменой запроса и состояниями загрузки.

51. Blazor

Статус: план.

1. Components, parameters, lifecycle, binding.
2. routing и forms.
3. render modes, server/WebAssembly.
4. DI scopes.
5. JS interop.
6. authentication state.
7. API client.
8. bUnit.
9. UI loading/error states.

Практический результат: Создать веб-клиент с компонентами и тестами пользовательских состояний.

52. Мобильные приложения и .NET MAUI

Статус: план.

1. Workloads и ограничения платформ.
2. XAML/MVVM.
3. navigation, permissions, lifecycle.
4. offline storage.
5. connectivity.
6. async UI.
7. packaging и signing.
8. платформенные интеграции.

Практический результат: Сделать мобильный журнал с локальной очередью синхронизации.

53. C# в игровых движках

Статус: план.

1. Unity и Godot .NET как разные среды.
2. доступные версии языка/runtime.

3. сцены, компоненты и lifecycle.
4. update loop и delta time.
5. события.
6. coroutine против Task.
7. allocations/GC на кадр.
8. serialization.
9. тестируемая игровая логика.

Практический результат: Реализовать инвентарь и сохранение без привязки правил к сцене.

54. Вычисления и обработка данных

Статус: план.

1. System.Numerics, vectors и SIMD.
2. MathNet.Numerics.
3. ML.NET и ONNX Runtime обзор.
4. tensors и batching.
5. CSV/JSON pipelines.
6. точность и воспроизводимость.
7. границы C# и Python.

Практический результат: Построить воспроизводимый анализ расходов и измерить скорость обработки.

X. Интеграция, поставка и итоговая работа

55–60; нужны общее ядро и выбранная ветка; ориентир 30–50 часов.

55. Interop и специальные среды

Статус: план.

1. P/Invoke и LibraryImport.
2. marshaling, ABI, calling convention.
3. SafeHandle и native ownership.
4. COM и Windows APIs.
5. Native AOT и trimming.
6. plugins/AssemblyLoadContext и unload.
7. process boundary.
8. sandbox misconceptions.

Практический результат: Вызвать нативную функцию и корректно освободить принадлежащий ей ресурс.

56. Существующие проекты на .NET Framework

Статус: план.

1. Framework против современного .NET.
2. packages.config, старый csproj, binding redirects и GAC.
3. AppDomain.
4. WCF, Web Forms, EF6 и ASP.NET MVC5 обзор.
5. Windows-only зависимости.
6. .NET Standard.
7. постепенная миграция и characterization tests.

Практический результат: Составить и проверить план миграции отдельной библиотеки.

57. Публикация и CI/CD

Статус: план.

1. Framework-dependent/self-contained.
2. RID.
3. single file, trimming/AOT.
4. Docker multi-stage.
5. Linux и Windows differences.
6. environment config и secrets.
7. CI test/build/publish.
8. migration strategy.
9. health/readiness.
10. rollback.

Практический результат: Развернуть приложение в чистом окружении по одной инструкции.

58. Эксплуатация и разбор сбоев

Статус: план.

1. SLO и пользовательские симптомы.
2. logs/metrics/traces.
3. CPU, memory, dump и threadpool.
4. timeout/cancellation cascade.
5. graceful shutdown.
6. backup/restore drill.
7. dependency outages.
8. incident report и исправляющий тест.

Практический результат: Воспроизвести сбой, локализовать причину и восстановить работу.

59. Итоговый проект

Статус: план.

1. Требования, архитектурное решение и ADR.
2. API или UI выбранной ветки.
3. БД.
4. импорт/интеграция.
5. auth.
6. фоновые задачи.
7. CI.
8. метрики.
9. тесты.
10. документация.
11. нагрузка.
12. review и передача.

Практический результат: Сдать ExpenseLedger v3 или равный по сложности проект по рубрике.

60. Проверка знаний и дальнейший маршрут

Статус: план.

1. Карточки по всему языку.
2. чтение незнакомого кода.
3. объяснение времени жизни и асинхронности.
4. поиск по официальной документации.
5. диагностика багов.
6. оценка компромиссов.
7. собеседование и разбор пробелов.
8. план углубления.

Практический результат: Защитить решения и самостоятельно выполнить изменение требований.

Сквозной проект и контрольные точки

Этап	Главы	Что предъявить
L01	01–10	CLI читает файл, считает отчёт и корректно отклоняет ошибочные данные
L02	11–18	Библиотека предметной области с независимыми правилами и контрактами

Этап	Главы	Что предъявить
L03	19–30	Импорт/экспорт, настройки, HTTP-клиент и управляемые ресурсы
L04	31–35	Ограниченный асинхронный конвейер с отменой и измерениями
L05	36–43	Тесты и настоящее хранилище с транзакциями и контролем конкуренции
L06	44–49	API с изоляцией пользователей, интеграционными тестами и диагностикой
L07	50–58	Клиент выбранной ветки и воспроизводимая поставка
L08	59–60	Самостоятельный проект, разбор сбоя и защита изменения требований

Каждая следующая работа сохраняет уже реализованные правила и добавляет новую ответственность. Сложность растёт от отдельной функции к работе с состоянием, затем к I/O, конкуренции и отказам внешних систем.

Карта библиотек

Это рабочий набор для курса, а не рейтинг. Сначала изучается стандартный API и задача, затем сторонний инструмент. Версию каждого внешнего пакета нужно фиксировать при написании и проверке соответствующей главы.

Задача	Основной инструмент	Сопоставление или дополнительный обзор	Главы
Данные в памяти	BCL Collections, LINQ	Immutable, Frozen	07, 15–16, 25
Форматы	System.Text.Json, XML	CsvHelper	27
HTTP и устойчивость	HttpClient, Microsoft.Extensions.Http.Resilience	Polly	28
Сборка приложения	Microsoft.Extensions.Hosting/DependencyInjection/Options/Logging	Serilog	29
CLI	System.CommandLine	Spectre.Console	30
Конкуренция	Task, Channels	TPL Dataflow	31–35
Тесты	xUnit, WebApplicationFactory	NUnit, MSTest, NSubstitute, FsCheck, Testcontainers	36, 43, 49
Измерения	BenchmarkDotNet, dotnet diagnostics	OpenTelemetry	34, 47, 58
Доступ к БД	EF Core, ADO.NET	Dapper	40–43
Веб	ASP.NET Core, Identity, OpenAPI	FluentValidation как вариант валидации	44–49
Интеграция	gRPC, SignalR	Redis, RabbitMQ, MassTransit, Quartz, Hangfire	47–48
UI	WPF/WinForms, Blazor, MAUI	Avalonia, CommunityToolkit.Mvvm, bUnit	50–52

Задача	Основной инструмент	Сопоставление или дополнительный обзор	Главы
Игры	Unity или Godot .NET	API конкретного движка	53
Вычисления	System.Numerics	MathNet.Numerics, ML.NET, ONNX Runtime	54

Критерии завершения всего курса

- Учащийся объясняет семантику значений и ссылок, null, обобщений, LINQ, исключений, async и освобождения ресурсов.
- Пишет приложение с внешними данными и выбранным UI/API, самостоятельно находит нужные API в документации.
- Проверяет корректность, границы, отмену, конкуренцию и отказ зависимостей; умеет читать чужие тесты.
- Пользуется Git, отладчиком, профилировщиком, пакетами и CI; воспроизводит сборку на чистой машине.
- Обосновывает модель БД, границы ответственности и решения по безопасности и производительности.
- Передаёт работоспособный проект с инструкцией, тестами, диагностикой и описанием ограничений.

Полнота языка контролируется картой покрытия. Работа во всех существующих C#-средах требует знания их собственных API; курс даёт общее ядро и маршруты специализации.